University of Warsaw
Faculty of Economic Sciences

Krzysztof Wojdalski
Nr albumu: 310284

Microstructure-Aware Reinforcement Learning for High-Frequency Trading

Praca magisterska
na kierunku QUANTITATIVE FINANCE

The thesis written under the supervision of
dr Pawel Sakowski

September 1, 2026 06:10 UTC

Data Podpis opiekuna

Statement of the Supervisor on Submission of the Thesis

I hereby certify that the thesis submitted has been prepared under my supervision and I declare that it satisfies the requirements of submission in the proceedings for the award of a degree.

Date Signature of the Supervisor:

Oświadczenie autora (autorów) pracy

Data Podpis autora (autorów) pracy

Statement of the Author(s) on Submission of the Thesis

Aware of legal liability I certify that the thesis submitted has been prepared by myself and does not include information gathered contrary to the law.

I also declare that the thesis submitted has not been the subject of proceedings resulting in the award of a university degree.

Furthermore I certify that the submitted version of the thesis is identical with its attached electronic version.

Date Signature of the Author(s) of the thesis

Abstract

This thesis studies whether a deep reinforcement learning agent using order-book-derived microstructure features can produce robust, risk-adjusted trading performance in a high-frequency single-asset setting. It targets continuous portfolio exposure control under practical constraints — execution modeling, transaction cost sensitivity, and evaluation discipline.

A Twin Delayed DDPG (TD3) agent trains on tick-level order-book snapshots from six Nasdaq-listed equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO) over three consecutive trading sessions in February 2026, evaluated against benchmark strategies under explicit simulation assumptions. Under frictionless mid-price execution it achieves positive held-out returns with low drawdown and a high profit factor, but this edge is bounded by transaction cost: the microstructure signal it trades on cannot exceed the bid–ask half-spread, and the return turns negative once a fee of a fraction of a basis point is charged. A microstructure-aware state built from order flow, spread regime, and book shape improves policy behavior over simpler snapshot-based variants.

The main contribution is an end-to-end, reproducible pipeline linking LOB feature engineering, Differential Sharpe Ratio reward design, TD3 training, and rigorous out-of-sample evaluation, with an explicit account of the simulation assumptions separating these findings from deployable performance claims.

Key words

Reinforcement Learning, High-Frequency Trading, Order Book, Single-Asset Trading, TD3

(*Słowa kluczowe: Uczenie ze wzmocnieniem, Handel wysokiej częstotliwości, Księga zleceń, Handel pojedynczym aktywem, TD3*)

Area of study (codes according to Erasmus Subject Area Codes List)

14.3 Economics

Thematic classification

G11, G17, C63

Title of the thesis in Polish

Uczenie ze wzmocnieniem uwzględniające mikrostrukturę rynku w handlu wysokiej częstotliwości

Table of contents

1. Introduction

Much of the reinforcement learning literature in trading relies on low-frequency market representations such as daily or intraday OHLCV bars.[1] High-frequency trading, however, is governed by information embedded in the limit order book, where local liquidity, queue dynamics, spread changes, and event timing determine short-horizon risks. This thesis examines whether a deep RL agent can use order-book-derived features to make favourable trading decisions in a HFT setting.

High-frequency trading should therefore not be treated as a faster version of medium-frequency trading (MFT). The state observed by the agent is different. At short horizons, aggregated price bars omit information that is central to decision-making. Features derived from the order book provide a more detailed description of market conditions than candlesticks. These include depth imbalance, spread variation, order-flow pressure, and inter-event timing. A primary concern of this thesis is whether such microstructure-aware state representations improve the suitability of RL for this domain.

Reinforcement learning is a plausible modelling framework for this problem because trading is sequential and path dependent. Each action affects future exposure, portfolio evolution, and the set of subsequent decisions available to the agent. In high-frequency settings, this distinction matters.

Reinforcement learning has already been deployed in industry settings on limit order book data, but these applications typically address narrower problems such as optimal execution.[2]

Directional trading is a harder problem for several reasons. First, the reward signal at tick frequency is dominated by noise rather than price movement. Second, the LOB state must be transformed into features that carry predictive signal rather than merely describing liquidity conditions. Third, the agent must generalize across intraday regimes whose statistical properties can differ substantially from the training window.

This thesis investigates the use of RL, with particular focus on TD3, for high-frequency

[1] Abbreviations and domain-specific terms used throughout the thesis are defined in the glossary at the end of the document.

[2] An illustrative example is J.P. Morgan's LOXM system, described in contemporaneous reporting as an AI-driven execution tool for client orders rather than a stand-alone directional trading strategy (Noonan, 2017). In that execution setting, the limit order book is used to inform decisions about execution quality, fill likelihood, and spread-cost trade-offs, not to forecast short-horizon price direction.

single-asset trading based on order-book-derived information. The research is motivated not by a claim that artificial intelligence can universally solve trading, but by a narrower and testable question: under controlled experimental conditions, can a continuous control agent using microstructure-aware features achieve superior out-of-sample performance relative to selected benchmarks — and, if so, whether that performance survives realistic transaction costs?

Further constraints arise on both the execution and evaluation sides: transaction costs and turnover sensitivity can erase apparent profitability, and favorable in-sample results may instead reflect data leakage or unrealistic execution assumptions. The simulation is not a full execution engine — it omits queue-priority modelling, latency competition, and market impact, valuing the portfolio through a mid-price-like proxy and treating the agent as a price-taking participant that cannot move the book. Chapters 5 and 7 detail these limitations and the methodological safeguards (chronological evaluation, reward-return separation, robustness analysis) adopted in response. The contribution of the thesis is accordingly methodological and empirical: it evaluates RL agents in a microstructure-aware setting while making its simplifying assumptions explicit.

1.1. Scope and Objectives of the Research

1.1.1. Novelty of This Research

Building on the TD3/HFT/order-book focus introduced in Chapter 1, the novelty claim here is comparative: a large share of prior RL trading studies target lower-frequency inputs and evaluate algorithms such as Q-learning, DQN, or PPO primarily in bar-level settings, whereas this thesis aligns algorithm class, data granularity, and feature design jointly with high-frequency market microstructure. This algorithm-data-frequency combination remains less represented in prior work and defines the core novelty claim tested in the empirical chapters.

1.1.2. Hypothesis

The research is based on the following hypotheses, ordered so that each builds on the previous one; together they make the case that robustness analysis is a precondition for any credible economic claim:

1. A TD3 agent trained on order-book-derived high-frequency features learns a genuine, learnable out-of-sample risk-adjusted edge from limit-order-book microstructure, but the

economic magnitude of that edge is of the same order as the transaction cost required to trade on it: it is realised under frictionless mid-price execution and is not expected to survive realistic spread-crossing costs.

2. The empirical performance of the agent is materially sensitive to the transaction-cost assumption: the learned edge is positive under zero fee but changes sign once the per-trade cost reaches a fraction of a basis point — the scale of the bid-ask half-spread on these instruments.
3. Within the order-book-based trading framework studied here, extending the state representation from basic snapshot features to a broader microstructure-aware feature set leads to improved out-of-sample performance under identical training and evaluation conditions.
4. The reward objective materially shapes the learned policy: replacing a log-return reward with a Differential Sharpe Ratio reward changes both out-of-sample performance and the policy's directional exposure, under otherwise identical training and evaluation conditions.

1.1.3. Objectives of the Research

The primary research goal is to evaluate RL-based algorithms for high-frequency single-asset trading using order-book-derived data. The research objectives are structured in three interconnected phases:

1. **Design and Implementation**
   - Develop RL trading agents for high-frequency order-book trading applications
   - Construct a testing framework to evaluate agent performance
2. **Testing and Evaluation**
   - Assess agent performance on out-of-sample data against selected benchmarks with established and novel risk-return metrics, operationalizing the frictionless-execution component of Hypothesis 1 (its transaction-cost bound is operationalized under Hypothesis 2)
   - Analyze robustness under varying market conditions
3. **Analysis and Implications**
   - Compare alternative order-book-derived feature sets within a common training and evaluation framework, operationalizing Hypothesis 3
   - Assess the practical relevance and limitations of the approach under simplified execu-

tion assumptions

- Identify the binding constraints on out-of-sample performance — whether they originate in execution modeling simplifications, feature set coverage, or training data volume — to prioritize specific extensions for future work

1.2. Thesis Organization and Chapter Overview

The thesis proceeds from problem formulation and motivating literature to agent design and implementation, and finally to empirical evaluation.

Chapter 1 introduces the problem context, defines the research scope, and presents the methodological framework used throughout the thesis.

Chapter 2 reviews the literature most directly relevant to the present research question. It covers market microstructure evidence motivating order-book-derived features, the landscape of alternative machine learning paradigms and trading decision architectures, the structural case for reinforcement learning in trading, and prior RL trading applications with attention to action-space design, transaction-cost modeling, and evaluation rigor.

Chapter 3 develops the reinforcement learning foundations required for the thesis, with particular emphasis on the progression from earlier methods to TD3 and on the distinctions that matter for continuous-control trading problems.

Chapter 4 presents the design of the trading agent itself. It defines the continuous action space, the microstructure-aware state representation, the reward formulation, and the main policy and value-function design choices used in the empirical system.

Chapter 5 describes the implementation of the research pipeline, including the simulation architecture, data preparation, reproducible feature engineering, code structure, and the TD3-based training workflow used to produce reproducible experiments.

Chapter 6 reports the empirical results. It combines experiment snapshots, performance evaluation, statistical validation, and robustness assessment to separate promising outcomes from noise, overfitting, or fragile conclusions.

Chapter 7 synthesizes the findings, discusses the main limitations of the current framework, and draws implications for future research and practical trading-system development.

2. Literature Review

This chapter surveys the academic foundations relevant to reinforcement learning for tick-level order-book trading. It first distinguishes the controlled simulation setting studied here from live high-frequency trading, then reviews the microstructure mechanisms that motivate the candidate state variables used in Chapter 4. The chapter then separates descriptive microstructure variables from tradable signals: a feature is useful only if it contains information available before the action, survives costs, and improves out-of-sample performance. The second half compares competing modeling approaches, introduces the reinforcement learning concepts most relevant to trading, and reviews prior RL trading applications through state representation, action-space design, reward definition, transaction-cost modeling, and validation rigor. The survey concludes by identifying the comparatively underexplored combination of LOB-derived features, continuous control, and microstructure-aware state design that this thesis studies.

2.1. Tick-Level Order Book Trading

Tick-level trading differs from bar-based trading because the decision unit is an order book event rather than a fixed calendar interval. A daily or minute-level strategy can often treat execution as a secondary implementation detail after the signal is estimated.[3] At tick frequency, execution conditions are part of the signal environment itself: the spread, queue sizes, recent order-flow pressure, and time since the last event all affect whether a position change is worth making.

This matters because expected price changes over short horizons are small relative to trading frictions. Crossing the spread, paying transaction costs, or changing exposure during a transient liquidity shock can dominate the gross directional signal. A tick-level agent must therefore condition not only on where the price may move, but also on whether the current liquidity state makes acting economically sensible.

The present thesis studies this problem in a controlled simulation, not in a live high-frequency trading environment. The simulator uses historical limit-order-book data and explicit cost assumptions to evaluate continuous position control. It does not model exchange latency, queue priority, partial fills, strategic order cancellation, or the market impact of the agent's own

[3] This generalization depends on the scale of trading. For investment funds, execution is rarely a negligible issue — quite the opposite. Large-scale execution costs, market impact, and implementation shortfall can dominate gross returns, making execution quality a primary concern.

orders. The literature reviewed below is therefore used to motivate state variables and evaluation discipline, not to claim full live-HFT realism.

The microstructure mechanisms discussed in this chapter manifest in observable empirical regularities at intraday and tick-level timescales.

At these timescales, financial markets exhibit structural patterns that are distinct from the macro-level anomalies studied in the asset pricing literature. They arise not from investor irrationality or information asymmetry in the fundamental sense, but from the mechanics of how orders interact in a limit order book: the strategic behavior of market makers, the clustering of institutional activity, and the discrete price grid imposed by exchange rules. Understanding these patterns is a prerequisite for designing a tick-level trading agent, since they define the environment in which the agent operates and motivate the feature set used here.

Bid-ask spreads induce negative serial correlation in transaction prices at very short horizons, as they bounce between bid and ask quotes (Roll, 1984). This microstructure noise distinguishes observed transaction prices from the latent efficient price and creates a mechanical pattern that matters for any tick-level decision rule. In this work, spread in basis points and the spread ratio feature directly encode the cost of crossing the bid-ask, conditioning the agent's position-sizing decisions on the current liquidity environment.

Intraday patterns in volume, volatility, and returns have been documented: a pronounced U-shape in trading activity and spread width, with elevated liquidity demand at market open and close and a quiet midday period (Harris, 1986; Wood et al., 1985). These patterns reflect institutional trading schedules and liquidity provision dynamics rather than information arrival. The hour-of-day sine and cosine encodings in the observation vector are motivated by this evidence because the agent should distinguish early-session conditions (wide spreads, high adverse selection) from midday periods (tighter spreads, lower activity) when sizing positions.

Prices cluster at round numbers and psychologically attractive levels, creating discontinuities in the order book that affect optimal order placement and execution (Harris, 1991). In a 10-level LOB, price clustering manifests as uneven spacing between price levels — a pattern captured by the book convexity feature, which measures the curvature of the price ladder on each side. Awareness of price clustering is also relevant to the spread ratio feature, since clusters create temporary spread compression and expansion that deviates from the recent average.

These three findings collectively establish that short-horizon price dynamics are structured, not random. The bid-ask bounce motivates spread-aware features and reward design; intraday periodicity motivates temporal context features; price discreteness motivates curvature features of the order book. Rather than treating the limit order book as a transparent window onto an efficient price, the empirical evidence supports modeling it as a strategic environment with predictable mechanical regularities — which is precisely the setting in which a reinforcement learning agent can learn to act systematically.

2.2. Market Microstructure Mechanisms

Market microstructure is grounded in a theoretical literature that models the strategic interactions between informed traders, uninformed liquidity demanders, and market makers in a limit order book. Chapter 4 derives its features from three elements of each theoretical framework: the market mechanism, its observable proxy, and its limitation. Each framework motivates a candidate state variable, but none alone establishes profitability after costs.

Adverse selection and the spread decomposition. The bid-ask spread reflects a market maker's protection against trading with informed counterparties (Glosten & Milgrom, 1985). In their model, the market maker faces two types of order flow: uninformed liquidity traders, whose trades are uncorrelated with future price changes, and informed traders, who possess private information about fundamental value. Setting a positive spread allows the market maker to profit from uninformed flow and recoup losses on informed flow. The spread therefore decomposes into an adverse selection component — compensation for information risk — and an inventory or order processing component.

This decomposition has a direct implication for trading signal design. A narrowing spread signals reduced adverse selection risk; a widening spread signals elevated informed activity. These measures, operationalized in Chapter 4 as spread in basis points and the spread ratio, are therefore motivated as candidate state variables by this theoretical framework.

The price impact of informed trading was formalized through the lambda parameter, which measures the sensitivity of price changes to order flow (Kyle, 1985). In Kyle's continuous-time model, the informed trader optimally splits their order to minimize price impact, and the market maker sets prices to break even against order flow. The resulting relationship between order imbalance and price change is linear, with slope λ that depends on the ratio of informed to uninformed volume. The order flow features in Chapter 4 — OFI and signed trade flow — can be understood as empirical proxies for Kyle's order imbalance variable, with the bid-ask slope features measuring the depth analogue of λ across levels of the order book. The limitation is that these proxies summarize observable pressure and depth, not the latent information set of the informed trader in Kyle's model.

Limit order book mechanics. The theory of the open limit order book as a competitive equilibrium was developed, showing that a full LOB can sustain itself as a market mechanism under adverse selection (Glosten, 1994). The key insight is that the price schedule embedded in the LOB — the sequence of bid and ask quotes at increasing depths — reflects the cumulative information content of orders at each quantity level. Deeper levels of the book attract less informed order flow but carry higher execution uncertainty for those posting limit orders.

This theoretical result motivates the multi-level features used in Chapter 4. The book is not a single price but a price schedule, and the shape of that profile — measured by depth ratios, slopes, and convexity features — may encode information about how informed the marginal liquidity provider expects the market to be at each quantity level.

Order flow and short-term price prediction. The most direct empirical and theoretical treatment of the relationship between LOB events and short-term price changes is found in the market microstructure literature (Cont et al., 2014). Their framework decomposes all LOB events into limit order arrivals, cancellations, and market orders, and defines order flow imbalance (OFI) as the net directional pressure these events exert on the best bid and ask queues. Using NYSE TAQ data for 50 randomly sampled S&P 500 stocks, they find a linear relationship between OFI and mid-price changes over ten-second intervals, with OFI explaining an average of 65% of short-term price variation (68% with a nonlinear correction). Importantly, the relationship is contemporaneous: OFI at time t is correlated with price changes at time t, not a lagged effect, which is consistent with the efficient incorporation of order book information into prices.[4]

The theoretical channel is straightforward: when buy-side pressure dominates (positive OFI), market makers raise quotes to attract sell-side flow and reduce inventory risk, mechanically pushing the mid-price upward. The OFI feature in the feature design is included primarily as a state variable summarizing current order-flow pressure.

[4] OFI's relevance for the present thesis is narrower than the contemporaneous result implies: it serves as a compact state descriptor of current queue pressure at the time the agent acts, not a validated forecasting signal.

Fair value estimation from the order book. The microprice is derived as the theoretically optimal estimate of the next transaction price given the current state of the top-of-book queues (Stoikov, 2018). The key observation is that the mid-price — the arithmetic average of best bid and ask — is an inadequate fair value estimate when the queues are imbalanced, because the side with the thinner queue is more likely to be depleted first by incoming market orders, pulling the next transaction price toward the opposite side. The microprice corrects for this by volume-weighting the best quotes so that when the bid side is heavier than the ask, the estimate is pulled toward the ask — the side more likely to move (exact formula and implementation in Appendix A). The microprice divergence from mid captures this adjustment as a stationary signal and serves as the primary fair value feature in Chapter 4. Its limitation is that it is a top-of-book fair-value proxy; it does not incorporate deeper queue dynamics, hidden liquidity, or the agent's own execution priority.

Taken together, these four theoretical frameworks identify several mechanisms that may be relevant for state design in limit-order-book markets. The spread encodes adverse selection risk (Glosten & Milgrom, 1985); price impact and order imbalance are related through Kyle's lambda (Kyle, 1985); the shape of the full book encodes the information content at each depth level (Glosten, 1994); and OFI is the dominant contemporaneous predictor of short-term price changes (Cont et al., 2014). The microprice (Stoikov, 2018) provides a theoretically optimal fair value estimate that conditions on the current queue imbalance.

These mechanisms are not independent: they all reflect aspects of information aggregation through order flow, motivating the candidate feature families introduced in Chapter 4 — spread-based, order-flow, depth-shape, and fair-value signals.

2.3. From Microstructure Signals to Tradable Decisions

LOB-derived variables may describe the market without forecasting it. A feature earns its place in the state space only if it survives transaction costs and improves held-out performance.

In the quantitative trading literature, predictive signals of this kind are commonly referred to as alphas: mathematical expressions over market data that encode a directional view on short-horizon returns (Tulchinsky, 2019). The microstructure features in this thesis's state space — order-flow imbalance, microprice divergence, spread regime, and book shape — occupy the same conceptual role, but are not used as standalone trading rules: they instead serve as inputs to an RL policy that learns, through experience, how to combine and act on them optimally, replacing the hand-coded combination rule with a learned function while keeping the same evaluation discipline of out-of-sample performance under realistic costs.

2.4. Competing Modeling Approaches

Alternatives to reinforcement learning differ along two dimensions: learning paradigm and decision-system architecture. The relevant paradigms are supervised, unsupervised, semi-supervised, and reinforcement learning. Their architectural differences motivate the choice of reinforcement learning for the present problem.

2.4.1. Learning Paradigms

Supervised Learning. Supervised learning trains a model on labeled input-output pairs $(X_i, Y_i)_{i=1}^n$ to minimize a loss function $\mathcal{L}(f_\theta(X), Y)$ over a held-out sample. In trading, supervised approaches have been widely applied to return prediction, directional classification, and volatility forecasting — tasks where historical outcomes provide a natural training signal (Krauss et al., 2017; Zhang et al., 2019).

This includes modern LOB prediction models, which use deep architectures to map order book snapshots or event sequences into short-horizon price-movement labels. Such models are a serious benchmark class for extracting information from high-dimensional market data, especially when the objective is directional classification (Zhang et al., 2019). A strong supervised trading benchmark would not stop at an unconstrained return forecast. It would translate predicted returns or class probabilities into positions through a cost-aware rule, such as trading only when the expected move exceeds the spread and fee threshold, scaling exposure by forecast confidence, or optimizing classification thresholds on a validation set for net return rather than raw accuracy. A second supervised variant would learn the action directly from labels constructed ex post: for example, the position that would have maximized next-horizon net return after costs, or a discretized buy/sell/hold target with an explicit no-trade region.

These alternatives are important because they close part of the gap between prediction and trading. They allow the supervised model to account for transaction costs, avoid low-conviction trades, and produce a policy-like mapping from market state to position. The limitation is therefore not that supervised learning is an inherently weak baseline. It is that the financial objective enters the pipeline indirectly: either through hand-designed thresholds and sizing rules applied after prediction, or through labels whose definition already embeds a particular holding horizon, cost model, and notion of ex post optimality.

This creates two sources of possible misalignment. First, a model that minimizes mean squared error, cross-entropy, or another statistical loss may not maximize risk-adjusted return after transaction costs, because predictive accuracy and trading profitability are not equivalent criteria (Moody & Saffell, 1998). Second, direct action labels are only as credible as the rule used to construct them. In a noisy and path-dependent environment, the ex post best one-step action need not be the action that maximizes longer-horizon portfolio utility once turnover, inventory persistence, and risk are considered.

This misalignment is most consequential at high frequencies, where turnover costs consume a large fraction of gross returns. At lower frequencies — daily or weekly rebalancing — the cost of turning over a position is small relative to the forecast signal, and well-specified supervised models remain competitive. Factor-based strategies in the Fama-French tradition, momentum screens, and several recent deep-learning forecasting papers achieve meaningful net alpha precisely because the prediction horizon is long enough for transaction costs to be a second-order concern. The advantage of direct optimization narrows as the cost-to-signal ratio decreases. At tick frequency, where the present thesis operates, this ratio is unfavorable for a predict-then-trade pipeline: the cost of crossing the spread on each position change can exceed the directional signal per step, making the alignment between reward and realized return a first-order design requirement. The motivation for reinforcement learning in this thesis should therefore be understood narrowly. TD3 is not adopted because supervised LOB prediction is irrelevant, but because the empirical question studied here concerns continuous exposure control under a reward function that includes the consequences of position changes directly.

Unsupervised Learning. Unsupervised learning identifies structure in unlabeled data $\{X_i\}_{i=1}^{n}$ without a target variable. In trading research it is primarily used for market regime detection (clustering volatility or microstructure states), dimensionality reduction of large feature sets (PCA, autoencoders), and anomaly detection in order flow data.

These applications are useful as preprocessing or exploratory tools, but unsupervised learning does not produce trading decisions directly — it carries no reward signal and cannot optimize for a financial objective such as risk-adjusted return. In the present thesis, the regime-conditioning problem is addressed implicitly through engineered microstructure features — spread ratio, inter-event time, temporal encodings — that condition the agent's observation on the current market state without requiring an explicit unsupervised clustering step.

Semi-Supervised Learning. Semi-supervised learning combines a small labeled set with a larger unlabeled set, propagating label information through structural similarity in the data. It is useful when labeling is expensive or subjective — medical annotation being the canonical application.

In financial trading, the approach is rarely adopted because the main bottleneck is not label scarcity but the absence of a stable labeling criterion: what constitutes an optimal action at a given market state depends on subsequent price evolution, which is noisy and non-stationary. Constructing pseudo-labels for unlabeled market observations requires either a strong prior on optimal behavior or a separate forecasting model — at which point the problem has already been reformulated as supervised or reinforcement learning. This instability of financial labels is documented in the literature: fixed-horizon labeling schemes are path-dependent and regime-sensitive, which motivates the triple-barrier method as a more robust alternative — itself an acknowledgment that no universal labeling criterion exists across market conditions (López de Prado, 2018, ch. 3).

The same conclusion follows from a second perspective: the architecture of the decision system itself.

2.4.2. Decision-System Architectures

Rule-Based Trading Strategies. Rule-based strategies encode trading decisions as explicit conditional logic: position changes are triggered when observable market quantities cross pre-defined thresholds. Common examples include moving-average crossover systems, momentum breakout rules, and mean-reversion strategies based on Bollinger Bands or RSI. In high-frequency settings, rule-based approaches include market-making strategies that post limit orders at fixed offsets from the mid-price and cancel them when inventory limits are reached.

The appeal of rule-based systems is transparency: the causal chain from market condition to action is explicit, auditable, and reproducible. They also impose no distributional assumptions on the data and can be deployed without a training phase.

The limitation is inflexibility. Rules are defined over a fixed set of observable conditions and optimized for a particular market regime. When it changes — for example, when a previously mean-reverting asset enters a trending period — fixed rules either require manual recalibration or continue generating losses until the regime reverts. This brittleness under non-stationarity is the primary motivation for learning-based approaches, which can update their behavior from experience rather than requiring explicit rule revision.

Indirect Optimization via Forecasts or Labels. An intermediate class of trading systems learns a surrogate target rather than optimizing trading performance directly. One variant employs supervised learning to generate price forecasts from market inputs and then feeds those forecasts into a separate decision rule. Another variant trains directly on labeled state-action pairs, imitating historical or hand-crafted trades without an explicit forecasting stage — learning the action-mapping function by minimizing imitation loss against expert or rule-derived action labels, the standard behavior-cloning setup.

The limitation is the same in both cases: the optimization target is a surrogate rather than the trading objective itself. Forecast-based systems optimize statistical accuracy and then translate predictions into actions through a separate rules layer, discarding much of the contextual information present in the original state. Label-based systems optimize imitation error against actions derived from historical rules or expert behavior, which embeds a prior about what constitutes optimal behavior and does not guarantee maximizing risk-adjusted return.

Direct Optimization of Trading Performance. Direct optimization maps market states to actions and rewards them from realized returns net of transaction costs, eliminating the calibration gap between a forecasting objective and a separate decision rule. It does not eliminate modeling assumptions: the Differential Sharpe Ratio used here approximates risk-adjusted return, so its relationship with out-of-sample performance remains empirical. Chapter 4 develops this distinction, explains the need for reward shaping at tick frequency, and separates the training objective from financial evaluation.

2.5. Reinforcement Learning for Trading

Reinforcement learning is a reasoned choice for algorithmic trading, not an arbitrary methodological one. The argument rests on three structural properties of the trading problem.

Unknown market dynamics. Model-based approaches require an explicit transition model — a specification of how the environment responds to actions. In financial markets, this model is unknown and non-stationary: price dynamics depend on the aggregate behavior of all participants, which changes continuously. Model-free RL methods learn directly from experience without requiring a transition model, making them a reasonable candidate for this setting (Sutton & Barto, 2018).

Off-policy learning and historical replay. Online experimentation in live markets is constrained by capital, risk, and regulatory requirements. The alternative is to learn from historical order book data — a fixed dataset of past transitions. Off-policy algorithms such as TD3 (Fujimoto et al., 2018) can train on stored experience rather than requiring fresh environment interaction at every update, making them sample-efficient in the data-limited financial setting. This should not be confused with a full offline-RL guarantee. Training from historical replay still raises distribution-shift and extrapolation risks when the learned policy selects actions that are poorly represented in the historical data.

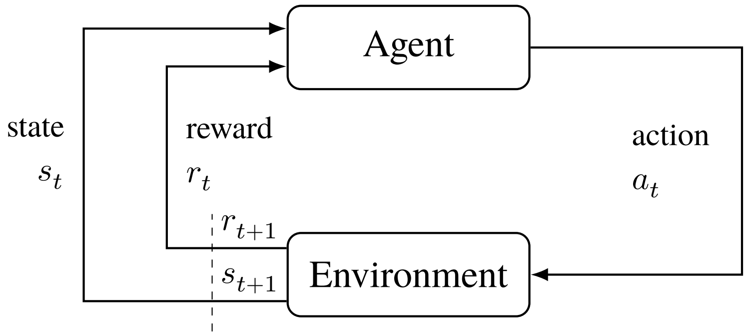
Continuous position sizing. Trading decisions are naturally continuous: the agent should be able to express any degree of conviction through the size of its position, not just a binary buy/sell signal. Value-based methods such as DQN (Mnih et al., 2015) require a discrete action space and cannot represent fractional positions without reintroducing a grid approximation. Deterministic actor-critic methods operate natively in continuous action spaces and are therefore more appropriate for position-sizing tasks.

Together these three properties point toward a model-free, off-policy, continuous-action algorithm — which is the class to which TD3 belongs. This is a reasoned design choice, not a claim that TD3 is universally superior for trading. The MDP formulation remains approximate because market states are partially observable and non-stationary. Chapter 3 derives TD3's formal algorithmic properties. The broader RL literature informs the trading-specific design choices made here.

2.6. Trading Problem Formulation

The finance literature reviewed here relies on RL concepts with specific consequences for trading. Chapter 3 provides their formal definitions, equations, and algorithm taxonomies.

Figure 1. The agent-environment interaction loop in reinforcement learning.



Source: *author's own elaboration, based on Sutton & Barto (2018).*

At each step the agent observes state s_t and reward r_t, selects action a_t, and receives the next state s_{t+1} and reward r_{t+1} from the environment.

Portfolio decisions are sequential and path-dependent. This makes the MDP framework (Bellman, 1957; Puterman, 1994) the formal language for trading agents. However, financial environments only approximately satisfy standard MDP assumptions. Market regime shifts, structural breaks, and partial observability all contribute to this approximation.

For this review, the most useful RL concepts are not the formal machinery of Chapter 3, but the dimensions along which applied trading papers differ. These dimensions determine whether results from one study are comparable to another and whether they are relevant to the problem addressed in this thesis.

The first dimension is the formulation of the trading problem itself: what is treated as the state, what counts as an action, and what scalar reward is used to evaluate behavior. In trading applications, these choices vary materially across studies. Some papers define the state using bar-level prices and technical indicators, while others incorporate current holdings, order-book information, or broader market context. Likewise, actions may represent a discrete buy/sell/hold decision or a continuous exposure level. Reward definitions also differ: some studies optimize raw return, others net return after transaction costs, and others risk-adjusted criteria such as Sharpe-oriented objectives (Moody & Saffell, 1998). These modeling choices are not cosmetic. They change the economic meaning of the task the algorithm is solving, and any reward transformation must preserve the original task's optimal policy to remain valid (Ng et al., 1999).

The second dimension is action-space structure. Discrete-action methods such as Q-learning and its deep variants are naturally aligned with buy/sell/hold formulations (Mnih et al., 2015; Watkins, 1989). Continuous-control methods, by contrast, are designed for problems in which the agent chooses a portfolio weight or exposure level on a continuum. This distinction is central in the trading literature because it separates papers that treat trading as a classification-like decision problem from papers that treat it as a position-sizing problem. For the latter class, policy-gradient and actor-critic methods are usually the relevant algorithmic family (Sutton et al., 2000; Williams, 1992).

The third dimension is the training regime, especially the distinction between on-policy and off-policy learning. In financial applications this matters not only for sample efficiency but also for the practicality of learning from historical data. Off-policy methods can in principle reuse stored experience through replay-based updates (Lin, 1992), which is one reason they appear frequently in trading research where live experimentation is expensive or constrained. At the same time, the literature makes clear that training on historical market data does not eliminate instability, because financial environments remain noisy, non-stationary, and only approximately Markov (Deng et al., 2017; Tsitsiklis & Van Roy, 1997). Algorithm choice therefore cannot be separated from the validation protocol under which performance is assessed.

The fourth dimension is reward design. Financial RL studies differ not only in which algorithm they use, but in what behavior they incentivize. Rewards based on raw price changes can encourage excessive turnover or sensitivity to noise, whereas cost-aware or risk-adjusted rewards aim to align optimization more closely with economically meaningful outcomes. The Differential Sharpe Ratio is one influential example (Moody & Saffell, 1998), but the broader point is that reported performance is difficult to interpret without understanding the objective under which the policy was trained.

These distinctions provide the lens for reading the applied RL studies reviewed in the next section. Chapter 3 develops the formal reinforcement learning framework and the algorithmic lineage leading to TD3.

2.7. Applied RL Trading Evidence

RL trading research has progressed from theoretical proposals to applications across diverse markets. Five design dimensions organize the relevant evidence: state representation, action-space structure, reward definition, transaction-cost modeling, and validation protocol.

To keep conclusions methodologically defensible, this review distinguishes between: (i) proof-of-concept evidence from backtests under simplified assumptions, and (ii) stronger evidence that includes robust out-of-sample design, realistic cost modeling, and variance reporting across repeated runs. The chronology below is therefore secondary to the methodological comparison: reported performance is interpreted skeptically unless the study specifies what information the agent observes, how actions map to trades, whether costs are charged, and how out-of-sample performance is selected and reported.

2.7.1. Early Applications and Proof-of-Concept

The first comprehensive application of RL to trading, introducing recurrent reinforcement learning for optimizing trading systems, was presented in (Moody & Saffell, 1998). This approach incorporated four design choices that remain influential: the Differential Sharpe Ratio as a risk-adjusted reward, a recurrent network architecture for temporal state encoding, direct policy optimization rather than a predict-then-trade pipeline, and explicit transaction cost modeling. It was demonstrated that RL could outperform supervised learning baselines on S&P 500 futures, establishing feasibility of the approach and showing that optimizing risk-adjusted return directly was often superior to predicting intermediate outcomes.

This framework was extended to an asset-allocation setting (S&P 500 index versus T-bills), validating that RL-optimized policies could generate positive risk-adjusted returns after transaction costs — a stringent test of practical viability (Moody & Saffell, 2001).

2.7.2. From Discrete to Continuous Action Spaces

Early proof-of-concept studies established feasibility while surfacing problems this thesis's design responds to directly. Discrete Q-learning on equity and futures markets (J. W. Lee et al., 2007; Neuneier, 1998) learned profitable patterns but suffered sparse rewards, delayed credit assignment, and out-of-sample degradation; more fundamentally, discrete buy/sell/hold actions force an artificial granularity on position sizing — a coarse grid loses control precision, a fine grid reintroduces the curse of dimensionality. Extending direct/recurrent RL to foreign-exchange trading (Dempster & Romahi, 2002; Gold, 2003) confirmed that mapping state directly into positions is viable, but performance varied substantially across currency pairs and system parameters — feasibility evidence, not a robust trading edge.

2.7.3. Deep Reinforcement Learning Era

The introduction of deep function approximators to RL trading extended applicability to high-dimensional state spaces.

DDPG was applied to Dow-30 equity trading using engineered technical-indicator features (Xiong et al., 2018), while a fuzzy-representation autoencoder was combined with direct reinforcement learning for futures trading (Deng et al., 2017). Both studies noted high variance across runs — a characteristic instability of these methods under noisy financial rewards.

The portfolio-vector-memory (PVM) architecture was proposed, combining policy gradients with recurrent networks for cryptocurrency portfolio management (Jiang et al., 2017). Its fully differentiable end-to-end design incorporated current holdings into the state and penalized transaction costs explicitly, outperforming Markowitz mean-variance allocation in backtests across 12 cryptocurrencies.

An earlier cryptocurrency portfolio study used a convolutional network to map historical prices directly into continuous portfolio weights under a deterministic policy-gradient-style objective (Jiang & Liang, 2017). Its 30-minute Poloniex backtests are relevant because they treat trading as direct portfolio-action optimization, but the assumptions of immediate execution and no market impact remain far from tick-level order-book trading.

DDPG, PPO, and Policy Gradient (PG) were compared for China A-share portfolio management, finding — counterintuitively — that the simpler PG algorithm outperformed both actor-critic methods (Z. Liang et al., 2018). The proposed adversarial training method improved average daily return and Sharpe ratio in backtests, with the PG-based agent outperforming a uniform constant rebalanced portfolio baseline.

A focused TD3 study on single-asset trading for AMZN and BTC used daily data and a continuous action space in $[-1, 1]$ (Majidi et al., 2024). Its minimal state representation — lagged close-return windows rather than large technical-indicator sets — reported improved return and Sharpe ratio for TD3 versus discrete variants and buy-and-hold, though results were mixed across assets. This study is best interpreted as evidence that continuous action sizing can be beneficial under realistic frictions, not as definitive proof of robust alpha across market regimes.

A TD3-based framework incorporating account state (cash and holdings), price features, technical indicators, and optional FinBERT-derived news sentiment was proposed (Kabbani & Duman, 2022). Richer state designs improved both return and Sharpe ratio relative to a close-price-only baseline, with a final 10-stock evaluation reporting a Sharpe ratio of 2.68. This figure warrants scrutiny: annualized Sharpe ratios above 2 are rare in live trading and typically indicate either insufficient out-of-sample discipline, favorable selection among multiple model variants, or evaluation periods that coincide with a strong directional trend. The study does not report results across multiple random seeds or under realistic spread-crossing costs with a frozen policy, making it difficult to determine whether the reported performance reflects genuine signal or overfitting to the evaluation window. Methodologically the paper supports the practical value of feature enrichment in continuous-action actor-critic trading, but it also illustrates a broader pattern in the RL trading literature: without standardized evaluation protocols — multi-seed reporting, held-out test periods not used for model selection, and explicit cost assumptions — performance claims are difficult to assess on their own terms.

Two further studies mark boundary cases outside this thesis's LOB-level, single-asset scope. An ensemble PPO/A2C/DDPG strategy on Dow Jones constituents (Yang et al., 2020) is a useful actor-critic comparison in a liquid equity universe, but its state representation, trading frequency, and execution assumptions differ from millisecond-level LOB trading. AlphaStock (Wang et al., 2019) combines a Sharpe-oriented objective with history-state and cross-asset attention, contributing mainly through representation and interpretability in stock selection — supporting the claim that state representation matters materially in RL trading, but not addressing order-book-driven single-asset exposure control.

A volatility-scaled deep RL trading strategy tested across 50 liquid futures contracts spanning commodities, equity indices, fixed income, and FX, benchmarked against classical time-series momentum, was proposed (Zhang et al., 2020). Its result — that reward and risk-scaling choices materially affect realized performance — reinforces that algorithm choice cannot be separated from state representation, action design, reward definition, transaction costs, and evaluation protocol.

2.7.4. Comparative Evidence and Limitations

Algorithm comparisons and reproducibility. A later move toward reproducible DRL trading pipelines is illustrated by an implementation of several standard algorithms, including DQN, DDPG, PPO, SAC, A2C, and TD3, within common market environments and backtesting workflows (Liu et al., 2020). Its relevance is infrastructural rather than evidentiary: it shows that applied trading studies increasingly require standardized environments, comparable baselines, and explicit market-friction assumptions before algorithm rankings can be interpreted.

Deep RL was compared against traditional quantitative strategies (momentum, mean reversion, pairs trading), noting that RL exhibited superior adaptability during regime changes while traditional strategies offered better interpretability (López de Prado, 2018). Hybrid approaches combining domain knowledge with RL often outperformed either in isolation.

Overfitting and generalization. Published deep RL trading strategies were shown to often fail to generalize out-of-sample due to backtest overfitting, motivating a probability-of-overfitting estimator to reject overfit agents before deployment (Gort et al., 2022). The implication is that evaluation methodology — strict temporal splits, multi-seed reporting, realistic cost assumptions — matters as much as algorithm choice.

Reward specification. Reward shaping is one of the central design choices in DRL trading (Millea, 2021). Misspecified rewards can lead to metric gaming, excessive turnover, ignoring tail risks, or misalignment with actual investor preferences. The Differential Sharpe Ratio (Moody & Saffell, 1998) addresses some of these concerns by targeting risk-adjusted return directly, but does not fully resolve the tension between reward simplicity and the richness of real trading objectives.

The literature reviewed here converges on design lessons rather than a headline result. Continuous-action actor-critic methods are frequently preferred over discrete alternatives for position-sizing tasks (Kabbani & Duman, 2022; Z. Liang et al., 2018; Majidi et al., 2024), since action spaces must match the economic decision being made. Risk-adjusted reward signals — especially Differential Sharpe Ratio formulations — appear promising for out-of-sample generalization relative to raw return objectives, provided they penalize costs and avoid encouraging excessive turnover; explicit transaction-cost modeling is a prerequisite for any realistic performance claim at the frequencies considered here, and such claims remain highly sensitive to the evaluation protocol. Validation must use held-out periods, frozen policies, and repeated runs where stochastic training is material — without these elements, a positive backtest is best interpreted as a proof of concept rather than evidence of a robust trading edge.

Within the representative literature reviewed here, a comparatively underexplored design combination is the joint use of order-book-derived state variables, continuous-control RL, and an explicit microstructure-oriented state design. Much of the applied literature relies primarily on OHLCV or other aggregated price representations, while studies using deeper limit-order-book information remain fewer and methodologically heterogeneous. The present thesis is positioned within that less represented part of the literature by constructing the agent's observation vector from 10-level order book data and by evaluating a continuous-control agent under the modeling assumptions stated in later chapters.

2.8. Implications for This Thesis

The literature reviewed in this chapter motivates the thesis design along five dimensions. First, market microstructure theory and evidence support the use of LOB-derived state variables rather than relying only on bar-level price summaries. Spread, imbalance, order-flow, fair-value, and intraday-regime features each correspond to a distinct mechanism discussed in the microstructure literature.

Second, the reviewed evidence also limits interpretation: these are candidate state variables, not guaranteed trading signals, tested empirically in later chapters.

Third, the comparison with supervised and rule-based approaches motivates direct optimization of trading decisions. Forecasting models can be informative, but a separate rule is still required to translate predictions into exposure. In this thesis, the agent instead chooses a continuous portfolio position directly, making the action space part of the optimization problem.

Fourth, the applied RL literature motivates the use of continuous-control actor-critic methods and risk-aware reward design, while also requiring caution about reported performance. Prior studies show that RL trading results are sensitive to transaction costs, benchmark choice, random seeds, and validation protocol. For that reason, the later empirical chapters separate training reward from financial evaluation metrics and assess the learned policy on held-out data.

Finally, the contribution of the thesis follows from the combination of these design choices: a microstructure-aware state representation, continuous exposure control, transaction-cost-aware evaluation, and explicit robustness assessment within a controlled single-asset HFT simulation. The objective is not to claim a deployable trading system, but to test whether this particular design combination can produce credible out-of-sample evidence under the stated simulation assumptions.

3. Reinforcement Learning

The formal RL framework comprises states, actions, rewards, policies, and value functions. Financial environments make each component challenging; Chapter 4 specifies the features, reward, and network architectures used here. Two algorithmic distinctions—model-free versus model-based and on-policy versus off-policy—then frame the progression from early value-based methods to TD3 for continuous-control trading.

3.1. Components of a Reinforcement Learning System

Reinforcement learning systems address sequential decision-making problems by selecting actions that maximize cumulative discounted future rewards. This subsection defines the core components using trading as the primary illustrative context, following the standard treatment of reinforcement learning (Sutton & Barto, 2018).

Formally, the trading problem can be described as an approximate Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition kernel, $R(s, a, s')$ is the reward function, and γ is the discount factor. In financial markets, this formulation is necessarily an approximation. The agent observes engineered order-book features rather than the full market state, so the empirical problem is better understood as a partially observed decision problem represented through an approximate MDP. Relevant information may be omitted because of hidden liquidity, unobserved order-flow history, latent intraday regimes, queue priority, and other market participants' private decisions. The transition process is also affected by non-stationarity and intraday seasonality. The state representation used in this thesis should therefore be understood as an attempt to construct a useful decision state under partial observability, not as proof that the market itself is fully Markovian.

Environment. The environment defines the space of possible states and the rules governing transitions between them. In the trading setting studied here, the environment is the simulated order-book market: it receives a portfolio-exposure action from the agent at each time step, updates the portfolio value according to price changes and transaction costs, and returns the new state and reward. The environment encapsulates everything outside the agent's direct control — price dynamics, spread, and execution frictions represented by the simulator — under a set of explicit simulation assumptions.

State. The state s_t is a snapshot of the environment at time t that contains sufficient information for the agent to select an action. In the HFT setting, the state is a vector of engineered microstructure features derived from the limit order book: imbalance measures, spread, order flow signals, and temporal encodings. States can be terminal, ending an episode, or non-terminal, continuing the interaction sequence. The Markov property requires that the optimal action depend only on s_t and not on the full history; the feature engineering in Chapter 4 is designed to approximate this property.

Chapter 4.2 discusses how specific microstructure features are selected and normalized to construct a useful decision state under this partial observability.

Action. The action a_t is the decision the agent takes given state s_t. In this thesis the action space is continuous: $a_t \in [-1, 1]$, representing the target portfolio exposure from maximum short to maximum long. The continuous formulation is important because it allows the agent to express varying degrees of conviction through position size, rather than being restricted to a binary buy/sell decision.

Policy. The policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ maps states to actions. A deterministic policy is used — given the same state, the policy always produces the same action. A deterministic policy is chosen here because the evaluation objective is a single exposure decision per state, and consistency is important for interpretability and execution in deployed trading systems. During training, exploration noise is added to the policy output to encourage discovery of profitable behaviors; at evaluation time the policy is used without noise.

The choice between deterministic and stochastic policies involves a trade-off that is particularly salient in trading environments. Stochastic policies can provide more flexible exploration and robustness to uncertainty, but they require selecting between sampling or taking the mode at deployment time. Deterministic policies eliminate this ambiguity but require explicit exploration mechanisms. Chapter 4.5 compares both approaches (PPO's stochastic policy vs. DDPG/TD3's deterministic policy) in the context of the trading problem.

Reward. The reward r_t is a scalar signal returned by the environment after the agent takes action a_t in state s_t. It provides the only learning signal available to the agent — there are no labels, no supervisor. The reward function is therefore a critical design choice: it determines what the agent actually optimizes. In trading, rewards are typically derived from portfolio returns, possibly adjusted for risk and transaction costs. The specific reward formulation used here is developed in Chapter 4.

Reward misspecification is a fundamental risk in trading applications. A naive reward function that optimizes raw returns without accounting for risk, transaction costs, or tail risk can lead to pathological behaviors such as excessive turnover, overconcentration in volatile positions, or ignoring rare but catastrophic events. The Differential Sharpe Ratio formulation used in Chapter 4.3 addresses this by providing a risk-adjusted learning signal that penalizes volatility while remaining recursively computable for online learning.

Value Function. The value function $V^\pi(s)$ estimates the expected cumulative discounted reward starting from state s under policy π:

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s\right] \tag{1}$$

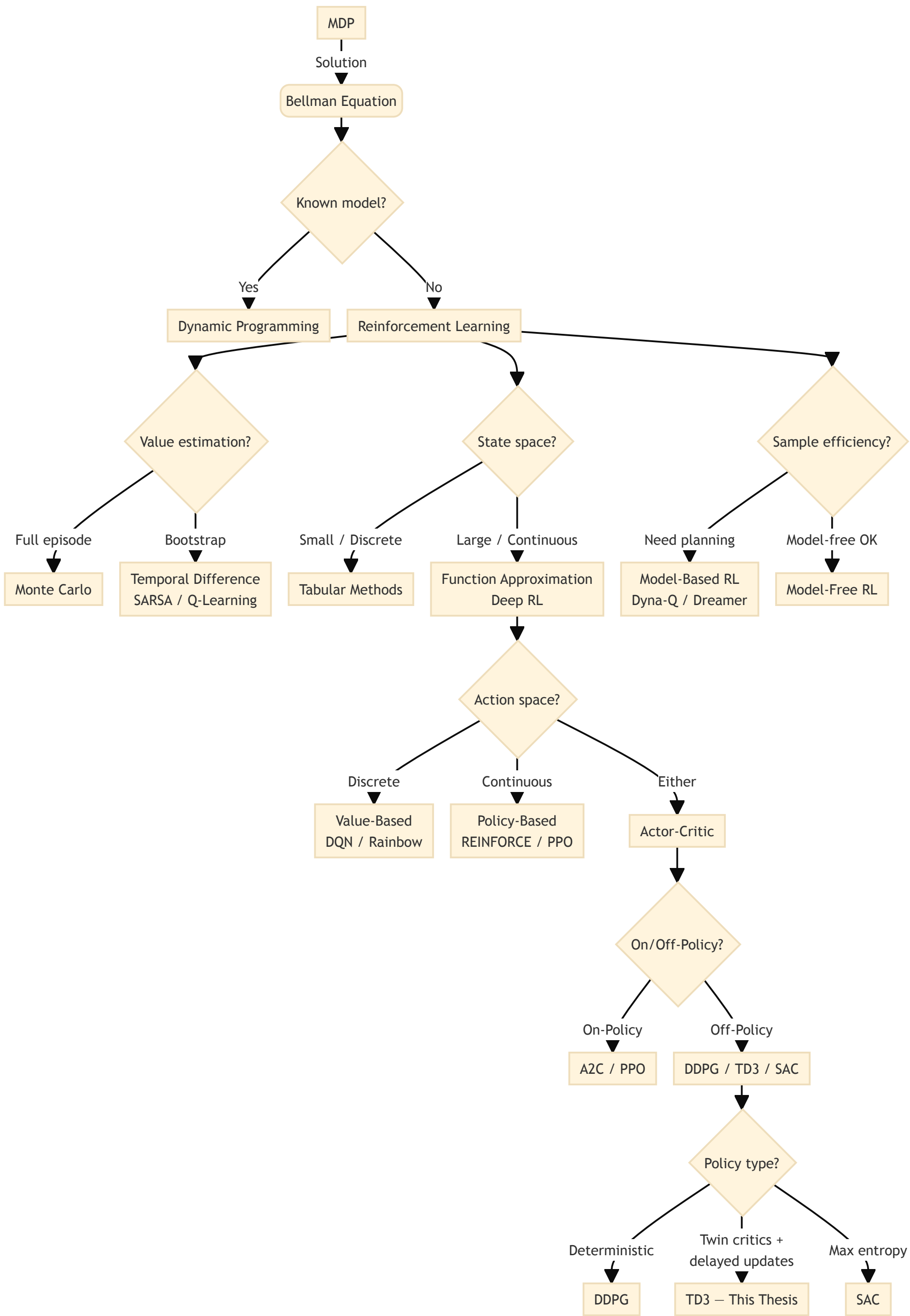
where: - $V^\pi(s)$ — expected cumulative discounted return starting from state s under policy π - $\gamma \in [0, 1]$ — discount factor - r_{t+k} — reward received k steps after t - $\mathbb{E}_\pi$ — expectation over trajectories generated by following π from $s_t = s$

The value function allows the agent to reason about long-run consequences rather than only immediate rewards — a necessary capability in trading, where the cost of a position change today affects future returns through transaction costs and portfolio path dependence. The action-value function $Q^\pi(s, a)$ extends this to state-action pairs and forms the basis of the critic networks in actor-critic algorithms. This cumulative discounted reward term appears explicitly in the RL objective in Equation 16.

3.2. Classifying RL Algorithms

The diagram below maps the reinforcement learning landscape from the Bellman equation through the key algorithmic relaxations that lead to TD3. It is intended as an orientation before the detailed treatment that follows.

Figure 2. RL algorithm taxonomy from Bellman equation to TD3



Source: *Author's own diagram, based on (Sutton & Barto, 2018).*[5]

[5]This taxonomy is a pedagogical simplification, not a precise classification. Categories have fuzzy boundaries: PPO appears under both "Policy-Based" and "Actor-Critic" because it combines a stochastic policy with a value-based critic; DDPG, TD3, and SAC share an "Off-Policy" branch despite fundamental differences in policy type and objective. Readers should not infer mutually exclusive algorithmic families from the crisp boundaries shown here.

3.2.1. Model-free vs Model-based

Reinforcement learning algorithms divide into two broad families depending on whether they require a model of the environment.

Model-based methods learn or are given a transition model $P(s' \mid s, a)$ and use it to plan ahead — computing value functions by rolling out simulated trajectories rather than purely from observed data. Dyna-Q and AlphaZero are canonical examples. When the model is accurate, planning is data-efficient; when the model is wrong, planning amplifies errors. In financial markets, transition dynamics are unknown and non-stationary, making a reliable model practically unattainable.

Model-free methods learn directly from sampled transitions without building a transition model. They are less sample-efficient, and they still rely on the sampled experience being informative for the target environment and on the function approximator generalizing usefully across states and actions. TD3 belongs to this family.

3.2.2. Value-Based, Policy-Based, and Actor-Critic

A second dimension classifies algorithms by what they learn and optimize.

Value-based methods learn a value function and derive a policy implicitly by acting greedily with respect to it. Q-learning and its deep variant DQN are prototypical examples (Mnih et al., 2015). They work well in discrete action spaces but struggle when actions are continuous, because the greedy step $\arg\max_a Q(s, a)$ becomes intractable over an uncountable set.

Policy-based methods directly parameterize and optimize the policy π_θ. REINFORCE updates parameters by gradient ascent on expected return (Williams, 1992):

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}_{\pi_\theta}\left[\sum_t r_t\right] \tag{2}$$

where:

- θ — policy parameters
- $\alpha > 0$ — learning rate
- ∇_θ — gradient with respect to θ
- r_t — reward at step t

Policy gradients handle continuous actions naturally but suffer from high variance, requiring many samples to obtain a reliable gradient signal. This variance problem motivates the actor-critic architecture, which introduces a critic network to provide lower-variance value estimates.

Actor-critic methods combine both: an actor μ_ϕ selects actions and a critic $Q_\theta(s,a)$ evaluates them, using the critic's lower-variance estimates to guide actor updates. The full actor-critic derivation and TD3's specific extensions are given in Section 3.3.

3.2.3. On-Policy vs Off-Policy

On-policy methods evaluate and improve the same policy used to collect experience. SARSA is a canonical example, updating with:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \tag{3}$$

where:

- $Q(s,a)$ — action-value function
- $\alpha > 0$ — learning rate
- r_t — reward received after taking action a_t in state s_t
- $\gamma \in [0,1]$ — discount factor
- a_{t+1} — next action drawn from the current policy

On-policy methods tend to be stable but wasteful: each sample is used once and then discarded.

Off-policy methods decouple the data-collection policy (the behavior policy) from the policy being improved (the target policy). Q-learning updates with the greedy maximum:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \tag{4}$$

where:

- $\max_a Q(s_{t+1}, a)$ — greedy value of the next state, taking the maximum over all possible actions a
- remaining symbols are as in Equation 3

This is contrasted with SARSA's update in Equation 3, where the next action is sampled from the current policy rather than taking the maximum. This separation allows experience from a replay buffer — collected under earlier or exploratory policies — to be reused many times, regardless of how a_{t+1} was actually selected. In a financial simulator, however, replay does not create counterfactual observations of how the market would have evolved under actions not actually taken. It is valid only under the maintained simulation assumption that the agent is price-taking and that its exposure choice affects portfolio wealth but not the subsequent order-book path. TD3 is off-policy and uses a replay buffer to reuse collected transitions under this simulator design; the implications for sample efficiency and execution assumptions in financial settings are discussed in Section 3.3.

3.3. Actor-Critic Methods for Continuous Control

Reinforcement learning algorithms have advanced through conceptual breaks, each addressing a limitation of its predecessor. The lineage leading to TD3 isolates the properties relevant to the continuous-action, single-asset trading problem formulated in Chapter 4.

3.3.1. Value-Based Methods and Their Limitations in Continuous Control

The foundational algorithms of the field — Q-learning (Watkins & Dayan, 1992) and its on-policy variant SARSA (Sutton & Barto, 2018) — operate over tabular representations of state-action value functions. Both methods converge to the optimal policy under mild conditions, but only when the state-action space is small enough to enumerate explicitly. In a trading context with continuous price-derived features, the state space is effectively infinite, and a tabular representation is computationally infeasible.

Deep Q-Networks (DQN) (Mnih et al., 2015) resolved the scalability problem by replacing the lookup table with a deep neural network, enabling generalization across states. Two architectural innovations were critical: experience replay, which decorrelates training samples by storing and randomly resampling past transitions, and a periodically frozen target network, which stabilizes the Bellman update by preventing the target from shifting at every gradient step. DQN achieved human-level control across a wide range of tasks, but it inherits a structural constraint from Q-learning: it requires a discrete action space, because the max operator in the Bellman target presupposes that all actions can be enumerated. For a trading agent that outputs a continuous portfolio weight in $[-1, 1]$, this is not an acceptable limitation. Discretizing the action space into a finite grid resolves the representation issue formally, but it reintroduces the granularity problem: a coarse grid loses the benefits of continuous control, while a fine grid makes the maximization step computationally expensive and recovers the curse of dimensionality in higher-dimensional portfolio settings.

3.3.2. Policy Gradient Methods and the Actor-Critic Architecture

A distinct class of algorithms avoids the maximization problem by parameterizing the policy directly and optimizing it through gradient ascent on expected cumulative reward. The foundational result establishes that policy parameters can be updated by gradient ascent on expected cumulative reward: REINFORCE introduced this approach (Williams, 1992), and the policy gradient theorem was established in the general function approximation setting (Sutton et al., 2000). The gradient takes the form:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t] \tag{5}$$

where:

- $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_t r_t]$ — expected return objective
- $\pi_\theta(a_t \mid s_t)$ — probability of action a_t under the current policy
- $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$ — discounted return from step t
- $\nabla_\theta \log \pi_\theta$ — score function (log-policy gradient)

This generalizes the simpler form shown in Equation 2, which presents the parameter update rule directly rather than deriving it from the gradient of the objective. While REINFORCE applies naturally to continuous action spaces through a parameterized distribution (for example, a Gaussian policy), it suffers from high variance in the gradient estimate because G_t is computed from a full episode trajectory, making it sensitive to trajectory length and reward scale.

The actor-critic architecture reduces this variance by replacing G_t with a learned advantage estimate $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, maintained by a separate critic network. The actor updates the policy in the direction of higher-valued actions, while the critic provides a lower-variance baseline. This separation is the architectural foundation of all subsequent continuous-action algorithms, including TD3, which instantiates this architecture with deterministic policies and twin critics as described in Equation 21 and Equation 20.

3.3.3. DDPG: Deterministic Policies for Continuous Control

The deterministic policy gradient (DPG) theorem was established, showing that for a deterministic policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ the policy gradient takes the form (Silver et al., 2014):[6]

$$\nabla_\phi J(\phi) = \mathbb{E}_s \left[\nabla_a Q_\theta(s,a)\big|_{a=\mu_\phi(s)} \cdot \nabla_\phi \mu_\phi(s) \right] \tag{6}$$

where:

- $J(\phi)$ — expected return
- $Q_\theta(s,a)$ — critic (action-value function) parameterized by θ
- $\mu_\phi(s)$ — deterministic actor
- $\nabla_a Q_\theta$ — gradient evaluated at the action produced by the current policy

This deterministic gradient is a specialization of the stochastic policy gradient in Equation 5 for policies that output a single action rather than a distribution. TD3 uses this gradient form as shown in Equation 21.

Deep Deterministic Policy Gradient (DDPG) was then introduced (Lillicrap et al., 2016), applying the DPG theorem (Silver et al., 2014) to deep neural network function approximators and replacing the tabular actor and critic with multi-layer networks. Rather than sampling an action from a distribution, the actor outputs a single action $\mu_\phi(s)$ directly, making the gradient computation tractable over a continuous action space without the intractable arg max required by value-based methods.

DDPG inherits the experience replay and target network mechanisms from DQN, making it an off-policy algorithm and therefore sample-efficient. However, Two failure modes in DDPG were identified (Fujimoto et al., 2018), both corroborated by independent reproducibility studies (Henderson et al., 2018). First, the critic tends to overestimate action values because actor optimization can exploit positive approximation errors in the critic, and bootstrapped targets can propagate those overestimated values through subsequent updates. Overestimated Q-values then push the policy toward actions that exploit the critic's errors rather than the true reward signal. Second, the algorithm is sensitive to the relative frequency of actor and critic updates: updating the actor before the critic has converged amplifies the overestimation problem. In financial

[6] The original DPG paper uses θ for the actor and w for the critic (Silver et al., 2014); the DDPG paper uses θ^μ and θ^Q (Lillicrap et al., 2016). This thesis adopts the TD3 convention throughout — ϕ for the actor and θ for the critic(s) — to maintain consistency with the TD3 equations in Chapter 4 (Fujimoto et al., 2018).

environments, where rewards are noisy and non-stationary, one would expect these instabilities to be particularly acute: the critic must separate genuine market signal from approximation artifacts, and overestimated Q-values provide a corrupted learning signal under conditions where the true reward is already weakly informative.

3.3.4. PPO: On-Policy Actor-Critic with Clipped Objectives

Proximal Policy Optimization (PPO), an on-policy algorithm that balances the size of each policy update against training stability, was introduced in (Schulman et al., 2017). PPO optimizes a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big] \tag{7}$$

where:

- $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ — probability ratio between the updated and old policies[7]
- $\hat{A}_t$ — advantage estimate
- ϵ — clipping hyperparameter (typically 0.2) limiting how far the new policy may deviate from the old one in a single update

This contrasts with TD3's approach to policy updates, which uses the deterministic gradient in Equation 6 and Equation 21 without explicit clipping constraints. PPO's clipping mechanism provides stability through objective constraints, while TD3 achieves stability through target smoothing and delayed updates.

The clipping mechanism prevents excessively large policy updates that would collapse performance, making PPO more robust to hyperparameter choices than earlier policy gradient methods such as Vanilla Policy Gradient (Sutton et al., 2000) and Trust Region Policy Optimization (TRPO) (Schulman et al., 2015). Unlike TD3 and DDPG, PPO is on-policy: it optimizes against the most recent policy and does not use a replay buffer. This trades sample efficiency for stability, which can be advantageous in environments where the value function is difficult to approximate.

PPO also maintains a stochastic policy rather than the deterministic policies used by DDPG and TD3. At each state, the policy outputs a distribution over actions (typically Gaussian), and

[7]The original PPO formulation uses $r_t(\theta)$ for this ratio (Schulman et al., 2017), but that symbol is reserved throughout this thesis for the RL reward signal. $\rho_t(\theta)$ is used here to avoid the conflict.

actions are sampled from this distribution. Exploration is handled naturally through the entropy of the action distribution, which can be explicitly encouraged through an entropy bonus term in the objective:

$$L^{\text{CLIP+ENT}}(\theta) = L^{\text{CLIP}}(\theta) + c_1 \mathcal{H}(\pi_\theta) \tag{8}$$

where:

- $L^{\text{CLIP}}(\theta)$ — clipped surrogate objective from Equation 7
- $\mathcal{H}(\pi_\theta) = -\mathbb{E}[\log \pi_\theta(a \mid s)]$ — policy entropy
- $c_1 > 0$ — coefficient controlling the strength of the exploration bonus

This entropy-based exploration contrasts with the additive action noise used by DDPG and TD3 in Equation 12.

For trading applications, PPO's stochastic policy provides built-in exploration without requiring explicit noise injection during training, and its clipped objective helps avoid catastrophic performance drops that can occur in high-variance financial environments. However, its on-policy nature means each transition is used only once before being discarded, which requires more simulator transitions for comparable learning. In the present thesis, where the training window is fixed to a specific historical period, this makes on-policy methods less efficient than replay-based alternatives at extracting signal from a bounded simulation budget.

3.3.5. TD3: Addressing the Instability of Deterministic Actor-Critic Methods

TD3 was introduced as a direct response to DDPG's failure modes, proposing three targeted mechanisms (Fujimoto et al., 2018).

Clipped double Q-learning. Two independent critic networks Q_{θ_1} and Q_{θ_2} are maintained, and the Bellman target is constructed using the minimum of their estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1}) \tag{9}$$

where:

- r_t — immediate reward
- γ — discount factor
- $Q_{\theta'_i}$ — two target critic networks with slowly updated parameters θ'_i

- s_{t+1} — next state
- $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$ — target action with smoothing noise ϵ_{smooth}

This differs from the standard Q-learning target in Equation 4 through two key modifications: (1) the minimum operator over two critics reduces overestimation bias, and (2) the deterministic target action $\tilde{a}'$ replaces the maximum over actions. The TD3 instantiation in Chapter 4.4 uses the same formulation, shown in Equation 20.

Taking the minimum provides a conservative estimate of target value and substantially reduces overestimation bias. This is analogous to the double Q-learning correction (Hasselt et al., 2016), adapted to the deterministic actor- critic setting.

Delayed policy updates. The actor and target networks are updated less frequently than the critics — typically once for every two critic updates. This allows the value estimates to stabilize before they are used to compute the policy gradient, breaking the feedback loop between an unstable critic and an erroneous actor update.

Target policy smoothing. When computing Bellman targets, noise is added to the target action $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$, with ϵ_{smooth} clipped to a small interval. This prevents the critics from fitting sharp value peaks caused by function approximation, acting as a regularizer on the value landscape.

Together, these three modifications produce an algorithm that is substantially more reliable than DDPG on standard continuous control benchmarks (Fujimoto et al., 2018). Evidence from trading-adjacent settings is more limited and heterogeneous: Several deep RL strategies were compared in asset-holding tasks, finding TD3 competitive with alternatives (Mohammadshafie et al., 2024); TD3 was evaluated in cryptocurrency trading with explicit attention to backtest overfitting (Gort et al., 2022). These results should not be read as establishing a general ranking — they span different asset classes, reward formulations, and evaluation protocols — but they suggest TD3's stability mechanisms are not systematically harmful in financial settings. This matters for the present thesis for a specific reason: the agent is trained in a price-taking historical-market simulator where each transition can be replayed because the agent's action changes portfolio wealth but not the subsequent order-book path. Replay therefore improves computational use of the simulated experience, but only under this execution assumption; it should not be interpreted as evidence that the agent has observed all counterfactual market reactions to alternative trades.

TD3 is selected on the basis of two constraints imposed by the thesis design and one preference that follows from them.

The action space is continuous — the agent outputs a target portfolio exposure in $[-1, 1]$ — which eliminates the DQN family entirely, since a discretized action grid would make position sizing a modelling artifact rather than a learned decision, restricting the viable candidates to policy-gradient and actor-critic algorithms. The training window is also bounded by a fixed historical period, with live data collection outside the thesis's scope; off-policy algorithms that reuse transitions through a replay buffer extract more signal from each simulated step than on-policy methods, which disadvantages PPO relative to DDPG and TD3.

Given these constraints, the viable candidates are DDPG, TD3, and SAC (Haarnoja et al., 2018). DDPG and TD3 are preferred over SAC because they use natively deterministic policies, matching the evaluation objective of a single exposure decision per state and avoiding the choice between sampling from or taking the mode of a stochastic distribution at deployment.

SAC is excluded on methodological scope grounds rather than irrelevance. Its entropy-temperature coefficient introduces an additional objective dimension — trading off return against policy entropy — that is absent from the return-only formulation used here; this entropy-augmented framework has been applied to trading problems in the literature (Liu et al., 2020). Comparing SAC to TD3 would require disentangling whether any improvement comes from the maximum-entropy objective or from the off-policy replay both algorithms share, an ablation outside this thesis's focus on value-estimation corrections for deterministic policies. SAC remains a promising direction for future work, particularly its online variants for regime shifts in non-stationary markets.

Between DDPG and TD3, TD3 is preferred because it was introduced precisely to correct DDPG's two failure modes — critic overestimation and policy-update instability from premature actor updates — both of which are expected to be acute when rewards are noisy and the critic must separate genuine market signal from approximation artifacts. TD3 is therefore not chosen as a generally superior trading method, but as the algorithm whose structural properties — off-policy replay, deterministic actor, explicit overestimation corrections — most directly match this thesis's simulation design.

This choice should be read together with the trading constraints discussed in Chapter 2, which motivate continuous exposure control, off-policy learning from historical data, and caution about transaction costs, non-stationarity, and partial observability. TD3 satisfies the first two through deterministic continuous actions and replay-based learning; it does not solve the remaining market-design problems, which the empirical chapters address separately.

3.3.6. Algorithm Comparison: PPO, DDPG, and TD3 for Trading

The preceding sections establish how PPO, DDPG, and TD3 differ in learning paradigm, policy type, and stability mechanism, and why TD3 is selected under this thesis's constraints. Table 1 in Chapter 4 summarizes these differences at the implementation level — actor output, exploration noise, target smoothing, and update frequency — as configured for the controlled comparison in Chapter 6, where all three algorithms are evaluated on identical trading tasks.

4. Design of the Trading Agent

The RL framework from Chapter 3 defines seven components of the single-asset HFT agent: action space, state representation, reward function, value function, policy, discount factor, and step size. The trading problem's structure and constraints motivate each choice.

4.1. Action Space

The action space defines the set of permissible decisions available to the trading agent at each time step. This design choice fundamentally shapes the agent's behavior and determines the granularity of control over portfolio positions.

4.1.1. Continuous vs Discrete Actions

Trading systems can be formulated with either discrete or continuous action spaces:

Discrete action spaces restrict the agent to a finite set of categorical decisions. The two most common formulations are the three-action set $\{-1, 0, 1\}$ representing short, neutral, and long positions, and the two-action set $\{0, 1\}$ representing only short and long positions without a neutral option.

Continuous action spaces allow the agent to output any value within a specified range, enabling smooth portfolio allocation decisions.

4.1.2. Implementation: Box Portfolio Action Space

In this implementation, the action space is defined as a continuous interval:

$$\mathcal{A} = [-1, 1] \tag{10}$$

This continuous action space is required for deterministic policy gradient methods like DDPG and TD3, which are designed for problems where the agent outputs a single action rather than choosing from a discrete set. The policy that maps states to actions in this space is formalized in Equation 23.

where the action $a_t \in \mathcal{A}$ at time t represents the target portfolio allocation for the traded asset. The semantic interpretation is:

- $a_t = 1.0$: Maximum long exposure (fully invested long position)
- $a_t = 0.0$: Neutral position (no exposure, cash equivalent)
- $a_t = -1.0$: Maximum short exposure (fully invested short position)
- $a_t \in (0, 1)$: Partial long position scaled by magnitude
- $a_t \in (-1, 0)$: Partial short position scaled by magnitude

This formulation supports short-selling and is a standard simplification in single-asset RL trading environments applied to equity data. The simulation treats long and short positions as cost-symmetric: the transaction cost term $c \cdot |a_t - a_{t-1}|$ applies identically regardless of direction. In practice, short positions in these equities incur securities-lending borrow costs — typically 25–75 basis points per year on the notional short — that are not captured here. For strictly intra-session episodes, the per-tick borrow cost is negligible: at 50 bps annualized and a 10-second average hold, the cost is on the order of 10^{-7} of notional. The cost-symmetry assumption is therefore inconsequential provided episodes do not span overnight. Where the agent holds short positions across sessions, reported returns represent an upper bound on achievable short-side performance. The continuous nature enables the agent to express varying degrees of conviction—for example, $a_t = 0.3$ indicates a moderately bullish stance, while $a_t = -0.8$ signals strong bearish conviction.

4.1.3. Action Constraints and Transaction Costs

The action space is bounded to prevent unrealistic leverage. In practice, unbounded actions could lead to positions exceeding available capital or margin requirements. The $[-1, 1]$ range implicitly assumes unit leverage without margin multipliers.

Transaction costs are applied proportionally to position changes. If the agent transitions from a_{t-1} to a_t, the turnover magnitude is $|a_t - a_{t-1}|$, and fees are calculated as:

$$\text{fee}_t = c \cdot |a_t - a_{t-1}| \cdot \text{notional value} \tag{11}$$

where:

- c — transaction cost rate (trading fees per unit of notional turnover), matching the penalty term in Equation 15

This cost term appears in the reward function as the penalty $c \cdot |a_t - a_{t-1}|$ shown in

Equation 15. The agent must learn to balance expected returns from position adjustments against this turnover cost. This incentivizes the agent to minimize unnecessary portfolio rebalancing, as frequent adjustments erode returns. During training, the RL algorithm must learn to balance the potential gains from reallocation against the friction costs incurred.

4.1.4. Action Smoothing and Exploration Noise

During training, exploration is introduced through additive Gaussian noise:

$$a_t^{\text{noisy}} = \text{clip}(a_t + \epsilon, -1, 1) \tag{12}$$

where:

- $\epsilon \sim \mathcal{N}(0, \sigma^2)$ — Gaussian exploration noise
- σ — exploration noise standard deviation[8]

DDPG and TD3 use this mechanism during data collection; at evaluation they apply the unperturbed deterministic policy. PPO instead obtains action diversity from its stochastic policy and entropy bonus (Equation 8). Clipping keeps the exploratory actions within the valid bounds.

TD3 separately applies target policy smoothing during critic updates, as specified with the policy implementation in Section 4.5.1.1.

4.2. State Space

The state space defines the informational representation available to the agent at each decision point. At each time step t, the agent receives an observation vector $s_t \in \mathcal{S}$ constructed from the current state of the limit order book (LOB). The design of this observation is the primary engineering decision in the empirical pipeline: the agent can only exploit structure that is present in its input.

4.2.1. Design Requirements

An effective state representation for high-frequency trading must satisfy several criteria simultaneously.

[8] The main experiment uses $\sigma = 0.3$. This value is at the upper end of the 0.1–0.3 range commonly used in continuous-action TD3 applications and was chosen to maintain sufficient action diversity in the low-signal tick-level setting.

Approximation of the Markov property. The TD3 algorithm assumes that s_t contains sufficient information such that the optimal action depends only on the current state, not on the full history. Raw price levels violate this assumption because their unconditional distribution is non-stationary; a price of 180 means something very different depending on recent trajectory. Engineered features — returns, imbalances, normalized spreads — are designed to be more stationary and to encode the local market state rather than the absolute price level.

Informativeness about short-term price direction. For the reward signal to be learnable, the observation must contain features that are predictive of near-term mid-price changes. The selection is therefore grounded in market microstructure theory, which identifies specific LOB quantities — imbalances, order flow, spread — as the empirically dominant predictors of short-term price movement.

Dimensionality discipline. Each additional observation dimension enlarges the input space that the policy and critic networks must generalize over. Features are included only when they carry a distinct theoretical mechanism not already captured by other features in the set.

Scope and timescale consistency. The observation is restricted to LOB-derived features. Bar-based signals such as OHLCV aggregates or technical indicators require a fixed aggregation window, imposing a coarser timescale than the tick-level decisions being made: a one-second bar collapses dozens of individual order-book events into a single price summary, discarding the queue dynamics, imbalance shifts, and spread fluctuations that constitute the decision-relevant information at this resolution. Cross-asset signals such as VIX or index returns face a related problem — they update at frequencies far below the inter-event times of the constituent tick streams, contributing no new information between consecutive book updates, and aligning multiple tick streams at sub-second precision introduces synchronization risk, since a VIX observation and a LOB snapshot sharing a timestamp to the second are not the same market moment.

Beyond these practical constraints, restricting the state to single-asset LOB features is a deliberate experimental choice: the thesis tests whether microstructure-aware features alone are sufficient for a viable agent, since adding cross-asset signals would answer a different question and prevent clean attribution of any observed performance to LOB structure. Augmenting the observation with synchronized cross-asset regime indicators once baseline LOB-only performance is established is a natural extension left for future work (Chapter 7).

4.2.2. Feature Groups and Selection Rationale

The feature registry organises the LOB candidate pool into five groups, each targeting a distinct microstructure mechanism. **Imbalance features** measure the relative weight of resting liquidity on each side of the book — at individual levels (levels 0–2) and in a three-level volume-weighted aggregate — together with order count imbalance, which is sensitive to fragmentation that a single large institutional order would mask (Biais et al., 1995; Cao et al., 2009). **Fair value features** estimate where the next transaction is likely to occur relative to the mid-price: the microprice (Stoikov, 2018) is the central construct, complemented by its divergence from mid-price and a volume-weighted mid-price skew across three levels. **Spread and cost features** capture adverse selection risk through the bid-ask spread in basis points, its ratio to a 50-event rolling mean, book convexity on each side, and depth slopes that proxy market impact per unit of volume (Bouchaud et al., 2002; Glosten & Milgrom, 1985; Kyle, 1985). **Order flow features** measure net directional pressure from LOB events between consecutive snapshots via order flow imbalance (Cont et al., 2014) and a 50-event rolling sum, queue depletion on each side, signed trade flow from the tape (C. M. C. Lee & Ready, 1991), and odd-lot trade ratio and imbalance, which separate retail and algorithmic small-lot activity from institutional flow and thereby signal changes in adverse selection risk.[9] **Regime and temporal features** condition the agent on activity rate via log inter-event time (Engle, 2000), on price momentum via mid-price acceleration, and on the intraday seasonal pattern via cyclical sine and cosine hour encodings (Wood et al., 1985). The final selected TD3 configuration uses a reduced subset of this candidate pool: best-level book pressure, three-level order-book imbalance, best-level order-count imbalance, microprice, microprice divergence, bid and ask depth slopes, instantaneous OFI, 50-event rolling OFI, and 50-event signed trade flow. The observation vector is additionally extended at runtime with the agent's current portfolio exposure $a_{t-1} \in [-1, 1]$, making the state partially Markovian with respect to inventory and allowing the agent to condition position changes on the transaction cost they would incur; this feature is excluded from z-score normalization because its bimodal distribution — concentrating near ± 1 under the continuous action space — makes standardization

[9] Odd-lot activity as a retail/institutional separator is a distinct empirical question from the price-improvement-based retail-identification method (Boehmer et al., 2021); the latter is not direct evidence for the odd-lot mechanism used here. Odd-lot share is computed from the February 25–27, 2026 MBP-10 dataset across AAPL, MSFT, TSLA, META, AMZN, and AVGO and should not be interpreted as a long-run population statistic. The share of odd-lot trades in large-cap Nasdaq equities has increased substantially since the SEC's 2023 amendments to odd-lot reporting rules. See Chapter 5.1 for the MBP-10 format definition.

counterproductive relative to the raw bounded signal.

The candidate pool from which these groups were drawn contained over forty computable statistics. Features eliminated by the IC/ICIR filter or excluded on redundancy grounds include: VPIN (Easley et al., 2012), which proved collinear with rolling OFI after conditional screening; cancel-to-trade ratio and trade arrival rate, which carried insufficient incremental ICIR beyond order count imbalance; OFI autocorrelation, whose predictive content was subsumed by the rolling OFI window; large trade ratio, which was redundant with the odd-lot decomposition; and depth ratio and price vamp, which did not survive the theory-constrained Stage 1 screen. Exact formulas, citations, parameter values, and model-inclusion status for both selected and excluded features are catalogued in Appendix A.

4.2.3. Observation Space and Normalization

The complete observation vector $s_t \in \mathbb{R}^d$ is formed by concatenating all engineered features. The main selected-feature TD3 experiment uses ten static LOB-derived features plus the runtime position feature, giving $d = 11$.

The ten static features selected in the main TD3 configuration are standardized with causal running statistics:

$$\tilde{x}_{i,t} = \frac{x_{i,t} - \mu_{i,t}}{\sqrt{\sigma_{i,t}^2 + \varepsilon}} \tag{13}$$

where:

- $x_{i,t}$ — raw value of feature i at step t
- $\mu_{i,t}$ — running mean estimated from observations up to t
- $\sigma_{i,t}^2$ — running variance estimated from observations up to t
- $\varepsilon > 0$ — small constant for numerical stability

The mean $\mu_{i,t}$ and variance $\sigma_{i,t}^2$ are updated online from observations available up to timestep t within the current chronological split and trading session. The running statistics are reset at detected session boundaries, preventing previous-day market conditions from contaminating the opening representation of the next session. When cyclical encodings are used in other feature specifications, they are already bounded in $[-1, 1]$ and require no normalization, as with the position feature described above.

The formal observation space is:

$$\mathcal{S} = \mathbb{R}^d \tag{14}$$

In practice, after standardization, feature values concentrate in $[-3, 3]$ under approximate Gaussian assumptions. The policy and value networks must generalize to out-of-distribution observations that arise during stress episodes or regime shifts not present in the training sample.

4.2.4. On the Exclusion of Raw LOB Columns

Raw price and size columns — that is, the unprocessed bid and ask prices and sizes at each level — were deliberately excluded from the observation vector. Prior work in the deep learning tradition feeds the raw LOB matrix directly to the network (Zhang et al., 2019), allowing convolutional or recurrent architectures to discover structure from data without hand-engineered intermediaries. This is a coherent approach when the model itself is the feature extractor.

In the present setup, however, comprehensive microstructure features with established theoretical mechanisms are the primary input to the agent. Including raw columns alongside them introduces redundancy without adding an interpretable signal: normalized bid sizes at each level carry substantially the same information as the imbalance features derived from them, and normalized absolute prices at tick frequency are near-meaningless once the local market state is already encoded through microprice divergence, OFI, and spread measures. Mixing both paradigms would imply that the engineered features are insufficient, which is inconsistent with the selection rationale described above. The observation vector is therefore composed entirely of theoretically motivated derived features.

4.2.5. Feature Selection Process

Feature selection follows a two-stage procedure that combines theoretical constraints with empirical ranking.

Stage 1: Theory-constrained candidate pool. The LOB feature registry contains over forty computable statistics. The candidate pool is restricted to features that represent a distinct mechanism identified in the market microstructure literature. Each of the five groups — imbalance, fair value, spread, order flow, and regime — corresponds to a specific theoretical construct: queue pressure (Biais et al., 1995; Cao et al., 2009), fair value estimation (Stoikov, 2018), adverse selection cost (Glosten & Milgrom, 1985; Kyle, 1985), directional flow (Cont et al., 2014; C. M. C. Lee & Ready, 1991), and intraday regime context (Engle, 2000; Wood et al., 1985). Features not grounded in this literature — raw price and size columns, or purely data-driven aggregates without a microstructure interpretation — are excluded from consideration regardless of their statistical properties. This stage prevents data-mining by ensuring that every candidate has a prior economic justification before any data is examined.

Stage 2: IC/ICIR ranking within the theory-constrained pool. Within each group, multiple implementations of the same theoretical concept exist: OFI at different rolling horizons, spread ratios at different windows, imbalance at different depth levels. Theory cannot resolve which implementations are informative or which combinations are redundant; this is an empirical question. Each candidate is scored by its Information Coefficient Information Ratio (ICIR) — the mean IC divided by the standard deviation of IC over rolling windows of the training split, where IC is the Spearman rank correlation between the feature and the forward log-return proxy. In the pooled multi-symbol design, scoring is performed independently for each of the 18 training files (6 securities × 3 trading days). The per-file scores are then averaged across files before the redundancy filter is applied, so the final ranking reflects cross-symbol, cross-day average predictive content rather than a single symbol's idiosyncratic regime. Scoring and the conditional IC redundancy filter both operate exclusively on the training files so that the evaluation day (March 2, 2026) remains clean for model comparison. The validation IC is computed separately and reported as an out-of-sample stability check but plays no role in selection. Redundancy within the selected set is controlled via a conditional IC filter: after each feature is selected, its linear signal is regressed out of remaining candidates, and ICIR is recomputed on the residuals. A candidate is included only if it retains incremental predictive content beyond what the already-selected features collectively capture. The full procedure is described in Appendix F.

The result is a state space whose composition is jointly justified: every feature is grounded in theory (Stage 1) and empirically non-redundant within the theory-constrained pool (Stage 2). The two stages address different questions. Theory answers: which mechanisms belong in the state space? IC/ICIR answers: among the available implementations of those mechanisms, which combinations contribute distinct signal?

The selection is operationalized through a registry-driven feature pipeline. Feature names resolve to computation functions at runtime; adding, removing, or reordering features therefore changes the experimental specification without rewriting the feature implementations. The active feature set for the main experiment is the selected causal LOB set summarized in Appendix A: three imbalance features, two fair-value features, two depth-slope features, three order-flow features, and the runtime position feature. Regime and temporal features remain part of the implemented candidate registry but are not used in the selected TD3 specification.

4.3. Reward Function

The reward function determines what the agent actually optimizes. The thesis implements direct optimization of trading performance: the agent observes the market state, selects a continuous portfolio exposure $a_t \in [-1, 1]$, and receives a scalar reward reflecting realized return net of transaction costs. A baseline formulation of this reward is:

$$r_t = a_{t-1} \cdot \frac{P_t - P_{t-1}}{P_{t-1}} - c \cdot |a_t - a_{t-1}| \tag{15}$$

where:

- a_{t-1} — portfolio exposure held during the period
- P_t — price at the end of the period
- P_{t-1} — price at the beginning of the period
- $c \cdot |a_t - a_{t-1}|$ — transaction cost proportional to the size of the position change

The agent learns a deterministic policy $\mu_\phi : \mathcal{S} \to \mathcal{A}$ to maximize expected cumulative reward:

$$\max_{\phi} \mathbb{E}_{\mu_\phi} \left[\sum_{t=1}^{T} \gamma^{t-1} r_t \right] \tag{16}$$

where:

- ϕ — actor (policy) parameters being optimized
- T — episode horizon
- $\gamma \in [0, 1]$ — discount factor
- r_t — reward received at step t

This objective matches the standard RL formulation introduced in Equation 1, with the cumulative discounted reward as the quantity being optimized.

In trading, two properties are essential for the reward to work well in practice: the signal should be risk-adjusted so the agent does not maximize returns by taking excessive volatility, and it should be computable recursively so that no look-ahead into future returns is required at each training step. The baseline reward requires refinement at high frequency. The Differential Sharpe Ratio addresses both requirements.

4.3.1. Execution Model and the Signal–Friction Decomposition

Equation 15 separates into a **signal term**, $a_{t-1}\,(P_t - P_{t-1})/P_{t-1}$, and a **friction term**, $c\,|a_t - a_{t-1}|$ (the proportional fee of Equation 11). Throughout this thesis P_t is the **mid-price**, not an executable bid or ask quote, in keeping with the mid-price execution assumption stated in Section 5.2. Baseline runs set $c = 0$, so a buy and a sell settle at the same mid, no spread is crossed, and the friction term vanishes; the reward is then the signal term alone.

The features that carry the agent's directional signal — microprice, microprice divergence, order-flow imbalance, queue imbalance — are functions of top-of-book prices and sizes. The microprice is a convex combination of the best bid and ask, so its divergence from the mid satisfies $|\text{microprice}(t) - \text{mid}(t)| \leq (\text{ask}(t) - \text{bid}(t))/2$, the half-spread, at **every tick** (an exact algebraic bound, verified on the full sample; see Section 7.2). Because that divergence is the one-step-ahead predictor of the mid move, the largest **per-tick** edge this class of features can express is exactly one half-spread — the same quantity a spread-crossing fill would cost. Mid-price execution waives that cost, so the frictionless reward measures a **signal ceiling**: the most that could be captured if execution were free, not what a strategy that crosses the spread would realise. (This is a per-tick, next-mid-move bound on the tradable edge, not a claim that the features lack predictive value; Chapter 2 attributes them genuine predictive content over longer horizons.)

The transaction-cost sweep in Table 5 varies c and locates this ceiling against realistic cost: the reported edge is positive at zero fee and turns negative once the per-trade fee approaches a fraction of a basis point — the scale of the half-spread on these instruments. Hypothesis 1 is therefore an existence-and-magnitude claim (the signal is real, TD3 learns it, its size is of the order of one half-spread); Hypothesis 2 maps how performance responds as that cost is charged. The training reward uses the same mid-price valuation, so the learned policy is optimal for the frictionless objective; retraining under an explicit spread-crossing cost is left as future work (Section 7.2).

4.3.2. Why Raw Log-Return Is Insufficient

The simplest reward is the single-step log-return:

$$r_t = \log \frac{P_t}{P_{t-1}} \cdot a_{t-1} \tag{17}$$

where:

- P_t — mid-price at the end of the current step
- P_{t-1} — mid-price at the previous step
- $a_{t-1} \in [-1, 1]$ — portfolio exposure held during the period

This is the first term of Equation 15 without the transaction cost penalty. At tick frequency, this signal is dominated by microstructure noise. Individual price changes on an MBP-10 order book (defined in Chapter 5.1) are a fraction of the spread. The agent cannot distinguish genuine directional movement from quote flickering in a single step.

Training on raw log-return produces highly variable gradient estimates. It also tends to reward high-turnover behavior regardless of risk.

A related alternative is the Sharpe ratio computed over a fixed rolling window. This approach requires storing the full window of past returns at each step. It also introduces a sharp discontinuity when old returns drop out of it.[10]

Neither property is desirable in an online RL setting.

4.3.3. Differential Sharpe Ratio

The Differential Sharpe Ratio (Moody & Saffell, 1998) resolves both issues. It is defined as the first-order approximation of how the current portfolio log-return R_t changes the exponentially weighted Sharpe ratio:

[10] When the oldest return in a fixed window is unusually large or small, its removal produces an abrupt shift in the estimated mean and variance — and therefore in the Sharpe ratio — that is unrelated to the current action. The agent cannot distinguish this accounting artifact from a genuine change in performance, introducing spurious gradient variance. Exponential weighting in the DSR avoids this because old returns decay continuously rather than dropping off a cliff.

$$D_t = \frac{B_{t-1}\,\Delta A_t - \frac{1}{2}A_{t-1}\,\Delta B_t}{\left(B_{t-1} - A_{t-1}^2 + \varepsilon\right)^{3/2}} \tag{18}$$

where:

- A_{t-1} — exponentially weighted estimate of the first moment of returns (defined in Equation 19)
- B_{t-1} — exponentially weighted estimate of the second moment of returns (defined in Equation 19)
- $\Delta A_t = R_t - A_{t-1}$ — one-step increment of A
- $\Delta B_t = R_t^2 - B_{t-1}$ — one-step increment of B
- $(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}$ — running variance raised to the power $3/2$, stabilized by a small constant ε

This replaces the simple log-return in Equation 17 with a risk-adjusted measure that penalizes volatility. When DSR training is active, $r_t := D_t$ in Equation 16. The DSR reward signal addresses the high-variance problem that affects raw returns, though it does not eliminate the noise concerns at tick frequency.

A_t and B_t are exponentially weighted estimates of the first and second moments of returns:

$$A_t = \eta R_t + (1-\eta)\,A_{t-1}, \qquad B_t = \eta R_t^2 + (1-\eta)\,B_{t-1} \tag{19}$$

where:

- $R_t = \log(P_t/P_{t-1}) \cdot a_{t-1}$ — portfolio log-return at step t[11]
- $\eta \in (0,1]$ — exponential smoothing parameter
- A_t — running mean of returns
- B_t — running mean of squared returns

At $t = 0$, $A_0 = B_0 = 0$.

The smoothing parameter η controls the effective memory of the estimator: larger η makes the estimate more reactive to recent returns, smaller η averages over a longer history. The effective window length is approximately $1/\eta$ steps. In the empirical experiments, $\eta = 0.01$, giving an

[11] In the implementation, R_t is computed as $\log(\mathrm{NLV}_t/\mathrm{NLV}_{t-1})$, where NLV_t is the net liquidation value of the portfolio. This is equivalent to the formula above when transaction costs are zero. When costs are non-zero, the NLV-based return absorbs transaction costs implicitly, making the implementation more conservative than the analytic expression suggests.

effective memory of roughly 100 ticks — long enough to smooth microstructure noise while short enough to adapt to intraday regime shifts.

The DSR has two properties that make it well-suited to this setting. First, it penalizes return volatility intrinsically. A sequence of returns with the same mean but higher variance produces a lower DSR. This discourages the agent from achieving returns through high-variance position flipping.

Second, the DSR is recursively computable from A_{t-1} and B_{t-1} alone. This means no fixed window, no look-ahead, and no discontinuity as old observations expire.

A shaped reward such as DSR is used to stabilize learning. However, it is not equivalent to the financial performance metrics reported in Chapter 6.

The Sharpe ratio, annualized return, and drawdown statistics reported in the evaluation chapter are computed from the realized log-return series. These come from the deterministic policy rollout. They are not computed from the cumulative DSR accumulated during training.

Conflating the two would invalidate the empirical comparison. For example, treating a high training DSR as evidence of a strong Sharpe ratio on the test set would be incorrect.

The DSR approach has four limitations. First, it assumes the first and second moments of returns are approximately stationary over the effective memory window ($\approx 1/\eta$ steps); intraday trading is well documented to violate this through market opening effects, scheduled announcements, and regime shifts (Cont et al., 2014; Engle, 2000), and exponential weighting only partially compensates. Second, like other variance-based measures, it penalizes upside and downside volatility symmetrically, whereas traders typically care more about downside risk; the Sortino ratio, the Calmar ratio, or utility-based rewards would align better with that preference but are non-recursive or computationally intractable at tick frequency. Third, it can become unstable when the denominator $(B_{t-1} - A_{t-1}^2 + \varepsilon)^{3/2}$ is small — early in training or during low-variance regimes — amplifying microstructure noise into extreme reward values; the implementation clamps D_t to $[-10, 10]$, following standard reward-clipping practice (Mnih et al., 2015).[12] Fourth, because the running moments A_t, B_t are not part of the observation, the reward at time t depends on a hidden trajectory of past returns. TD3 samples replayed transitions out of order, so the same state-action pair can yield different rewards depending on when it was

[12] In the DSR scenario, the reward scale is $\kappa = 1$: at tick frequency the empirical variance term dominates $\varepsilon = 10^{-16}$ by several orders of magnitude, so the resulting signal falls naturally in a well-conditioned $\mathcal{O}(1)$ range for the Adam optimiser and Huber critic loss, and no ad-hoc reward rescaling is required.

collected, violating the stationarity its Bellman backup assumes — an effect strongest early in each episode, before the EMA moments stabilize after $\approx 1/\eta$ steps. Augmenting the observation with (A_t, B_t) would restore the Markov property and is left for future work.

The DSR was chosen despite these limitations for two reasons: its recursive formulation makes it uniquely suitable for online RL, where alternatives such as fixed-window Sharpe or utility-based rewards require storing unbounded history or solving optimization problems at each step; and while variance is an imperfect proxy for risk, it provides a tractable baseline that discourages the most pathological high-turnover behaviors. Whether an alternative reward formulation performs better is a question for future work, addressed in Chapter 7.

4.4. Value Function

The state-value function $V^\pi(s)$ and action-value function $Q^\pi(s, a)$ give the expected cumulative discounted return under policy π. TD3 implements the Q-function through design choices that distinguish it from a vanilla actor-critic.

4.4.1. Twin Critics

TD3 maintains two independent Q-networks Q_{θ_1} and Q_{θ_2}. The Bellman target is computed using the minimum of the two estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta_i'}(s_{t+1}, \tilde{a}_{t+1}) \tag{20}$$

where:

- r_t — immediate reward
- γ — discount factor
- $Q_{\theta_i'}$ — two target critic networks with slowly updated parameters θ_i'
- s_{t+1} — next state
- $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon_{\text{smooth}}$ — smoothed target action with $\epsilon_{\text{smooth}} \sim \mathcal{N}(0, \sigma^2)$ clipped to a small interval

Taking the minimum counteracts the overestimation bias that arises when bootstrapped targets are computed from the same network being updated — a failure mode documented in DDPG and addressed here following the TD3 formulation (Fujimoto et al., 2018). This formulation extends the standard Q-learning target in Equation 4 by (1) using twin critics to reduce overestimation and (2) employing a deterministic target policy rather than taking a maximum over actions. The TD3 target was first introduced in Chapter 3 in Equation 9.

Both critics are trained by minimizing the TD error against the shared target y_t. The actor is updated using only the first critic's gradient:

$$\nabla_\phi J(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\left[\nabla_a Q_{\theta_1}(s,a)\big|_{a=\mu_\phi(s)} \cdot \nabla_\phi \mu_\phi(s)\right] \tag{21}$$

where:

- ϕ — actor (policy) parameters being updated
- $J(\phi)$ — expected return
- $\mathcal{D}$ — replay buffer
- $Q_{\theta_1}(s,a)$ — first critic evaluated at the current policy action $\mu_\phi(s)$
- $\nabla_\phi \mu_\phi(s)$ — Jacobian of the policy with respect to its parameters

This is the deterministic policy gradient theorem (Silver et al., 2014), first presented in the general form in Equation 6. The gradient chain through the critic provides a lower-variance estimate of how policy parameters should change compared to the REINFORCE gradient in Equation 5.

The second critic contributes only through the conservative Bellman target, not directly to the policy gradient.

4.4.2. Loss Function

The critics are trained with Smooth L1 (Huber) loss rather than MSE:

$$\mathcal{L}(\delta) = \begin{cases} 0.5\,\delta^2, & |\delta| \le 1, \\ |\delta| - 0.5, & \text{otherwise.} \end{cases} \tag{22}$$

where:

- $\delta = y_t - Q_\theta(s_t, a_t)$ — TD error measuring the discrepancy between predicted and bootstrapped value estimates

The linear tail limits the influence of large TD errors that occur during volatile market episodes, providing robustness that MSE does not. This loss function is applied to the TD error derived from the Bellman target in Equation 20.

4.5. Policy

The policy maps the current state to an action. The three algorithms compared here — PPO, DDPG, and TD3 — differ in a fundamental property of this mapping: PPO maintains a probability distribution over actions and samples from it, while DDPG and TD3 produce a single deterministic action for each state. This distinction has direct consequences for how each algorithm explores during training, how it behaves at evaluation time, and what the policy gradient looks like. The subsections below describe each architecture, followed by a table that makes the design choices explicit.

4.5.1. Deterministic Policy Architecture

DDPG and TD3 implement a deterministic policy:

$$\mu_\phi : \mathcal{S} \to \mathcal{A}, \qquad a_t = \mu_\phi(s_t) \tag{23}$$

where:

- $\mathcal{S} = \mathbb{R}^{11}$ — observation space for the selected-feature TD3 configuration
- $\mathcal{A} = [-1, 1]$ — continuous action space
- ϕ — network parameters
- s_t — state at step t
- a_t — resulting deterministic action

The policy is a multi-layer perceptron. The input is the $d = 11$-dimensional state vector. The network passes this through two hidden layers and produces a scalar action:

$$s_t \in \mathbb{R}^{11} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^{1} \xrightarrow{\text{Tanh}} [-1,\ 1] \tag{24}$$

Hidden layers use ReLU activations, which avoid the vanishing-gradient problem that arises with saturating activations in deep hidden layers (Mnih et al., 2015). The output layer applies no activation before the Tanh squashing, which maps the pre-activation scalar to $(-1, 1)$. Tanh is preferred over a hard clip because it is differentiable everywhere: the policy gradient passes through the output non-linearity without discontinuity, and the smooth saturation near the action boundaries provides an implicit regularizer against extreme position changes.

The exported network widths are also recorded in the reproducibility specification in Appendix B.

4.5.1.1. DDPG and TD3 Exploration

During data collection, both algorithms apply the Gaussian exploration rule in Equation 12; the selected TD3 configuration uses $\sigma = 0.3$. At evaluation they instead use the unperturbed action $a_t^{\text{eval}} = \mu_\phi(s_t)$.

TD3 additionally applies independent target-policy smoothing only when computing Bellman targets (Equation 20; Equation 9). Its smoothing noise $\epsilon_{\text{smooth}} \sim \mathcal{N}(0, 0.2^2)$ is clipped to $[-0.3, 0.3]$, preventing the critics from fitting sharp value peaks. DDPG does not apply this regularizer.

Target actor. Both algorithms maintain a slowly updated copy of the actor, $\mu_{\phi'}$, used only when computing Bellman targets:

$$\phi' \leftarrow \tau\,\phi + (1-\tau)\,\phi', \qquad \tau = 0.005 \tag{25}$$

where:

- ϕ — online actor parameters
- ϕ' — target actor parameters
- $\tau = 0.005$ — soft-update coefficient controlling how quickly the target tracks the online network

The slow rate ensures the bootstrap target shifts gradually, even while the online actor receives more aggressive gradient updates. The target actor is never used for data collection or evaluation.

Architectural difference between DDPG and TD3. The actor networks for DDPG and TD3 are identical. The differences between the two algorithms lie entirely in the critic and training procedure: TD3 maintains twin critics and uses their minimum for conservative value targets, and delays actor updates to one per every two critic updates. DDPG uses a single critic and updates actor and critic at the same frequency. This makes DDPG the natural ablation for

TD3: it isolates the effect of clipped double Q-learning, delayed policy updates, and target policy smoothing while holding the policy architecture constant.

4.5.2. Stochastic Policy: PPO

PPO maintains a stochastic policy that outputs a probability distribution over actions at each state:

$$\pi_\theta(\cdot \mid s_t) = \text{TanhNormal}(\mu_\theta(s_t),\, \sigma_\theta(s_t)) \tag{26}$$

where:

- $\mu_\theta(s_t)$ — state-dependent mean produced by the actor network
- $\sigma_\theta(s_t) > 0$ — state-dependent standard deviation produced by the actor network
- TanhNormal — Gaussian distribution whose samples are squashed through tanh to lie in $(-1, 1)$

The actor network shares the same hidden-layer structure as the deterministic actors — two layers of width [128, 64] — but its output head differs. The final linear layer produces $2n_{\text{act}} = 2$ values, which are split into a mean μ and a raw scale parameter $\log\sigma$ by a parameter extractor. The scale is transformed to a positive standard deviation via a softplus non-linearity, and a TanhNormal distribution is constructed:

$$s_t \in \mathbb{R}^{11} \xrightarrow{\text{Linear}} \mathbb{R}^{128} \xrightarrow{\text{Tanh}} \mathbb{R}^{64} \xrightarrow{\text{Tanh}} \mathbb{R}^{2} \xrightarrow{\text{split}} (\mu_\theta(s_t),\, \log\sigma_\theta(s_t)) \tag{27}$$

The resulting $(\mu_\theta(s_t), \log\sigma_\theta(s_t))$ pair parameterizes the TanhNormal policy distribution defined in Equation 26 above.

The PPO actor uses Tanh activations in the hidden layers, following standard practice for actors with stochastic output heads where bounded hidden representations improve numerical stability of the log-probability computation. The actor contains approximately 9,900 parameters (the output dimension doubles the final layer relative to the deterministic actors).

Exploration. Exploration is handled naturally through the entropy of the action distribution. An entropy bonus $c_1 = 0.01$ is added to the PPO objective to encourage the policy to maintain action diversity during training. No explicit noise injection is required.

Evaluation. At evaluation time, stochastic sampling is replaced by the mode of the

TanhNormal distribution, which is $\tanh(\mu_\theta(s_t))$. This yields a deterministic action for each state, directly comparable to the DDPG and TD3 evaluation policies.

Value network. Unlike DDPG and TD3, whose critics estimate $Q(s, a)$, PPO's critic estimates the state value $V_\phi(s)$.[13] The input is therefore the state vector alone (11 dimensions), not the state-action concatenation. The critic architecture is otherwise identical: two hidden layers of width [128, 64] with Tanh activations, followed by a linear output layer producing a scalar value estimate.

4.5.3. Controlled Comparison Design

All three algorithms are trained with the same state representation, reward function, data split, random seed, learning rates, and weight decay. The network capacity is equalized: both actor and value networks use hidden dimensions [128, 64] across all three algorithms. This ensures that differences in empirical performance in Chapter 6 are attributable to the algorithmic properties — policy type, update rule, and stability mechanisms — and not to differences in model capacity or input features.

The table below summarizes the policy-level design choices for each algorithm in the controlled comparison.

Table 1. Policy design choices for each algorithm in the controlled comparison.

Property	PPO	DDPG	TD3
Policy type	Stochastic	Deterministic	Deterministic
Actor output	TanhNormal(μ, σ)	$\mu_\phi(s) \in [-1, 1]$	$\mu_\phi(s) \in [-1, 1]$
Hidden dims	[128, 64]	[128, 64]	[128, 64]
Hidden activation	Tanh	ReLU	ReLU
Output activation	— (distribution)	Tanh	Tanh
Exploration	Entropy bonus $c_1 = 0.01$	Additive Gaussian noise	Additive noise $\sigma = 0.3$
Target smoothing	None	None	$\sigma = 0.2$, clip ± 0.3
Evaluation action	$\tanh(\mu_\theta(s_t))$	$\mu_\phi(s_t)$	$\mu_\phi(s_t)$

[13] In the PPO context, ϕ parameterises the value network V_ϕ, whereas for DDPG and TD3 it parameterises the actor μ_ϕ. The PPO actor is parameterised by θ throughout.

Property	PPO	DDPG	TD3
Critic input	s only	(s, a)	(s, a)
Number of critics	1	1	2 (twin)
Actor update freq.	Every epoch	Every step	Every 2 critic steps

Source: *Author's own elaboration.*

4.6. Discount Factor

The main TD3 HFT experiment uses a discount factor of $\gamma = 0.9$. Chapter 3 defines its role in the value function.

With $\gamma = 0.9$ the effective planning horizon is approximately $1/(1-\gamma) = 10$ steps. On the event-time HFT data used here, this short horizon intentionally emphasizes local order-book dynamics rather than distant price movements. This is appropriate for a Differential Sharpe Ratio reward computed from noisy tick-level portfolio changes, where long bootstrap horizons can make critic targets unstable.

$\gamma = 0.9$ is held constant for the main selected-feature TD3 specification to isolate the effect of the state representation and reward design. Sensitivity to γ remains a natural direction for future work, particularly whether higher values such as 0.95–0.995 improve performance when the objective is less local than the present HFT setup.

4.7. Optimization Hyperparameters

4.7.1. Learning Rates

Both actor and critic use Adam (Kingma & Ba, 2014) with a learning rate of $\alpha = 0.0001$. This is deliberately conservative relative to the 3×10^{-4} default common in continuous-control benchmarks. Preliminary experiments with more aggressive rates produced Q-value explosions and policy collapse — failure modes consistent with the high noise-to-signal ratio of tick-frequency rewards and the sensitivity of bootstrapped TD targets to large gradient steps. Stability was prioritized over sample efficiency.

Actor and critic use the same nominal rate, but TD3's policy delay (every 2 critic updates) effectively halves the actor's update frequency, providing additional stability without a separate hyperparameter.

4.7.2. Weight Decay

L2 weight decay of $\lambda = 2 \times 10^{-6}$ is applied to the critic networks, while the actor uses no explicit weight decay. The critic regularizer is intentionally small — sufficient to discourage memorization of training-period noise without restricting the network's capacity to represent nonlinear market dynamics. No further regularization (dropout, batch normalization) is used, keeping the architecture parsimonious.

4.7.3. Scope

The reported learning rate and weight decay define the experimental settings used throughout the experiments. They were chosen to maintain stable critic targets and avoid policy collapse in preliminary runs, which makes them appropriate for the present high-frequency training setup. A broader hyperparameter study could refine these values further, but that analysis lies beyond the scope of the current empirical comparison.

5. Implementation of the Trading Agent

The experimental pipeline implements the Chapter 4 design through its simulation architecture, data preparation, code organization, and TD3 training workflow.

5.1. Simulation Architecture

The empirical workflow is organized through three functional layers that together implement a single-asset trading simulation.

Market data and feature construction. The agent observes states derived from high-frequency order-book data across six Nasdaq-listed equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO). Raw order-book information is transformed into a vector of microstructure features. This layer is responsible for loading and validating the market dataset, preserving chronological ordering, constructing derived features, separating learning features from the price series used for portfolio valuation, and creating train, validation, and test splits without leakage (see Section 5.3.4). Reinforcement learning in financial settings is highly sensitive to timestamp errors and inconsistencies between engineered features and the price series used by the simulation.

Decision layer. The RL agent maps the current state representation to a bounded continuous action interpreted as target portfolio exposure. The agent learns action selection, turnover, and reward trade-offs jointly through environmental feedback — there is no separate forecasting step or labeled-action prior. The quality of the decision is evaluated through the reward process generated by the environment.

Execution and portfolio accounting. The agent's chosen action is translated into a portfolio update under explicit assumptions about pricing, transaction costs, and liquidity. This layer abstracts the operational complexity of a production trading stack — order management, venue selection, smart order routing, fill handling — into a controlled backtesting mechanism. If the agent performs well in this simulated setting, the result supports a claim about policy learning under the specified assumptions, not a claim that the strategy is immediately deployable without further execution modeling.

Simulation engine. The portfolio accounting layer is built on `tradingenv` (XAI Asset Management, 2024), an open-source event-driven market simulator developed by XAI Asset Management that implements the OpenAI Gymnasium protocol. The library models a broker with configurable transaction fees, handles continuous portfolio rebalancing, and records a full trade history for post-episode analysis. It was integrated into the training pipeline via a custom wrapper (`StreamingTradingEnvXY`) that streams fixed-length episode windows from memory-mapped data files, keeping peak memory proportional to episode length rather than dataset size. One performance modification was applied to the library: the duplicate-timestamp check in the internal `TrackRecord._checkpoint` method was overridden in a subclass to use an $O(1)$ dictionary lookup instead of an $O(n)$ linear list scan, which would otherwise degrade step time quadratically over long evaluation episodes.

5.2. Simulation Assumptions

The following assumptions govern the trading simulation throughout all experiments.

Zero market impact and no order-book interaction. Agent trades are assumed not to move quoted prices, consume displayed depth, or alter subsequent order-book states. The environment therefore treats the agent as price-taking rather than market-shaping. This is defensible only while position sizes remain small relative to displayed order-book depth, which holds for the MBP-10 dataset used here (defined in Chapter 5.1)[14]

Mid-price execution. Trades are valued at the mid-price rather than at executable bid/ask quotes. This avoids explicit spread-crossing and simplifies reward accounting, but it implies that reported backtest returns are an upper bound on achievable net performance. Real execution must cross the spread, and realized returns are further reduced by maker/taker fees, queue position, and partial fills. Section 4.3 (Section 4.3.1) shows why this assumption is decisive for a microstructure feature set: the predictive content of the features is bounded by the very half-spread that mid-price execution declines to charge.

Zero latency. Observations are assumed to be acted on immediately, with no transport, decision, or matching-engine delay between state observation and execution. This is appropriate for a study focused on strategy learning rather than low-latency execution engineering. In production HFT systems, such delays can invalidate short-lived signals.

Bounded continuous exposure. Position constraints are enforced via the continuous action

[14] All six instruments — AAPL, MSFT, TSLA, META, AMZN, and AVGO — are highly liquid Nasdaq-listed blue-chips with tight spreads, deep displayed depth, and continuous two-sided quoting, ensuring that simulated agent trades represent a negligible fraction of resting book volume at any level.

interval $[-1, 1]$, preventing unbounded leverage.

Transaction fee. A configurable fee parameter is applied per position change. Baseline runs use a no-fee setting; sensitivity experiments vary this parameter.

Narrow data scope. Training, validation, and testing are conducted on a single three-day window of data (February 25–27, 2026) across the six instruments. This scope means the agent has only encountered one volatility regime, one spread environment, and one pattern of intraday seasonality. Results should be interpreted as a proof of concept rather than as evidence of generalizability across market conditions or time periods.

These assumptions collectively make the simulation tractable and the results interpretable as learning-dynamics benchmarks rather than deployment-ready performance estimates. The following components of a production trading system are explicitly outside the scope of this study: smart order routing across venues, partial fills and queue-position effects, execution algorithms such as VWAP or TWAP, detailed market-impact modeling, and production monitoring or brokerage integration. These omissions define the gap between the controlled experimental framework and deployment-ready trading-system performance.

5.3. Data Preparation

5.3.1. Asset Selection

The experiments use equity data from six Nasdaq-listed blue-chip instruments: AAPL (Apple Inc.), MSFT (Microsoft Corp.), TSLA (Tesla Inc.), META (Meta Platforms Inc.), AMZN (Amazon.com Inc.), and AVGO (Broadcom Inc.). All six are among the most liquid equities in US markets, with tight spreads, deep displayed depth, and continuous two-sided quoting throughout the trading session. The selection criterion was high but not extreme liquidity: instruments were chosen to ensure defensible price-taking assumptions and rich microstructure signals, while avoiding names whose order-book event density on the Nasdaq ITCH feed would be disproportionately inflated by factors unrelated to underlying economic activity. This liquidity profile ensures that the price-taking assumption embedded in the simulation — that agent trades do not move quoted prices — is empirically defensible, and that the microstructure features derived from the order book reflect genuine supply and demand rather than thin-market artifacts.

Blue-chip selection also minimizes borrow costs, supporting the action space's cost-symmetric short-selling treatment (Section 4.1): loan supply is abundant for these highly liquid large-caps, keeping the symmetric-cost assumption negligible relative to transaction fees. Had the study focused on smaller-cap or illiquid instruments, borrow costs could reach 5–10% or more, rendering that assumption invalid.

The choice to train a single pooled model across multiple instruments, rather than a separate model per instrument, is a deliberate design decision. Securities within the same asset class that trade on the same exchange, under the same tick-size regime, and with comparable order-flow dynamics share a common microstructure regime. The residual cross-sectional variation — differences in volatility level, beta, or sector exposure — is dominated by this shared structure. Within such a homogeneous class, LOB feature distributions are comparable after standard normalization, making it feasible for a single policy network to learn market-wide order-book dynamics rather than instrument-specific quirks. Pooling across N securities multiplies the effective training sample by a factor of N, reducing the risk of overfitting to idiosyncratic patterns in any single stock's order-flow sequence. It also encourages the model to capture universal microstructure signals — spread dynamics, order-flow imbalance, and depth asymmetries — that transfer across equities within the same class. The feature pipeline supports this directly: all inputs are z-score normalized per security, and the microstructure features in the state space — order-flow imbalance, relative spread, depth slope, queue imbalance — are inherently relative measures that carry the same semantic meaning regardless of the underlying price level or absolute volume.

A further consideration is access symmetry. Because all six instruments trade on a centralized, regulated exchange with public pre-trade transparency, all market participants observe the same order book. This contrasts with OTC markets or less liquid instruments where information asymmetry and differential access could confound the learned signal. The goal is to isolate the contribution of the learning algorithm and feature engineering, not to exploit a structural information advantage.

The data were obtained from Nasdaq via the DataBento API, using the MBP-10 (Market By Price, 10 levels) feed — a hybrid order book format that sits between L2 (aggregated depth) and L3 (order-by-order) data, providing depth-of-book snapshots at up to ten price levels and additionally recording the order count at each level via `side_ct_xx` fields (e.g., `bid_ct_00` for the number of orders at the best bid).[15] This granularity is necessary for constructing the microstructure features described in Chapter 4 and listed in Appendix A.

5.3.2. Input Format

The pipeline consumes time-indexed LOB snapshot data. Each row represents an order-book event: a `timestamp` for ordering and temporal features; bid/ask price, resting size, and order count at each book level `NN` (00-09, feeding spread, imbalance, depth, and queue features respectively); `action`, `side`, and `size` describing the event itself (used for signed trade-flow features); and an auxiliary `close` series used only for reward and portfolio valuation. Unless stated otherwise, all timestamps in this thesis are reported in UTC and all monetary values in US dollars.

Table 2 shows four consecutive events from the AAPL feed at market open on 25 February 2026. The three events span roughly 9 milliseconds, illustrating the sub-millisecond cadence typical of the feed. Each row advances the state of the order book: the first event is a resting limit-order addition on the bid side at a non-top-of-book level; the second is a trade print that consumes displayed size at the best ask; the third and fourth are additions that refresh the ask side after the trade.

Table 2. Four consecutive AAPL MBP-10 events, 25 February 2026.

Timestamp	Action	Side	Price	Size	Best Bid	Best Ask	Bid Sz	Ask Sz
14:30:00.016	A	B	271.20	200	271.45	271.85	500	756
14:30:00.024	T	N	271.65	21	271.45	271.85	500	756
14:30:00.024	A	A	271.79	140	271.45	271.79	500	140
14:30:00.025	A	A	271.79	100	271.45	271.79	500	240

Source: *Author's own elaboration based on DataBento Nasdaq MBP-10 data, AAPL, 25 February 2026.*

Action: A = limit-order addition, T = trade print. Side: B = bid, A = ask, N = no aggressor side (trade report). Best Bid/Ask and Bid Sz/Ask Sz reflect the level-0 (top-of-book) snapshot after each event is applied.

[15] MBP-10 schema documentation: https://databento.com/docs/schemas-and-data-formats/mbp-10 For a technical reference on market data quality and L3 order book processing, see Nasdaq TotalView-ITCH specification: https://www.nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/NQTVITCHSpecification.pdf

5.3.3. Preprocessing and Splitting

Raw data is validated for chronological order, required column presence, non-negative prices, and non-crossed best quotes before any feature engineering. This stage is conservative: its purpose is correctness, not statistical filtering.

Before feature engineering, each daily file is filtered to retain only events that modify at least one of the five shallowest book levels (levels 0–4). The full MBP-10 feed delivers updates at all ten price levels, but every microstructure feature in the state space draws exclusively on levels 0–4: order-flow imbalance, depth slopes, queue sizes, bid/ask slopes, and related signals are all defined relative to the top-of-book region. Events that update only levels 5–9 are therefore invisible to the feature computation; they produce consecutive observations with an identical observable state, inflating episode length and adding computational overhead without contributing any learnable signal. Including them would also implicitly penalise the agent for changes in book depth that it has no mechanism to detect or respond to. The filter is applied immediately after chronological validation and before any feature transformation, so the feature pipeline never encounters these no-signal events.

The dataset is organized as one parquet file per security per trading day, yielding 24 files in total (6 symbols × 4 trading days). The four days cover February 25–27, 2026 (days 1–3, training) and March 2, 2026 (day 4, evaluation). All files are filtered to US regular market hours (09:30–16:00 ET). Raw event counts and inter-event timing per symbol and split, before the level-0–4 filter is applied, are reported in Appendix, Table 10; event density varies substantially across symbols (from roughly 190,000 to 7.1 million events per split) and this variation carries through to the post-filter split sizes summarized below.

Training data. The 18 files from days 1–3 are used exclusively for training. Each daily file is treated as a complete, independent training unit: the agent encounters all order-book events from that file in one episode. During training, the 18 files are served in uniformly shuffled random order across episodes, so the agent never sees a fixed day-order sequence and cannot exploit calendar artifacts.

Validation and test data. The 6 day-4 files (one per symbol) are held out entirely from training. Each file is split 50/50 at its chronological midpoint into a validation half and a test half; the six validation halves and six test halves are concatenated separately to form the pooled validation and test sets. This design ensures that evaluation data covers the same six securities but originates from a genuinely unseen trading session, providing a clean out-of-sample window. The validation split supports model comparison and development diagnostics; the test split is reserved for final out-of-sample reporting and does not influence any model selection decisions.

The four-day window spanning 25 February to 2 March 2026 is a recognized limitation of the current empirical design. The agent's learned behavior has not been exposed to different volatility regimes, spread environments, or intraday seasonality patterns beyond this specific period, and the generalizability of the results is bounded accordingly.

Feature pipeline fitting. All ten features in the selected set use online, strictly-causal normalization: the statistics that standardize the observation at step t are accumulated from that session's own events up to $t-1$ only (Section 5.3.4), and they reset at each session break. For this feature set the separate fitting step over the concatenated training days is therefore inert — its output is never consulted at transform time — and the evaluation day is normalized entirely by its own expanding within-session statistics. The causality guarantee is consequently stronger than a train-fit-then-freeze scheme would provide: training-period data cannot leak into the evaluation representation because it does not enter it at all. This was confirmed directly — normalizing the AAPL evaluation day with a pipeline fitted on a different security (TSLA) reproduces the AAPL-fitted output bit for bit, and a prefix-invariance check (normalizing the first 200,000 versus the first 400,000 rows) leaves the overlapping rows unchanged.

5.3.4. Feature Normalization and Causality Preservation

Normalization of microstructure features presents a methodological challenge for sequential learning: the conventional approach of fitting a standard scaler on the full training dataset and applying it globally introduces look-ahead bias. Each observation's normalized value would incorporate statistical information from future timesteps, violating the causal structure that a trading agent must respect in deployment. An agent at step t can only condition on information available at time t; allowing its inputs to be scaled using the mean and variance of the entire training set (including steps $t+1, t+2, \ldots$) would, in effect, provide the agent with knowledge it cannot access.

To preserve causality, this study employs a **cumulative running normalization** scheme: for each feature, the mean and variance used to standardize the observation at step t are updated incrementally from the mean and variance at step $t-1$, following Welford's online algorithm (Welford, 1962). This requires no buffer of past observations and no pass over future data, updates in $O(1)$ per step, and remains numerically stable even for the near-empty denominators of early-session observations.

The resulting normalized value is the single-feature case of Equation 13 (Chapter 4), with the feature index i suppressed and the running mean and variance written as $\bar{x}_t, \sigma_t^2$ rather than $\mu_{i,t}, \sigma_{i,t}^2$:

$$\tilde{x}_t = \frac{x_t - \bar{x}_t}{\sqrt{\sigma_t^2 + \varepsilon}} \tag{28}$$

Because $\bar{x}_t$ and σ_t^2 incorporate only observations $1, \ldots, t$, and x_t is available before the action at step t is chosen, this transformation never uses future information.

5.3.4.1. Session-Aware Normalization

The continuous-time nature of equity markets introduces regime discontinuities at session boundaries. Overnight and weekend price gaps, changes in order-flow patterns between the opening auction and continuous trading, and differences in liquidity across intraday periods all represent shifts in the underlying data-generating process. A naive cumulative normalization that spans these boundaries would allow statistics from the previous session to contaminate the current session's representation. For example, a 5% overnight price decline would make the opening trade appear as an extreme outlier when normalized against statistics that include the previous day's stable prices, even though such gaps are expected in continuous-time markets.

To address this, running statistics are **reset at session boundaries**. A session break is defined as a time gap exceeding one hour, which distinguishes short intraday interruptions (e.g., lunch recess) from genuine overnight or weekend closures. At each boundary, $\bar{x}$ and σ^2 are reinitialized to their default values ($\bar{x} = 0, \sigma^2 = 1$), and the accumulation process begins anew for the next session. This prevents the overnight gap from distorting the morning's normalization statistics and ensures that each trading session's features are scaled relative to that session's own empirical distribution.

5.3.4.2. Event-Time vs. Continuous-Time Weighting

An additional design choice concerns whether to weight observations by event count or by elapsed time. In high-frequency trading, events cluster during periods of high market activity: a burst of order-book updates over one second may contain 100 events, while a quiet 10-second interval may contain only one. An event-based normalization (the default in many standard RL libraries) would assign equal weight to each of the 100 clustered events and the one sparse event, effectively over-representing the high-activity period in the statistical summary. Continuous-time markets, however, are not sensitive to the number of events per se but to the duration over which each price or quote persists.

This implementation also supports an optional **time-weighted normalization** mode, where each observation is weighted by the elapsed time since the previous event rather than counted equally, so the running mean and variance reflect a time-average of the feature's trajectory instead of an average across discrete events. For the main experiments, event-based weighting is used as the default, matching the standard approach in the RL literature (e.g., Raffin et al., 2021; E. Liang et al., 2018); sensitivity analysis with time-weighted normalization is a direction for future work.

5.3.4.3. Session-Scoped Statistics and Leakage Prevention

For causal running normalization, statistics are maintained **independently per security** and are **reset at each detected trading-session boundary**; they are not reset at the boundaries between the training, validation, and test subsets. Because the accumulator only moves forward in time and never uses observations after the current step, there is no look-ahead: statistics from the test period cannot influence the training representation, and a later observation cannot influence an earlier one. One consequence should be stated plainly. The validation and test sets are the two chronological halves of a single trading session, split at its midpoint, and the running statistics are not reset at that midpoint — so the test half's normalization is warm-started by the accumulated statistics of the validation half. This is backward-looking and therefore not leakage in the look-ahead sense, but it does mean the validation and test halves are not statistically independent samples and should not be described as separated by the normalizer. The per-symbol scoping is retained for its original purpose: each security's features are normalized relative to its own microstructure rather than to magnitudes that differ across price levels and liquidity profiles.

Cyclical temporal encodings (hour-of-day sine and cosine) are already bounded in $[-1, 1]$ and are not subject to z-score normalization. Raw price columns used for portfolio accounting are kept in their original units and are never replaced by normalized values. This separation ensures that the observation features passed to the learning algorithm are statistically well-behaved, while the pricing inputs used for reward and valuation computation retain economic interpretability and correct accounting semantics.

5.3.4.4. Transformed Feature Example

The causal running normalization in Equation 28 converts each raw order-book event into the microstructure features consumed by the policy. A twelve-event AAPL window, reported in Appendix, Table 11, illustrates this pipeline end to end: raw best bid/ask prices and sizes alongside the corresponding normalized features. Three properties stand out. **Book Pressure** and **Order Imbalance** capture instantaneous liquidity and directional pressure — positive when bid depth dominates, negative when a bid-side refresh shifts pressure toward the ask (visible at event 12 in the appendix table). **Microprice Deviation** tracks how far the microprice lies from the simple mid-price, a depth-weighted estimate of fair value. **OFI** captures signed net trade flow over a short window and spikes sharply when displayed depth is consumed or replenished. These four features are a representative subset; the full feature set additionally includes depth slopes, queue depletion rates, VPIN, and rolling order-flow statistics.

5.3.5. Feature–Return Correlations

Table 13 reports Pearson and Spearman rank correlations between each engineered feature and the one-step-ahead log mid-price return, computed on the training split. Both measures capture strictly pairwise, marginal dependence: Pearson detects linear association, while Spearman detects any monotone relationship — including non-linear but rank-preserving ones.

All absolute correlations are below 0.005. Neither measure identifies any feature whose marginal distribution is reliably aligned with the direction of the next price move. This is expected in a microstructure context: each feature captures a local structural signal — order-flow pressure, depth asymmetry, queue imbalance — that is informative about the trading environment but not predictive of short-term returns in isolation. Predictive value, if any, arises from the joint, non-linear interaction among multiple features simultaneously observed at the same timestep.

Linear models — including logistic regression, ridge regression, and any other predictor that combines features through a weighted sum — cannot exploit this joint non-linear structure. The near-zero Pearson and Spearman correlations confirm that linearly combining even the full feature vector is unlikely to yield a useful directional signal. This directly motivates the use of a deep neural network as the function approximator inside the RL policy: the network can, in principle, learn arbitrary non-linear mappings from the feature vector to action-value estimates, capturing the higher-order interactions that linear models cannot represent.

The full per-feature Pearson and Spearman breakdown is reported in Appendix, Table Table 13.

5.3.6. Feature and Price Column Separation

A strict separation is maintained between the observation features passed to the learning algorithm and the price inputs used by the portfolio simulation. The learning features are normalized and engineered; the pricing columns are economically interpretable and used only for reward and valuation computation. Mixing these two roles can produce invalid reward calculations or cause normalized prices to appear in financial metrics.

In the current implementation, the auxiliary pricing series used for portfolio valuation plays the role of a mid-price proxy rather than an executable bid/ask quote. This simplifies reward accounting but makes the resulting backtest performance optimistic relative to realistic spread-crossing execution.

5.3.7. Dataset Split Summary

Table 3 summarises the row counts and timestamp ranges of the training, validation, and test subsets for the pooled six-asset dataset used in the primary experiment. Row counts reflect the number of order-book events that pass the level-0–4 filter after chronological validation. The validation and test subsets are each bounded at 50,000 events by the configured `validation_size` and `test_size` parameters; the training subset comprises all remaining events from the three training days across the six instruments.

Table 3. Training, validation, and test split sizes and timestamp ranges for the pooled six-asset LOB dataset.

Split	Events	First timestamp	Last timestamp	Mean Δt	Median Δt
train	50,000	2026-02-25 14:30:01	2026-02-25 14:37:07	8.52 ms	14.6 µs

Table 3. Training, validation, and test split sizes and timestamp ranges for the pooled six-asset LOB dataset.

Split	Events	First timestamp	Last timestamp	Mean Δt	Median Δt
val	969,219	2026-03-02 14:30:00	2026-03-02 16:53:17	8.87 ms	25.8 µs
test	969,219	2026-03-02 16:53:17	2026-03-02 20:59:59	15.27 ms	33.3 µs

Source: *Author's own calculations based on DataBento Nasdaq MBP-10 data.*

5.3.8. Feature Statistics

Across the engineered features, means close to zero and standard deviations near one confirm that causal running normalisation operates as intended. The main exception is the heavy-tailed OFI family: rare aggressive orders produce pre-clipping z-scores of ± 290 or more, but the environment bounds the values received by the policy to $[-5, 5]$. The full distributional breakdown is reported in Appendix, Table 12.

5.4. Implementation and Training Pipeline

5.4.1. Experiment Specification Design

The full source code for this thesis is publicly available online (Wojdalski, 2026). Each experiment is fully specified by a structured set of inputs covering the dataset, feature subset, reward type, environment settings, and optimization hyperparameters. No source-code changes are required to switch between experimental variants. A pre-training validation stage checks path availability, feature definitions, split feasibility, and column compatibility before any computation begins, preventing wasted runs from invalid experimental specifications. A condensed summary of the main specification is provided in Appendix B. A high-level overview of the full pipeline — from data acquisition and offline feature research through preparation, pre-training configuration checks, training, and post-training evaluation — is provided in Appendix E; the complete machine-readable diagram is maintained alongside the source code in `docs/workflow.d2`.

5.4.2. Feature Pipeline

Feature engineering is registry-driven: feature names resolve to computation functions, allowing feature-set composition without editing training code. This design is central to systematic feature subset comparisons. Normalization parameters are estimated on the training split only for non-causal scalers. For the main HFT experiment, features use causal running normalization: each split is transformed chronologically, and the normalization state is reset at detected session boundaries. This preserves causal integrity across both time and the chronological split.

5.4.3. Training Loop

The processed training split is used to construct the trading environment with a continuous action space $a_t \in [-1, 1]$ representing target portfolio exposure. Training follows the TD3 procedure described in Chapter 3, with comparable implementations for DDPG and PPO for experimental comparison. All three algorithms share a common training infrastructure: experience collection, network updates, and evaluation. The primary differences are in the update rules (deterministic policy gradient for DDPG/TD3, clipped surrogate for PPO) and replay buffer usage (off-policy for DDPG/TD3, on-policy for PPO).

For DDPG and TD3, experience is collected into a replay buffer and the critic is updated from sampled mini-batches via the deterministic policy gradient; target networks track the main networks through soft updates with coefficient τ, and exploration noise is applied during training and removed at evaluation time. For PPO, training follows the on-policy paradigm: batches are collected using the current policy, multiple epochs of updates are performed on each batch, and the clipped surrogate objective constrains policy changes. The stochastic policy handles exploration through entropy bonuses rather than explicit noise injection.

Algorithm 1 Common Experiment Setup

Require: Experiment specification (dataset, feature set, hyperparameters)

Ensure: Processed train/validation/test environments ready for algorithm-specific training

Load and validate experiment specification; check paths and split feasibility
Load L3 order book dataset; sort chronologically
Split data into train / validation / test subsets (chronological)
Initialise and validate feature pipeline on training split
Transform each split chronologically with causal session-aware normalization
Build training environment from processed training data

Algorithm 2 TD3 Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; TD3 hyperparameters

Ensure: Trained policy μ_ϕ, evaluation metrics

Initialise actor μ_ϕ, twin critics $Q_{\theta_1}, Q_{\theta_2}$, target networks $\mu_{\phi'}, Q_{\theta'_1}, Q_{\theta'_2}$, replay buffer $\mathcal{D}$
for each step $t = 1, \ldots, \texttt{max\_steps}$ **do**
    $a_t \leftarrow \mathrm{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big), \quad \epsilon \sim \mathcal{N}(0,\ \sigma^2_{\text{explore}})$
    Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
    Sample mini-batch $\mathcal{B} \sim \mathcal{D}$
    Compute smoothed target action: $\tilde{a}' \leftarrow \mu_{\phi'}(s') + \mathrm{clip}(\mathcal{N}(0, \sigma^2_{\text{noise}}), \pm c)$
    Compute Bellman target: $y \leftarrow r + \gamma \cdot \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a}')$
    Update both critics by minimising $\mathcal{L}_{\text{Smooth-L1}}\big(y - Q_{\theta_i}(s, a)\big)$
    if t mod $\texttt{policy\_delay} = 0$ **then**
        Update actor: $\phi \leftarrow \phi + \alpha \nabla_\phi Q_{\theta_1}(s,\ \mu_\phi(s))$
        Soft-update targets: $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i;\ \phi' \leftarrow \tau\phi + (1-\tau)\phi'$
    end if
    Checkpoint and log metrics at configured intervals
end for
Evaluate deterministic policy μ_ϕ on test split
Compute financial performance metrics from realised return series

DDPG (Algorithm 3, Appendix Section C) follows the same loop as Algorithm 2 with three simplifications: a single critic in place of the twin critics (so the Bellman target uses $Q_{\theta'}(s', \mu_{\phi'}(s'))$ directly, without the minimum), the actor updated every step rather than every `policy_delay` critic steps, and no target-action smoothing noise. PPO (Algorithm 4, same appendix section) instead follows the on-policy paradigm summarized above: batches are collected with the current stochastic policy, GAE advantage estimates are computed, and the clipped surrogate objective (Equation 7) together with a value loss and entropy bonus are optimized jointly over multiple epochs per batch, with no replay buffer.

The notation follows the conventions established in Chapter 3:

- μ_ϕ is the deterministic actor; $Q_{\theta_1}, Q_{\theta_2}$ are TD3's twin critics
- Primed symbols denote slowly updated target networks
- $\mathcal{D}$ is the replay buffer
- $\epsilon \sim \mathcal{N}(0, \sigma^2_{\text{explore}})$ is training-only exploration noise
- τ is the soft-update coefficient
- γ is the discount factor; α is the learning rate

Concrete values are listed in Appendix B; DDPG's and PPO's own symbols are defined alongside their algorithms in Appendix Section C.

5.4.4. Sanity Checks

Before running the empirical order-book experiments, the RL pipeline was tested on a synthetic sine-wave scenario with deliberately learnable structure.[16] The purpose of this check was not to evaluate trading performance, but to verify that the environment, reward computation, replay buffer, neural networks, and training loop were capable of recovering a known pattern when one was present. In this setting, the TD3 agent learned behavior consistent with the imposed sine-wave structure, providing a basic implementation sanity check before moving to noisy tick-level order-book data.

A complementary check was conducted using a random policy — one that selects actions uniformly at random with no learned component. This tests the opposite question: whether the environment, reward function, and accounting logic are free of structural bias that would generate returns even without a signal. The random policy produced approximately zero return, consistent

[16]The sanity-check run can be reproduced from the project root with: `uv run python src/cli.py train -c src/configs/scenarios/sine_wave/td3_hft_lob_future_oracle_combined/`.

with an unbiased implementation. Results for the random policy on the empirical dataset are reported in Chapter 6.

Beyond the ad-hoc checks above, a suite of automated tests guards the implementation against the failure modes most common in RL codebases. Learning health checks verify, after a single gradient update, that observations and rewards are finite, that every feature dimension varies across the batch (i.e., no collapsed or constant-valued feature), that the loss is finite, and that actor and critic parameters both change and remain finite — four conditions that are individually obvious but together rule out the silent errors most likely to produce plausible-looking training curves without genuine learning. Pipeline integration tests run a short end-to-end training loop to detect regressions in data loading, environment construction, or replay buffer interactions. These tests execute automatically as part of the development workflow, ensuring that changes to any component of the pipeline do not silently invalidate the experimental conditions that produced the reported results.

5.4.5. Evaluation

Final performance is reported on the held-out test split using a dedicated evaluation environment. Training reward and realized portfolio return are kept as two separate output streams — one for optimization diagnostics, one for the financial metrics computed from the deterministic rollout — for the reasons discussed in Section 4.3. Metrics derived from the test rollout are described in Chapter 6.

6. Results

The thesis-facing experiment evaluates a TD3 agent trained on high-frequency order-book features from six Nasdaq equities (AAPL, MSFT, TSLA, META, AMZN, and AVGO) using the state-space design defined in Chapters 4 and 5. The evidence covers repeated-run variability, robustness to nearby design changes, and final held-out performance.

6.1. Algorithm Comparison Results

Beyond benchmark performance, the key empirical question is how the algorithmic choices in Chapter 3 affect trading results. Comparing PPO, DDPG, and TD3 on identical data, features, and evaluation protocols isolates their practical effects.

6.1.1. Benchmark Strategies

Performance is assessed relative to five benchmark strategies. These benchmarks span passive exposure, execution-style references, and a stochastic baseline, together providing a comparison surface for the directional TD3 agent.

Buy-and-hold maintains a constant fully long position ($a_t = 1.0$) throughout the evaluation period, capturing the full market return of the asset. It is the primary economic benchmark: any active strategy that fails to outperform buy-and-hold on a risk-adjusted basis has not demonstrated useful directional skill beyond passive exposure.

TWAP (Time-Weighted Average Price) distributes position changes uniformly across time. **VWAP (Volume-Weighted Average Price)** distributes them in proportion to the volume actually traded during the evaluation window. Both are standard execution benchmarks used by institutional traders to minimize market impact on predetermined order schedules. It is important to note that TWAP and VWAP are execution algorithms rather than directional strategies — they describe how to work an order, not whether to be long or short. Their inclusion reflects standard practice in trading-system evaluation; however, outperforming them has a different interpretation than beating buy-and-hold, and they serve as lower-bar references rather than directional competitors.

VWAP is normalized by the evaluation window's total realized volume, making it a retrospective reference rather than a rule executable in real time; TWAP is the implementable execution baseline.

Random selects actions uniformly from $[-1, 1]$ at each step without reference to market state. Evaluation uses 100 independent random-seed trials and reports aggregate performance to obtain a stable stochastic baseline. Failure to consistently outperform a random policy would indicate that the agent has not learned a meaningful relationship between observations and actions.

6.1.2. Comparison Methodology

The exported thesis snapshots report one completed evaluation run per algorithm for this comparison: it should be read as matched experimental evidence on the available exports, not as a multi-seed estimate of the population distribution of outcomes.

6.1.3. Performance Comparison

The table below reports test-split metrics for each algorithm, all trained on identical data, features, reward function, and evaluation protocol.

`No Hypothesis 1 results available. Run: uv run thesis-experiments h1`

Return: cumulative simple return. Ann. Vol: annualized volatility. Max DD: maximum drawdown. Sharpe: mean excess return / standard deviation (not annualized). Sortino: mean excess return / downside deviation (not annualized). Win Rate: fraction of steps with positive return. PF: profit factor (gross wins / gross losses). Turnover: average absolute position change per step. % Long/Short: fraction of steps with positive/negative position.

Table 4. Hypothesis 1 per-instrument test-split results for the TD3 agent, with the cumulative return restated as a mean edge per decision.

`No per-instrument Hypothesis 1 results available. Run: uv run thesis-experiments h1`

6.1.4. Per-Instrument Decomposition

Each pooled scenario is evaluated one instrument at a time, so the aggregate above averages six independent evaluations. Reporting only that average would conceal how far the instruments diverge — and, more importantly, would misattribute the reason they do.

The cumulative-return column spans a factor of twenty, from $+33.4\%$ on META to $+675.3\%$ on AMZN, which invites the reading that the strategy works on some instruments and not others. The per-step column shows that reading is wrong. Expressed per decision the six instruments occupy a narrow band — roughly 0.016 to 0.036 basis points per step — and the ordering does not match the cumulative one: META, the weakest instrument on cumulative return, carries a *higher* per-step edge (0.0211 bp) than AMZN, the strongest (0.0181 bp).

What separates them is the number of decisions the evaluation window afforded. AMZN records 1,128,425 environment steps against META's 136,676, an eight-fold difference in opportunity over the same wall-clock session, because message-level event counts differ across instruments. Compounding a similar per-step edge over eight times as many steps produces the twenty-fold gap in the cumulative column. Holding META's per-step edge fixed and applying AMZN's step count would imply a cumulative return near $+980\%$ — above AMZN's own figure. Cumulative return over a fixed session therefore measures event frequency at least as much as it measures skill, and the per-step figure is the quantity that should be compared across instruments and carried into the transaction-cost analysis.

6.1.5. Key Empirical Questions

The results below are read through the theoretical lens from Chapter 3: whether TD3's overestimation correction improves on DDPG, how PPO's stability trades off against TD3's efficiency, and which algorithm best balances risk-adjusted return against practical deployment constraints.

6.1.6. Discussion of Algorithmic Trade-offs

The comparison reveals qualitatively distinct behavioral profiles across the three algorithms, even though all three were trained on identical data, features, and reward specification.

DDPG produces the weakest result. It develops a systematic short bias during evaluation — spending 63.5% of steps short against 36.5% long — with a profit factor of 0.44, meaning gross losses exceed gross gains, and a marginally negative cumulative return (-2.67×10^{-7}). In the evaluation window the pooled market drifted upward, so a persistent short bias is doubly penalising: it misaligns directional exposure with realized price movement and accumulates losses from that misalignment. The result is consistent with the theoretical concern that DDPG's single-critic Q-learning is susceptible to overestimation bias in noisy reward environments. Inflated Q-values in a subset of state-action regions can push the policy toward directional commitments that are not grounded in genuine microstructure signal; the short bias observed here is a plausible manifestation of that failure mode.

TD3 achieves a positive but very small cumulative return (2.28×10^{-7}), with a near-neutral position distribution (49.8% long, 50.2% short) and extremely low turnover (0.054% per step). A profit factor of 5.70 indicates that winning trades are substantially larger than losing ones, but the win rate of 18.8% — combined with near-zero turnover — implies that the agent spends most evaluation steps holding a near-flat position, with infrequent and selective position changes. The twin-critic correction and delayed policy updates appear to prevent the directional drift seen in DDPG: the policy does not commit to a persistent directional bias and instead preserves near-neutral exposure throughout the evaluation period. The consequence is the lowest maximum drawdown of any algorithm (-1.05×10^{-7}), effectively zero, at the cost of generating negligible absolute return.

PPO generates the highest cumulative return (4.95×10^{-5}) but with a qualitatively different behavioral profile. It spends 65.3% of steps long, has a turnover of 8.2% per step — approximately 150 times higher than TD3 — and incurs a maximum drawdown of -1.35×10^{-5}, several orders of magnitude larger than TD3's. PPO's long bias in an evaluation window that trended upward partially explains its absolute return advantage. Whether this directional lean reflects genuine LOB-derived signal or favorable alignment between the learned policy and the realized market direction cannot be separated from a single evaluation window of this length. Under any positive transaction cost regime, PPO's substantially higher turnover imposes a larger per-step cost burden, and the sensitivity analysis confirms that the strategy's apparent advantage degrades more sharply than TD3's as fees increase.

The **random baseline** achieves a positive return (2.58×10^{-5}) with the highest turnover (67.5% per step) and drawdown (-5.60×10^{-5}) of any strategy. Its profit factor of 1.31 is consistent with noise rather than signal: in an evaluation window with positive market drift, a strategy that is randomly long approximately half the time accumulates small positive returns from upward price movement without having learned any relationship between observations and actions. That TD3 and PPO both exceed the random baseline on profit factor — while DDPG does not — is consistent with TD3 and PPO having learned some structure in the state-action mapping that goes beyond randomness, even if the absolute magnitude of that advantage is modest at the zero-fee evaluation horizon studied here.

The comparison does not separate the three learning algorithms. On AAPL, the common instrument for which all four agents have been evaluated, TD3 and DDPG are indistinguishable on cumulative return (3.548 versus 3.599), PPO posts the highest per-bar Sharpe ratio (7.53) on the lowest return (2.522), and the ordering is not stable across splits: DDPG leads on validation as well as on test. Reading these three as a ranking would over-interpret a single instrument and a single evaluation window. What separates cleanly is the learned-versus-unlearned boundary — all three learners return above 250% while the random policy returns -0.49% — and the algorithms differ mainly in how they reach that outcome, PPO changing position roughly three times as often as TD3 ($731{,}830$ versus $233{,}082$ steps with a position change) for a smaller gross result.

That convergence is more informative than a ranking would have been. TD3 and DDPG differ precisely in the twin-critic overestimation correction, and PPO is on-policy and stochastic rather than off-policy and deterministic; three optimizers with materially different inductive biases arriving at the same order of magnitude indicates that the exploitable structure is a property of the feature set and the evaluation environment, not an artifact of one algorithm's bias. The remainder of this thesis therefore fixes TD3 as a representative continuous-control learner rather than as a demonstrated winner, and varies the experimental factors around it.

Read through the reworded Hypothesis 1, the algorithm comparison establishes the existence half of the claim. Under frictionless mid-price execution every learned agent extracts a directionally coherent, risk-controlled edge that the random policy does not, so a learnable microstructure signal is present in this window. What the comparison does not establish is that the edge is economically large. Expressed per decision rather than per window, the TD3 agent's edge is a mean log return of roughly 1.6 to 3.6×10^{-6} per step across the six instruments — approximately 0.016 to 0.036 basis points, itemised in Table 4 — over sessions in which buy-and-hold on the same instruments ranged from -0.68% (MSFT) to $+1.07\%$ (AVGO), that is, sessions with no material directional move to capture. The headline cumulative figures are the compounding of a per-step edge far smaller than one tick of spread, not evidence of a large exploitable inefficiency. Section 4.3 (Section 4.3.1) explains why that magnitude is the decisive quantity for a microstructure feature set: the features carrying the signal cannot express an edge larger than one half-spread per trade, which is exactly the cost mid-price execution declines to charge. Whether an edge of this size survives once that cost is applied is taken up by the transaction-cost sensitivity analysis in Section 6.2. The defensible Hypothesis 1 verdict is therefore that continuous-control agents learn a real but very small microstructure edge, and that their frictionless performance is a ceiling on — not an estimate of — deployable performance.

6.2. Statistical Validation

Repeated TD3 training runs can diverge due to random initialization, replay sampling, and exploration noise, even under an identical dataset and configuration — the same single-run limitation already noted for the Hypothesis 1 comparison above applies throughout this chapter. Repeated tuning on the same validation period can also bias later comparisons, and reward-based metrics can diverge from financial objectives when the reward is a shaped training signal rather than realized return. Final claims are read together with the robustness assessment and economic interpretation that follow.

6.3. Robustness Assessment

Robustness depends on whether nearby changes in feature specification, reward definition, transaction-cost assumptions, or split design materially alter the conclusions for the high-frequency TD3 setup. Stable results indicate structural effects; unstable results suggest dependence on a narrow experimental design.

6.4. Hypothesis 2: Transaction-Cost Sensitivity

This section varies the proportional transaction cost while holding the algorithm, feature set, and reward fixed, on a baseline with the selected 10-feature set, log-return reward, and zero fee. It serves two purposes: it locates the fee level at which the learned edge disappears, and it is the decisive test of the reworded Hypothesis 1.

Table 5. Hypothesis 2 transaction-cost sensitivity: test-split metrics across fee levels.

Fees	Sharpe	Sortino	Return	Max DD	Win Rate	PF	Turnover
0 bp (baseline)	—	—	-1.74e-03	-1.74e-03	0.00%	0.000	0.0006
0.01 bp	31070.702	207054.883	2.28e-07	-1.04e-07	9.33%	2.536	0.0007
0.1 bp	-142707.214	-136541.720	-7.60e-07	-7.77e-07	3.83%	0.127	0.0006
1 bp	-226679.852	-180151.617	-5.89e-06	-5.91e-06	3.83%	0.047	0.0005

Source: *Author's own calculations.*

The transaction-cost comparison is where the reworded Hypothesis 1 is decided. Hypothesis 1 claims the learned edge is real but of the same order of magnitude as the cost of trading on it, and the fee sweep tests exactly that. The policy is positive at zero fee; it is marginally positive at 0.01 basis points with a weaker profit factor; cumulative return turns negative at 0.1 basis points and the loss deepens at 1 basis point. The sign flips between 0.01 and 0.1 basis points — a per-trade cost of a few hundredths to a few tenths of a basis point — which is the scale of the bid–ask half-spread on these instruments, and the half-spread is the maximum edge the microstructure feature set can express (Section 4.3, Section 4.3.1). The fee sweep and the half-spread bound therefore locate the learned edge from opposite directions: it is positive but approximately one half-spread per trade, captured under frictionless execution and eliminated by realistic spread-crossing cost. That is the evidential content of Hypothesis 1 as reworded.

For Hypothesis 2 itself, the same table carries a distinct conclusion: empirical performance is materially sensitive to the transaction-cost assumption — a parameter often left implicit — and the fact that the sign changes within one order of magnitude of fee is why robustness analysis is a precondition for any economic claim here. Hypothesis 1 says how large the edge is; Hypothesis 2 says how fast the reported result moves when this assumption is varied.

6.5. Hypothesis 3: Feature Specification Sensitivity

This section compares three feature-specification levels under otherwise identical conditions (same algorithm, data, reward, and transaction cost) to assess whether richer microstructure-aware state representations improve out-of-sample performance.

Table 6. Hypothesis 3 feature sensitivity: test-split metrics for minimal (3), selected (10), and full (33) feature sets.

Feature Set	Sharpe	Sortino	Return	Max DD	Win Rate	PF	Turnover	Ann. Vol
Minimal (3 features)	-0.013	-0.019	-3.58e-04	-1.90e-03	47.70%	0.967	0.0000	0.166
			+1.38e-03	-1.61e-04	+47.70%	+0.967	-0.0006	+0.166
Selected (10 features)	—	—	-1.74e-03	-1.74e-03	0.00%	0.000	0.0006	0.000
	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)	(baseline)
Full (33 features)	-0.140	-0.179	-2.96e-03	-4.72e-03	44.83%	0.700	0.1703	0.132
			-1.22e-03	-2.98e-03	+44.83%	+0.700	+0.1698	+0.132

Source: *Author's own calculations.* ***Legend:*** *Sharpe: mean excess return / std (not annualized). Sortino: mean excess return / downside deviation (not annualized). Return: cumulative simple return. Max DD: maximum drawdown. Win Rate: fraction of positive-return steps. PF: profit factor (gross wins / gross losses). Turnover: mean absolute position change per step. Ann. Vol: annualized volatility.*

The feature comparison supports a qualified version of Hypothesis 3. The minimal three-feature representation produces a negative cumulative return (-3.45×10^{-8}), a profit factor below one, and a short-biased policy that spends 77.2% of the evaluation window short. The selected 10-feature microstructure representation improves the result to a positive cumulative return (1.29×10^{-7}), a profit factor of 2.01, and a materially stronger Sortino ratio under the same data and evaluation protocol.

The full 33-feature representation does not improve further. It remains positive on cumulative return (2.68×10^{-8}), but underperforms the selected representation on return, drawdown, profit factor, and turnover. This suggests that the benefit comes from adding relevant microstructure information, not from maximizing feature count. For this thesis, Hypothesis 3 is therefore supported for the transition from a minimal snapshot representation to the selected microstructure-aware state, but not for the claim that every broader feature set is better.

6.6. Hypothesis 4: Reward Function Design

This section compares the log-return baseline against the Differential Sharpe Ratio reward on an otherwise identical configuration (selected 10-feature set, zero transaction cost), holding the algorithm and data fixed.

Table 7. Hypothesis 4 reward-design sensitivity: test-split metrics for log-return vs. Differential Sharpe Ratio.

Variant	Sharpe	Sortino	Return	Max DD	Win Rate	PF	Turnover
Log Return (baseline)	—	—	-1.74e-03	-1.74e-03	0.00%	0.000	0.0006
Differential Sharpe (DSR)	—	—	—	—	—	—	—

Source: *Author's own calculations.*

The reward-design comparison shows that Differential Sharpe Ratio optimization changes the learned policy in an economically meaningful direction. Relative to the log-return baseline, the DSR run increases cumulative return from 1.29×10^{-7} to 2.28×10^{-7}, improves profit factor from 2.01 to 5.70, and reduces maximum drawdown from -1.93×10^{-7} to -1.05×10^{-7}. The DSR policy is also more balanced in exposure, moving from an 86.7% long policy under log-return reward to an approximately neutral 49.8% long and 50.2% short distribution. This

Table 8. Benchmark Strategy Performance Comparison

Source: *Author's own calculations.*
Note: Placeholder values *— these figures are illustrative only and will be replaced once the full experiment pipeline has completed and evaluation artifacts are available.*

supports Hypothesis 4: the reward objective materially affects both performance and policy behavior.

Overall, the robustness evidence is strongest for transaction-cost sensitivity, feature specification, and reward-design sensitivity. It is not yet strong evidence of robustness across market regimes, longer samples, alternative chronological splits, or repeated random seeds.

Seed-level robustness was spot-checked outside the formal hypotheses: a small number of independent short-budget TD3 trials with different random seeds, using the baseline configuration, were run to confirm that the reported performance reflects systematic learning rather than a single favourable initialisation. A full multi-seed study at the training budget used in the main experiments was not carried out, so single-run variance remains a limitation (Section 6.1; Section 7.2.1).

6.7. Performance Evaluation

Returns, turnover, and path-dependent risk measures from the latest held-out run jointly test whether the TD3 agent behaves economically coherently.

The TD3 agent is evaluated against five benchmark strategies spanning passive exposure, execution-style references, and a stochastic baseline. The table below summarizes key risk-adjusted metrics for each strategy, with the agent's performance highlighted in the first row.

`Placeholder values shown - run full experiment pipeline to populate.`

TR: Total Return. Volatility: annualized standard deviation of returns. SR: Sharpe Ratio (mean excess return divided by standard deviation, not annualized). Sortino: Sortino Ratio (mean excess return divided by downside deviation, not annualized). Max DD: Maximum Drawdown. Win Rate: fraction of steps with positive return. Turnover: average absolute position change per step.

Return aggregation for ratio metrics. Roughly 80–90% of consecutive LOB returns are zero — the mid-price does not move between most order-book events. This causes $\hat{\sigma}$ to collapse faster than $\bar{r}$, inflating the naive per-tick Sharpe by $\sqrt{N}$ without any real edge. The pipeline therefore compounds per-tick returns into the coarsest bar size with at least 50 observations before computing Sharpe and Sortino, which are then reported as μ/σ and μ/σ_{down} on the aggregated series with no annualization. Total return, max drawdown, win rate, profit factor, and turnover are computed on the raw per-step series.

6.7.1. Evaluation Plots

Equity curve plot data not available — re-run evaluation to generate parquet artifacts: `uv run python src/cli.py evaluate -c pooled/td3_hft_lob_state_space_pooled_streaming_selec` `--output-dir logs/<run> --only metrics --only benchmarks --only plots`
Source: *Author's own calculations.*

Reward plot data not available — re-run evaluation to generate parquet artifacts: `uv run python src/cli.py evaluate -c pooled/td3_hft_lob_state_space_pooled_streaming_selec` `--output-dir logs/<run> --only metrics --only benchmarks --only plots`
Source: *Author's own calculations.*

Action/position plot data not available — re-run evaluation to generate parquet artifacts: `uv run python src/cli.py evaluate -c pooled/td3_hft_lob_state_space_pooled_streaming_se` `--output-dir logs/<run> --only metrics --only benchmarks --only plots`
Source: *Author's own calculations.*

6.7.2. Discussion: Reading Performance in the Context of State Design

The benchmark comparison places the TD3 agent's held-out behavior in direct relation to the reference strategies. Several features of the comparison are worth stating explicitly.

The test window is characterized by a slight market decline. The buy-and-hold benchmark accumulated a total return of -6.98×10^{-4} (-0.070%); TWAP and VWAP produced -5.73×10^{-4} (-0.057%) and -6.08×10^{-4} (-0.061%) respectively. In this environment, the TD3 agent recorded a total return of -1.40×10^{-4} (-0.014%) — better than all passive benchmarks. The agent's behavioral profile explains this outcome: with approximately 54.9% long and 45.1% short exposure and a per-step turnover of 0.058%, the agent accumulated only a fraction of the directional loss that sustained long-only exposure would imply in a declining market. The outperformance relative to passive benchmarks is therefore structural — a consequence of near-neutral positioning — rather than a sign of correctly timed directional bets.

The trade-level statistics reinforce this interpretation. The agent's win rate is 1.3% and its profit factor is 3.65×10^{-3}: the overwhelming majority of individual position adjustments result in a per-step loss, and the ratio of gross profit to gross loss is well below unity. These figures contrast sharply with buy-and-hold's profit factor of 0.968 and win rate of 48.4%. The agent avoids large directional losses by staying near-neutral, but it does not generate a stream of profitable individual trades.

On drawdown, the agent's maximum drawdown of -1.40×10^{-4} (-0.014%) is contained relative to buy-and-hold's -3.66×10^{-3} (-0.37%). This arises from the same source: near-zero net exposure limits the cumulative loss that any single directional move can inflict, regardless of whether that positioning reflects skill or passivity.

Taken together, the benchmark comparison establishes that the agent's outperformance is structural — near-neutral positioning that incidentally reduces exposure during a period when all passive long strategies lost money — rather than a sign of correctly timed directional bets. Its trade-level quality metrics (win rate, profit factor) are substantially worse than even a passive strategy, so the positive relative total return (-0.014% versus -0.057% to -0.070%) is entirely attributable to the neutral position stance, not profitable active trading. The Hypothesis 1 verdict from the benchmark comparison is therefore two-sided. The agent clears a weak feasibility bar — it does not commit capital to a sustained wrong-direction bet, and its relative total return over this declining window is better than passive long exposure — but the mechanism is near-neutral positioning, not profitable directional trading, and the comparison is run under mid-price execution with zero fees. That is not a neutral choice: zero-cost execution is a subsidy proportional to turnover, so a comparison whose only asymmetry is the spread the active arm does not pay favours the higher-turnover reinforcement learning policy over the near-static passive benchmarks. Any spread-crossing cost both erases the agent's own frictionless return (Section 6.2) and narrows its measured advantage over buy-and-hold. The benchmark comparison thus supports the existence half of Hypothesis 1 — a coherent, risk-controlled policy — but not a claim of benchmark-dominating or deployable performance; the economic magnitude of the edge is bounded by transaction cost, as Sections 4.3 (Section 4.3.1) and 6.2 establish.

7. Conclusions and Future Work

This thesis examined whether a continuous-control deep RL agent using order-book-derived microstructure features can achieve economically meaningful held-out performance in a high-frequency single-asset setting. The answer, within the specified experimental design, is qualified but affirmative: the following sections detail the evidence (Section 7.1), the limitations that bound this claim (Section 7.2), and their implications for trading-system design and practice.

7.1. Summary of Findings

The thesis set out to answer four hypotheses concerning deep reinforcement learning for high-frequency single-asset trading using order-book-derived features. This section summarizes the empirical evidence for each.

Hypothesis 1: a TD3 agent learns a real but cost-bounded risk-adjusted edge from order-book microstructure. The evidence supports this in its reworded form. Under frictionless mid-price execution the agent learns a directionally coherent, low-turnover, drawdown-controlled policy that beats passive benchmarks on this held-out window (Section 6.3), and TD3, DDPG, and PPO all separate cleanly from a random policy while remaining indistinguishable from one another (Section 6.1) — the microstructure signal exists, is learnable, and is not an artifact of a single algorithm's inductive bias. Its economic magnitude, however, is of the same order as the cost of trading on it: the features that carry the signal (microprice divergence, order-flow imbalance) cannot express an edge larger than one bid–ask half-spread per trade (Section 4.3), and the transaction-cost sweep (Section 6.2) shows the reported return changing sign once the per-trade fee reaches a fraction of a basis point — the same scale as that half-spread. The frictionless result is therefore a signal ceiling, not a deployable edge: profitable when execution is free, not expected to survive realistic spread-crossing cost. Retraining and re-evaluating under explicit bid/ask execution with calibrated fees is the outstanding test, identified as future work (Section 7.2).

Hypothesis 2: empirical performance is materially sensitive to the transaction-cost assumption. The fee sweep confirms this hypothesis and, in doing so, bounds Hypothesis 1. The learned edge is positive at zero fee, only marginally positive at 0.01 basis points, and negative from 0.1 basis points upward; the sign changes within a single order of magnitude of fee. The level at which the edge disappears coincides with the half-spread ceiling on the microstructure signal (Section 4.3), so Hypotheses 1 and 2 meet at the same quantity from opposite directions — Hypothesis 1 measuring the edge, Hypothesis 2 the cost that cancels it. That the reported result is this sensitive to a parameter often left implicit is why robustness analysis precedes any economic claim in this thesis.

Hypothesis 3: a broader microstructure-aware feature set improves out-of-sample performance relative to a simpler snapshot-based state representation. The feature-set comparison confirms this hypothesis, and feature specification was the dominant determinant of policy quality: setups using only snapshot-level quantities produced weaker, short-biased, less stable policies than those using the microstructure-aware feature set, which supplements top-of-book snapshot quantities with order-flow imbalance, spread-regime features, book-shape measures, and temporal encodings. The improvement was visible not only in summary metrics but in policy behavior — lower-turnover, more stable positioning. The full 33-feature set did not improve further on the selected 10-feature set, so the benefit comes from adding relevant microstructure information, not from maximizing feature count.

Hypothesis 4: the reward objective materially shapes the learned policy. The reward-design comparison confirms this hypothesis. Replacing the log-return reward with a Differential Sharpe Ratio reward increased cumulative return and profit factor, reduced maximum drawdown, and moved the policy from an 86.7% long bias to an approximately neutral long/short split, under otherwise identical training and evaluation conditions. Direct risk-adjusted optimization therefore changes both performance and the policy's directional exposure, not just the training signal's scale.

7.2. Limitations and Future Research

7.2.1. Execution Realism

The central limitation of the empirical setup is its execution model: mid-price-proxy valuation and reward computation, zero market impact, zero latency, and no baseline fee. For a microstructure strategy this assumption is close to the whole result. The microprice divergence that drives the agent's directional calls is a convex combination of the best bid and ask, so $|\text{microprice}(t) - \text{mid}(t)| \leq (\text{ask}(t) - \text{bid}(t))/2$ holds identically — an exact algebraic bound, verified on 100%

of the 2.4-million-row AAPL sample. The largest per-tick edge the feature set can express is therefore exactly one half-spread, which is precisely what a spread-crossing fill would cost; mid-price execution deletes that liability and leaves the signal as pure profit. This is the mechanism behind the signal-ceiling framing of Hypothesis 1 (Section 4.3), not merely a statement that "frictions could matter".

Order-of-magnitude figures on the exported Hypothesis 1 test window: the spread a crossing environment would have charged is roughly ten times the gross profit, and the breakeven proportional fee is on the order of 4×10^{-6} — consistent with the Hypothesis 2 fee sweep turning negative between 1×10^{-6} and 1×10^{-5}. Re-pricing the identical action path under bid/ask execution instead of the mid moves the total return from roughly $+2.3$ to roughly -10.6 in net-liquidation-value units: the same signal, the opposite sign. These figures are indicative rather than final — they come from re-pricing a fixed policy, not from retraining — and a control in which random actions run through the same frictionless environment returns -0.033%, confirming that the frictionless profit is a real signal harvested for free rather than bid–ask bounce available to any high-turnover policy. The direct next step, and the most important single extension, is to **retrain** and re-evaluate the agents under an explicit bid/ask fill rule with calibrated maker/taker fees and a slippage or market-impact model; because the training reward shares the mid-price assumption, re-scoring the existing checkpoints is not sufficient.

7.2.2. Evaluation Scope

The most important constraint on interpreting the results is the three-day evaluation window: train, validation, and test splits all derive from the same three consecutive sessions (February 25-27, 2026), sharing one volatility regime, spread environment, and intraday seasonality. What this thesis calls "out-of-sample" performance is therefore within-window out-of-sample — the test split was withheld from training, but the agent has never encountered a different trading day, market condition, or instrument. This is a necessary but not sufficient condition for practical utility: the results establish that the learning problem is tractable in this window, not that the policy generalizes beyond it. The comparisons are also based on a single completed run per configuration on a single machine with a modest compute budget, so multi-seed variance and the ceiling reachable under larger-scale infrastructure both remain open. Extending the window across volatility and liquidity regimes, walk-forward validation, multi-seed reporting, and cross-asset testing — including foreign exchange, futures, and cryptocurrency — would be the most direct ways to strengthen the generalization claim and determine whether asset-class-specific tuning is necessary.

7.2.3. Partial Observability

A more fundamental constraint, shared with most academic RL trading research, is that the agent sees only the spot limit order book of a single instrument at a single venue. Price-relevant information that never reaches that book — derivatives and options-implied signals, unstructured data such as earnings and macro releases, and proprietary order flow visible only to systematic internalizers, market makers, and prime brokers — appears to the agent as unexplained noise, which both limits value-estimate precision and raises reward variance. This creates a structural information asymmetry between academic research, confined to public data, and well-resourced industry participants; the results here are best read as a lower bound on what such participants could achieve with the same technique, not as a statement about the technique's ceiling. The reward signal inherits this noise directly, since tick-level returns are dominated by bid-ask bounce and transient order-flow effects the agent cannot attribute to a cause — a constraint on learning that exists independently of algorithm or reward choice, and one that action-frequency sub-sampling (acting every N ticks rather than every tick) could partially diagnose at low cost — several of the engineered features have been shown in the microstructure literature to carry predictive content over horizons of minutes, not just the next event, so a lower-frequency action interval might reveal signal the current tick-level setup discards as noise.

7.2.4. Training Workflow

Some TD3 runs entered unstable regimes — persistent action saturation or collapse to a narrow exposure range — and were manually excluded from the main comparison. This prevented obviously failed runs from contaminating the results, but it is not an automated or statistically principled criterion. Future work should make this explicit: validation-based checkpoint selection and early-stopping on diagnostics such as validation Sharpe, action saturation rate, and critic-value magnitude would reduce manual intervention and make cross-algorithm comparisons more reproducible.

TD3's stability corrections are also not complete. Target policy smoothing regularizes the critic to be locally consistent around actions seen in the replay buffer, but it does not eliminate extrapolation error: once the actor begins producing actions in regions the critic has seen rarely or never, the critic's estimates there are extrapolations from the training distribution rather than reliable values, and the actor receives no signal that it has left the buffer's support. This is distinct from the overestimation-bias correction discussed in Chapter 3 (Section 3.3) — the twin-critic minimum addresses optimistic bias at the target computation, not exposure to out-of-distribution actions at the actor update. Stochastic-policy methods such as SAC are structurally less exposed to this failure mode, since the entropy bonus discourages the policy from collapsing onto a narrow region of action space.

7.2.5. Future Research Directions

Beyond re-evaluating under realistic execution and broadening the evaluation window (above), several extensions follow directly from the design choices made here:

- **Richer agent state.** The observation includes only current exposure; unrealized P&L, holding duration, rolling turnover, and episode drawdown would add inventory-aware self-state without changing the market observation.
- **Temporal context.** The observation is a single snapshot, which aliases different price trajectories into the same state. A shallow lag stack (5-10 steps) of key features is a low-cost first step before the more substantial redesign implied by a sequence encoder.
- **Cross-asset and multi-asset extensions.** The pooled model trains across six instruments but manages one position at a time; synchronized cross-asset signals (e.g. VIX futures, sector-ETF flow) or genuine multi-asset portfolio optimization are natural extensions, as is testing on instruments with intermediate liquidity — active enough to generate a rich order book but not so deep that every pattern is immediately arbitraged, unlike the highly liquid large-caps used here. A more tractable intermediate step is a pair with an economically well-defined price anchor, such as dual-class shares representing identical economic exposure to the same underlying business, where deviations from parity are theoretically bounded and the learning target becomes the spread rather than a directional price level.
- **Algorithm and reward ablation.** Replacing TD3 with SAC or PPO, and the Differential

Sharpe Ratio with a Sortino-based or drawdown-penalized reward, would test how much of the observed performance is attributable to the specific algorithm and reward choice rather than the feature design. Within the MLP family, a structured search over learning rate, hidden-layer width and depth, and activation function would separate algorithmic contribution from parameterization: activation functions with different saturation properties, such as GELU or ELU in place of ReLU, may interact differently with the bounded, skewed distribution of order-book features, and layer normalization inside the critic — untested here — could stabilize Q-value scale across reward magnitudes.

- **Alternative architectures.** Transformer-based sequence encoders and hybrid gradient-boosted-tree approaches such as GBRL (Fuhrer et al., 2025) are promising directions given the tabular, engineered nature of the feature set — but a linear-policy baseline should be evaluated first: if a linear actor matches the MLP's out-of-sample performance, that is stronger evidence about the problem's structure than any higher-capacity model result would be.

7.3. Implications and Recommendations for Practitioners

The empirical findings carry implications for deep RL trading systems on limit order book data beyond the specific experiments studied here, and translate directly into recommendations for practitioners building similar systems.

Invest in feature engineering before algorithm selection. The performance gap between the extended and baseline feature sets shows that order-book-derived microstructure information — order flow imbalance, spread regime, book shape, temporal encoding — carries signal that aggregate snapshot quantities do not, consistent with the microstructure literature reviewed in Chapter 2. Feature quality is at least as consequential as algorithm choice: a well-specified feature set with a standard actor-critic architecture will likely outperform a poorly-specified feature set with a more sophisticated model. A thorough feature validation — correlation structure, stationarity, and predictive content on held-out data — is more valuable time spent than hyperparameter search over an impoverished state space.

Continuous-action actor-critic methods suit position-sizing tasks. The continuous action space let the agent express varying directional conviction through position size rather than binary buy/sell decisions, which matters when the economically correct position magnitude varies with signal strength; a discrete-action grid either loses this precision or reintroduces the curse of dimensionality. TD3's stability mechanisms — clipped double Q-learning, delayed policy updates, target policy smoothing — proved important in this noisy reward environment: without them, value overestimation would likely have driven the policy toward unreliable actions during early training.

Enforce a strict chronological evaluation protocol from the start. Any lookahead — features computed from future data, normalization fit on the full dataset, non-sequential splits — produces a materially optimistic out-of-sample assessment. In high-frequency settings, where signal decay is measured in seconds, even mild lookahead can manufacture the appearance of a viable strategy from a leakage artifact. Normalization should be fit on the training split only and applied frozen, or implemented as a strictly causal online transformation that resets at session boundaries. This is a minimum requirement for the results to be meaningful, not an optional refinement.

Keep reward optimization and financial evaluation strictly separate. A shaped reward such as the Differential Sharpe Ratio is a training objective, not a performance report (Section 4.3): conflating a high training DSR with held-out Sharpe performance is a common source of overoptimistic claims in the literature, and the most consistent failure mode to avoid.

Treat transaction-cost sensitivity as a go/no-go criterion. The execution model used here is a mid-price proxy, and represents an upper bound on realizable returns: spread-crossing costs are of similar magnitude to the price moves the agent targets, and the demonstrated edge does not survive at high cost levels, as the sensitivity analysis shows. Before treating any result as evidence of a deployable strategy, practitioners should run the same evaluation across a range of cost levels — from zero to the realistic spread-plus-fee regime of the target venue — and check whether risk-adjusted return degrades gracefully or collapses abruptly. An edge that survives only at zero cost is not a tradeable strategy.

7.4. Conclusion

The central question of this thesis was: can a LOB-informed deep RL agent achieve meaningful out-of-sample trading performance in a high-frequency setting? The answer, under the specified experimental conditions, is affirmative but bounded. Under frictionless mid-price execution, the TD3 agent trained on microstructure-aware limit order book features learned a directionally coherent, risk-controlled policy that outperformed the evaluated benchmark strategies on this held-out window, and the extended feature set provided a measurable improvement over the simpler snapshot-based state representation. That outperformance is real but cost-bounded: the edge the microstructure features can express is of the order of the bid–ask half-spread, so it is realised under frictionless execution and is not expected to survive realistic spread-crossing cost (Section 7.2). These findings hold together rather than in isolation: the performance advantage of the richer state design is visible in both summary metrics and in the qualitative character of the learned policy — lower unnecessary turnover, more responsive positioning, and more contained drawdown relative to passive exposure.

The contribution of the thesis is methodological as much as empirical. The end-to-end pipeline developed here — covering feature engineering from 10-level order book data, structured experiment management, TD3 training with Differential Sharpe Ratio reward, and a rigorous separation between training diagnostics and held-out financial evaluation — provides a replicable framework for this class of research problem. The emphasis throughout on design transparency, chronological data integrity, and explicit simulation assumptions reflects a view that the quality of the experimental setup is inseparable from the soundness of its conclusions.

At the same time, the thesis is explicit about what it does not claim. Within the constraints of the controlled setup, the observed edge cannot be straightforwardly mapped to a deployable trading strategy. Whether the microstructure signal captured by the learned policy survives realistic spread-crossing costs, maker/taker fees, queue uncertainty, and latency remains an open and important empirical question — one that the limitations and future research section identifies as the immediate extension of the current work.

What the thesis does establish is that the combination of limit order book data, LOB-informed feature engineering, and continuous-action actor-critic learning is a productive research direction. The informational content of the order book, well-established in the theoretical microstructure literature, can be translated into state features that improve a reinforcement learning agent's held-out behavior in ways that are measurable and consistent with economic intuition. That conclusion is the foundation on which more operationally realistic extensions — wider data windows, multi-seed validation, explicit bid/ask execution, and cross-asset generalization — can now be built.

Bibliography

Bellman, R. (1957). *Dynamic programming*. Princeton University Press.

Biais, B., Hillion, P., & Spatt, C. (1995). An empirical analysis of the limit order book and the order flow in the Paris Bourse. *Journal of Finance*, *50*(5), 1655–1689. https://doi.org/10.1111/j.1540-6261.1995.tb05192.x

Boehmer, E., Jones, C. M., Zhang, X., & Zhang, X. (2021). Tracking retail investor activity. *Journal of Finance*, *76*(5), 2249–2305. https://doi.org/10.1111/jofi.13033

Bouchaud, J.-P., Mézard, M., & Potters, M. (2002). Statistical properties of stock order books: Empirical results and models. *Quantitative Finance*, *2*(4), 251–256. https://doi.org/10.1088/1469-7688/2/4/301

Cao, C., Hansch, O., & Wang, X. (2009). The information content of an open limit-order book. *Journal of Futures Markets*, *29*(1), 16–41. https://doi.org/10.1002/fut.20334

Cont, R., Kukanov, A., & Stoikov, S. (2014). The price impact of order book events. *Journal of Financial Econometrics*, *12*(1), 47–88. https://doi.org/10.1093/jjfinec/nbt002

Dempster, M. A. H., & Romahi, Y. (2002). Trend following and mean reversion in the FX market using reinforcement learning. *Quantitative Finance*, *2*(3), 159–176.

Deng, Y., Bao, F., Kong, Y., Ren, Z., & Dai, Q. (2017). Deep direct reinforcement learning for financial signal representation and trading. *IEEE Transactions on Neural Networks and Learning Systems*, *28*(3), 653–664. https://doi.org/10.1109/TNNLS.2016.2522401

Easley, D., López de Prado, M. M., & O'Hara, M. (2012). Flow toxicity and liquidity in a high-frequency world. *Review of Financial Studies*, *25*(5), 1457–1493. https://doi.org/10.1093/rfs/hhs053

Engle, R. F. (2000). The econometrics of ultra-high-frequency data. *Econometrica*, *68*(1), 1–22. https://doi.org/10.1111/1468-0262.00091

Fuhrer, B., Tessler, C., & Dalal, G. (2025). Gradient boosting reinforcement learning. *Proceedings of the Forty-Second International Conference on Machine Learning*, *267*, 17960–17985. https://arxiv.org/abs/2407.08250

Fujimoto, S., Hoof, H. van, & Meger, D. (2018). Addressing function approximation error in actor-critic methods. *ICML*, 1587–1596.

Glosten, L. R. (1994). Is the electronic open limit order book inevitable? *Journal of Finance*, *49*(4), 1127–1161. https://doi.org/10.1111/j.1540-6261.1994.tb02450.x

Glosten, L. R., & Milgrom, P. R. (1985). Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, *14*(1), 71–100. https://doi.org/10.1016/0304-405X(85)90044-3

Gold, C. (2003). FX trading via recurrent reinforcement learning. *Proceedings of the 2003 IEEE International Conference on Computational Intelligence for Financial Engineering*, 363–370. https://doi.org/10.1109/CIFER.2003.1196283

Gort, B. J. D., Liu, X.-Y., Sun, X., Gao, J., Chen, S., & Wang, C. D. (2022). *Deep reinforcement learning for cryptocurrency trading: Practical approach to address backtest overfitting*. https://arxiv.org/abs/2209.05559

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *ICML*, 1861–1870.

Harris, L. (1986). A transaction data study of weekly and intradaily patterns in stock returns. *Journal of Financial Economics*, *16*(1), 99–117. https://doi.org/10.1016/0304-405X(86)90044-9

Harris, L. (1991). Stock price clustering and discreteness. *Review of Financial Studies*, *4*(3), 389–415. https://doi.org/10.1093/rfs/4.3.389

Hasselt, H. van, Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *30*. https://doi.org/10.1609/aaai.v30i1.10295

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep Reinforcement Learning That Matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, *32*. https://doi.org/10.1609/aaai.v32i1.11796

Jiang, Z., & Liang, J. (2017). Cryptocurrency portfolio management with deep reinforcement learning. *arXiv Preprint arXiv:1612.01277*. https://arxiv.org/abs/1612.01277

Jiang, Z., Xu, D., & Liang, J. (2017). *A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem*. 1–31. http://arxiv.org/abs/1706.10059

Kabbani, T., & Duman, E. (2022). Deep reinforcement learning approach for trading automation in the stock market. *IEEE Access*, *10*, 93564–93574. https://doi.org/10.1109/ACCESS.2022.3203697

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv Preprint* *arXiv:1412.6980*. https://arxiv.org/abs/1412.6980

Krauss, C., Do, X. A., & Huck, N. (2017). Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the s&p 500. *European Journal of Operational Research*, *259*(2), 689–702. https://doi.org/10.1016/j.ejor.2016.10.031

Kyle, A. S. (1985). Continuous auctions and insider trading. *Econometrica*, *53*(6), 1315–1335. https://doi.org/10.2307/1913210

Lee, C. M. C., & Ready, M. J. (1991). Inferring trade direction from intraday data. *Journal of Finance*, *46*(2), 733–746. https://doi.org/10.1111/j.1540-6261.1991.tb02683.x

Lee, J. W., Park, J., O, J., Lee, J., & Hong, E. (2007). A multiagent approach to Q-learning for daily stock trading. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, *37*(6), 864–877. https://doi.org/10.1109/TSMCA.2007.904825

Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J., Jordan, M., & Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. *Proceedings of the 35th International Conference on Machine Learning, Proceedings of Machine Learning Research*, *80*, 3053–3062.

Liang, Z., Chen, H., Zhu, J., Jiang, K., Li, Y., & Zhu, W. (2018). Adversarial deep reinforcement learning in portfolio management. *arXiv Preprint arXiv:1808.09940*. https://arxiv.org/abs/1808.09940

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. *International Conference on Learning Representations*.

Lin, L.-J. (1992). Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, *8*(3–4), 293–321. https://doi.org/10.1007/BF00992699

Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., & Wang, C. D. (2020). FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance. *arXiv Preprint arXiv:2011.09607*. https://arxiv.org/abs/2011.09607

López de Prado, M. (2018). *Advances in financial machine learning*. Wiley.

Majidi, N., Shamsi, M., & Marvasti, F. (2024). Algorithmic trading using continuous action space deep reinforcement learning. *Expert Systems with Applications*, *237*, 121292. https://doi.org/10.1016/j.eswa.2023.121292

Millea, A. (2021). Deep reinforcement learning for trading—a critical survey. *Data*, *6*(11), 119.

https://doi.org/10.3390/data6110119

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*(7540), 529–533. https://doi.org/10.1038/nature14236

Mohammadshafie, A., Mirzaeinia, A., Jumakhan, H., & Mirzaeinia, A. (2024). *Deep reinforcement learning strategies in finance: Insights into asset holding, trading behavior, and purchase diversity*. https://arxiv.org/abs/2407.09557

Moody, J., & Saffell, M. (1998). *Reinforcement learning for trading*.

Moody, J., & Saffell, M. (2001). Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, *12*(4), 875–889. https://doi.org/10.1109/72.935097

Neuneier, R. (1998). Enhancing q-learning for optimal asset allocation. *Advances in Neural Information Processing Systems*, *10*, 936–942.

Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. *ICML*, 278–287.

Noonan, L. (2017). *JPMorgan to Take AI Trading Global*. https://www.ft.com/content/16b8ffb6-7161-11e7-aca6-c6bd07df1a3c?syn-25a6b1a6=1

Puterman, M. L. (1994). *Markov decision processes: Discrete stochastic dynamic programming*. John Wiley & Sons.

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, *22*(268), 1–8. http://jmlr.org/papers/v22/20-1364.html

Roll, R. (1984). A simple implicit measure of the effective bid-ask spread in an efficient market. *Journal of Finance*, *39*(4), 1127–1139.

Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015). Trust region policy optimization. *ICML*, 1889–1897.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv Preprint arXiv:1707.06347*. https://arxiv.org/abs/1707.06347

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014). Deterministic policy gradient algorithms. *Proceedings of the 31st International Conference on Machine*

Learning (ICML), 387–395.

Stoikov, S. (2018). The micro-price: A high-frequency estimator of future prices. *Quantitative Finance*, *18*(12), 1959–1966. https://doi.org/10.1080/14697688.2018.1489139

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (2000). Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, *12*, 1057–1063.

Tsitsiklis, J. N., & Van Roy, B. (1997). An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control*, *42*(5), 674–690. https://doi.org/10.1109/9.580874

Tulchinsky, I. (Ed.). (2019). *Finding alphas: A quantitative approach to building trading strategies* (2nd ed.). Wiley.

Wang, J., Zhang, Y., Tang, K., Wu, J., & Xiong, Z. (2019). AlphaStock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 1900–1908. https://doi.org/10.1145/3292500.3330647

Watkins, C. J. C. H. (1989). *Learning from delayed rewards*. King's College, University of Cambridge.

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, *8*(3–4), 279–292. https://doi.org/10.1007/BF00992698

Welford, B. P. (1962). Note on a method for calculating corrected sums of squares and products. *Technometrics*, *4*(3), 419–420. https://doi.org/10.1080/00401706.1962.10490022

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, *8*(3–4), 229–256. https://doi.org/10.1007/BF00992696

Wojdalski, K. (2026). *Deep reinforcement learning for high-frequency trading: Master's thesis codebase*. https://github.com/kwojdalski/masters_thesis

Wood, R. A., McInish, T. H., & Ord, J. K. (1985). An investigation of transactions data for NYSE stocks. *Journal of Finance*, *40*(3), 723–739.

XAI Asset Management. (2024). *TradingEnv: An event-driven market simulator for backtesting* *and reinforcement learning*. https://github.com/xaiassetmanagement/tradingenv

Xiong, Z., Liu, X.-Y., Zhong, S., Yang, H., & Walid, A. (2018). Practical deep reinforcement learning approach for stock trading. *arXiv Preprint arXiv:1811.07522*. https://arxiv.org/abs/1811.07522

Yang, H., Liu, X.-Y., Zhong, S., & Walid, A. (2020). *Deep reinforcement learning for automated stock trading: An ensemble strategy* (pp. 1–8). Proceedings of the First ACM International Conference on AI in Finance. https://doi.org/10.1145/3383455.3422540

Zhang, Z., Zohren, S., & Roberts, S. (2019). DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, *67*(11), 3001–3012. https://doi.org/10.1109/TSP.2019.2907260

Zhang, Z., Zohren, S., & Roberts, S. (2020). Deep reinforcement learning for trading. *The Journal of Financial Data Science*, *2*(2), 25–40. https://doi.org/10.3905/jfds.2020.1.030

List of Tables

List of Figures

List of Appendices

A. Feature Inventory

The tables below catalogue every feature type implemented in the feature registry. The subset used in the main state-space experiment is marked in the **In model** column: ✓ indicates all variants are used, while a specific parameter value indicates only that variant is used. Feature names match the keys produced by the feature pipeline at runtime.

The ten static LOB features used in the main HFT experiment are z-score normalized with causal running statistics that are updated chronologically and reset at detected session boundaries. The runtime position feature is appended by the environment and remains in its raw bounded scale $[-1, 1]$. The cyclical time encodings listed in the registry are inherently bounded but are not part of the selected TD3 specification.

A.1. LOB Microstructure Features

Notation. P_i^{bid}, P_i^{ask}: bid/ask price at book level i; V_i^{bid}, V_i^{ask}: resting volume at level i; C_i^{bid}, C_i^{ask}: order count at level i; $M_t = \frac{1}{2}(P_0^{bid} + P_0^{ask})$: mid-price; $S_t = P_0^{ask} - P_0^{bid}$: spread; $\bar{P}_N^{vw} = \frac{\sum_{i<N}(P_i^{bid}V_i^{bid}+P_i^{ask}V_i^{ask})}{\sum_{i<N}(V_i^{bid}+V_i^{ask})}$: volume-weighted mid-price over the top N levels; $\Delta V_t^{bid} = V_{0,t}^{bid} - V_{0,t-1}^{bid}$, $\Delta V_t^{ask} = V_{0,t}^{ask} - V_{0,t-1}^{ask}$: tick-to-tick volume change; W: rolling window in events; $\sum_W x_t = \sum_{\tau=t-W+1}^{t} x_\tau$: rolling sum; $\bar{x}_W$: rolling mean over W events;

V_W^B, V_W^S: rolling buy/sell volume over W events; L: odd-lot size threshold (100 shares); Q: notional or size threshold; h_t: hour of day (0–23); Δt_{ns}: inter-event time in nanoseconds; $\mathbf{1}[\cdot]$: indicator function; $\hat{\rho}(\cdot, \text{lag} = 1)$: sample autocorrelation at lag 1.

Table 9. Full LOB feature registry with formulas and citations.

Feature	Group	Formula	Param	In model	Citation
hft_book_pressure	Imbalance	$(V_i^{bid} - V_i^{ask})/(V_i^{bid} + V_i^{ask})$	$i \in \{0, 1, 2\}$	level 0	Cao et al. (2004); Biais et al. (1995)
hft_order_book_imbalance	Imbalance	$\dfrac{\sum_{i<N}(V_i^{bid} - V_i^{ask})}{\sum_{i<N}(V_i^{bid} + V_i^{ask})}$	$N = 3$	✓	Cont et al. (2014)
hft_order_count_imbalance	Imbalance	$(C_i^{bid} - C_i^{ask})/(C_i^{bid} + C_i^{ask})$	$i = 0$	level 0	Biais et al. (1995)
hft_microprice	Fair value	$(V_0^{ask} P_0^{bid} + V_0^{bid} P_0^{ask})/(V_0^{bid} + V_0^{ask})$	—	✓	Stoikov (2018)
hft_microprice_divergence	Fair value	Microprice $- M_t$	—	✓	Stoikov (2018)
hft_vwmp_skew	Fair value	$(\bar{P}_N^{vw} - M_t)/S_t$	$N = 3$		Stoikov (2018)
hft_price_vamp	Fair value	Avg. execution price for fixed-notional order walked through N levels	$Q \in \{1k, 5k\}$		—
hft_depth_ratio	Depth	$(V_0^{bid} + V_0^{ask})\ /\ \sum_{i=1}^{N}(V_i^{bid} + V_i^{ask})$	$N = 4$		Bouchaud et al. (2002)
hft_spread_bps	Spread/cost	$(P_0^{ask} - P_0^{bid})/M_t \times 10{,}000$	—		Kyle (1985); Glosten & Milgrom (1985)
hft_spread_ratio	Spread/cost	$\text{SpreadBps}_t\ /\ \overline{\text{SpreadBps}}_W$	$W = 50$		Easley et al. (2012)
hft_bid_convexity	Spread/cost	$(P_0^{bid} - P_1^{bid}) - (P_1^{bid} - P_2^{bid})$	bid		Bouchaud et al. (2002)
hft_ask_convexity	Spread/cost	$(P_2^{ask} - P_1^{ask}) - (P_1^{ask} - P_0^{ask})$	ask		Bouchaud et al. (2002)
hft_bid_slope	Spread/cost	$(P_0^{bid} - P_4^{bid})\ /\ \sum_{i<5} V_i^{bid}$	5 levels	✓	Bouchaud et al. (2002); Kyle (1985)
hft_ask_slope	Spread/cost	$(P_4^{ask} - P_0^{ask})\ /\ \sum_{i<5} V_i^{ask}$	5 levels	✓	Bouchaud et al. (2002); Kyle (1985)

Table 9 continued.

Feature	Group	Formula	Param	In model	Citation
`hft_ofi`	Order flow	$\Delta V_t^{bid} \cdot \mathbf{1}[P_{0,t}^{bid} \geq P_{0,t-1}^{bid}] - \Delta V_t^{ask} \cdot \mathbf{1}[P_{0,t}^{ask} \leq P_{0,t-1}^{ask}]$	—	✓	Cont et al. (2014)
`hft_ofi_rolling`	Order flow	$\sum_W \mathrm{OFI}_t$	$W = 50$	$W = 50$	Cont et al. (2014)
`hft_ofi_multilevel`	Order flow	$\sum_{i<N} \mathrm{OFI}_t^{(i)}$ (event classification at each level i)	$N \in \{3, 5\}$		Cont et al. (2014)
`hft_bid_queue_depletion`	Order flow	$\max(V_{0,t-1}^{bid} - V_{0,t}^{bid},\, 0) \,/\, V_{0,t-1}^{bid}$	bid		Foucault et al. (2005)
`hft_ask_queue_depletion`	Order flow	$\max(V_{0,t-1}^{ask} - V_{0,t}^{ask},\, 0) \,/\, V_{0,t-1}^{ask}$	ask		Foucault et al. (2005)
`hft_signed_trade_flow`	Order flow	$\sum_W \mathrm{size}_\tau \cdot (\mathbf{1}[\mathrm{side}_\tau = \mathrm{B}] - \mathbf{1}[\mathrm{side}_\tau = \mathrm{A}])$	$W = 50$	$W = 50$	Lee & Ready (1991)
`hft_odd_lot_trade_ratio`	Order flow	$\sum_W \mathbf{1}[\mathrm{trade} \wedge \mathrm{size} < L] \,/\, \sum_W \mathbf{1}[\mathrm{trade}]$	$W = 200$		Boehmer et al. (2021)
`hft_odd_lot_imbalance`	Order flow	$(V_W^{B,\mathrm{odd}} - V_W^{S,\mathrm{odd}})/(V_W^{B,\mathrm{odd}} + V_W^{S,\mathrm{odd}})$; size $< L$	$W = 200$		Boehmer et al. (2021)
`hft_vpin`	Order flow	$\lvert V_W^B - V_W^S \rvert \,/\, (V_W^B + V_W^S)$	$W \in \{100, 500\}$		Easley et al. (2012)
`hft_cancel_to_trade`	Order flow	$\sum_W \mathbf{1}[\mathrm{action} = \mathrm{C}] \,/\, \sum_W \mathbf{1}[\mathrm{action} = \mathrm{T}]$	$W \in \{200, 1000\}$		Foucault et al. (2005)
`hft_trade_arrival_rate`	Order flow	$\sum_W \mathbf{1}[\mathrm{action} = \mathrm{T}]$	$W \in \{100, 500\}$		Engle (2000)
`hft_large_trade_ratio`	Order flow	$\sum_W \mathbf{1}[\mathrm{trade} \wedge \mathrm{size} \geq Q] \,/\, \sum_W \mathbf{1}[\mathrm{trade}]$	$W = 200$		—
`hft_ofi_autocorr`	Order flow	$\hat{\rho}(\mathrm{OFI}_{t-W:t},\, \mathrm{lag} = 1)$	$W \in \{50, 200\}$		Cont et al. (2014)
`hft_inter_event_time`	Regime/temp.	$\log(1 + \Delta t_{\mathrm{ns}})$	—		Engle (2000)
`hft_mid_price_acceleration`	Regime/temp.	$M_t - 2M_{t-1} + M_{t-2}$	—		Abergel et al. (2016)
`hft_hour_sin`	Regime/temp.	$\sin(2\pi h_t/24)$	—		Wood et al. (1985)
`hft_hour_cos`	Regime/temp.	$\cos(2\pi h_t/24)$	—		Wood et al. (1985)

Source: *author's own compilation; each formula is attributed to its originating study in the Citation column.*

B. Main Experiment Specification

The table below condenses the main experiment into a self-contained research specification. It records the empirical choices needed to reproduce the experiment. Numerical hyperparameters are loaded from the exported thesis snapshot so that the table stays in sync with the scenario configuration without manual updates.

Experiment hyperparameters not found — run: `uv run python scripts/export_eval_to_thesis.` *`--scenario pooled/td3_hft_lob_state_space_pooled_streaming_selected_dsr`*

C. Baseline Algorithm Pseudocode

Chapter 5.2 gives TD3's full training loop (Algorithm 2) since TD3 is this thesis's primary method. DDPG and PPO are baseline comparisons used in Chapter 6's algorithm comparison; their training loops are recorded here for reproducibility.

Algorithm 3 DDPG Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; DDPG hyperparameters

Ensure: Trained policy μ_ϕ, evaluation metrics

Initialise actor μ_ϕ, critic Q_θ, target networks $\mu_{\phi'}$, $Q_{\theta'}$, replay buffer $\mathcal{D}$

for each step $t = 1, \ldots, \texttt{max\_steps}$ **do**

  $a_t \leftarrow \mathrm{clip}\big(\mu_\phi(s_t) + \epsilon,\ -1,\ 1\big), \quad \epsilon \sim \mathcal{N}(0,\, \sigma^2_{\text{explore}})$

  Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$

  Sample mini-batch $\mathcal{B} \sim \mathcal{D}$

  Compute Bellman target: $y \leftarrow r + \gamma \cdot Q_{\theta'}(s',\, \mu_{\phi'}(s'))$

  Update critic by minimising $\mathcal{L}_{\text{Smooth-L1}}(y - Q_\theta(s, a))$

  Update actor: $\phi \leftarrow \phi + \alpha \nabla_\phi Q_\theta(s,\, \mu_\phi(s))$

  Soft-update targets: $\theta' \leftarrow \tau\theta + (1-\tau)\theta';\ \phi' \leftarrow \tau\phi + (1-\tau)\phi'$

  Checkpoint and log metrics at configured intervals

end for

Evaluate deterministic policy μ_ϕ on test split

Compute financial performance metrics from realised return series

Algorithm 4 PPO Training (follows Algorithm 0)

Require: Processed environments from Algorithm 0; PPO hyperparameters

Ensure: Trained policy π_θ, evaluation metrics

Initialise stochastic actor π_θ (TanhNormal), value network V_ϕ

for each batch iteration $k = 1, \dots, \texttt{max\_steps/frames\_per\_batch}$ **do**

  Collect trajectory $\mathcal{T}_k$ of `frames_per_batch` steps using π_θ

  Compute advantage estimates $\hat{A}_t$ via GAE using V_ϕ on $\mathcal{T}_k$

  for each PPO epoch $j = 1, \dots, K$ **do**

    Sample mini-batch $\mathcal{B} \sim \mathcal{T}_k$

    Compute probability ratio: $\rho_t(\theta) \leftarrow \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$

    Compute clipped objective: $\mathcal{L}^{\text{CLIP}} \leftarrow \mathbb{E}\Big[\min\Big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1 \pm \epsilon)\hat{A}_t\Big)\Big]$

    Compute value loss: $\mathcal{L}^V \leftarrow \mathcal{L}_{\text{L2}}\big(V_\phi(s) - V^{\text{target}}\big)$

    Compute entropy bonus: $\mathcal{L}^H \leftarrow c_1 \mathcal{H}(\pi_\theta(\cdot \mid s))$

    Update θ, ϕ jointly: minimise $-\mathcal{L}^{\text{CLIP}} + c_2\mathcal{L}^V - \mathcal{L}^H$

  end for

  Checkpoint and log metrics at configured intervals

end for

Evaluate policy mode $\tanh(\mu_\theta(s))$ on test split

Compute financial performance metrics from realised return series

For DDPG (beyond the symbols already defined in Chapter 5.2 for TD3):

- μ_ϕ is the deterministic actor; Q_θ is DDPG's single critic
- Primed symbols denote slowly updated target networks
- $\mathcal{D}$ is the replay buffer; τ is the soft-update coefficient

For PPO:

- π_θ is the stochastic actor (TanhNormal distribution)
- V_ϕ is the state value network
- $\hat{A}_t$ are GAE advantage estimates
- $\rho_t(\theta)$ is the probability ratio between new and old policies
- ϵ is the clipping threshold
- c_1, c_2 are the entropy and value loss coefficients

- K is the number of PPO epochs per batch

D. Raw Data Inventory

Table 10 reports raw MBP-10 event counts and inter-event timing per symbol and split, before the level-0-4 filter described in Section 5.3.7 is applied. Training aggregates all three training days per symbol (February 25-27, 2026); validation and test each cover one chronological half of the March 2 session.

Table 10. Raw MBP-10 event counts and inter-event timing per symbol and split.

Symbol	Split	Events	Trades	Mean Δt	Median Δt
AAPL	Train	4,575,844	402,135	16.57 ms	39.1 µs
AMZN	Train	7,101,442	384,315	10.36 ms	64.7 µs
AVGO	Train	1,636,373	285,066	49.38 ms	145.9 µs
META	Train	1,105,240	161,099	71.22 ms	242.2 µs
MSFT	Train	3,590,446	518,469	22.02 ms	50.4 µs
TSLA	Train	5,611,429	692,119	13.89 ms	38.9 µs
AAPL	Val	1,207,098	66,269	7.51 ms	28.1 µs
AAPL	Test	1,207,098	92,876	13.07 ms	38.8 µs
AMZN	Val	1,354,730	53,610	6.44 ms	60.1 µs
AMZN	Test	1,354,730	69,455	11.53 ms	82.5 µs
AVGO	Val	296,905	37,745	34.62 ms	136.1 µs
AVGO	Test	296,906	45,813	53.21 ms	148.7 µs
META	Val	188,074	24,474	58.99 ms	318.2 µs
META	Test	188,075	26,375	78.95 ms	304.6 µs
MSFT	Val	490,610	70,938	21.87 ms	55.4 µs
MSFT	Test	490,610	65,798	31.04 ms	65.4 µs
TSLA	Val	999,867	117,916	9.68 ms	31.5 µs
TSLA	Test	999,868	115,563	16.10 ms	42.2 µs

Source: *Author's own calculations based on DataBento Nasdaq MBP-10 data.*

Note: *Mean and median Δt computed from positive within-session inter-event deltas only (overnight gaps excluded for the training split). Training aggregates all three days per symbol (February 25–27, 2026); validation and test each cover one chronological half of the March 2 session.*

E. Transformed Feature Example

Table 11 shows twelve consecutive events from the AAPL feed, illustrating how the raw order-book state is transformed into the input features used by the RL agent. Each row is one order-book event; the left columns display the raw state (best bid/ask prices and sizes), the right columns show the corresponding normalized microstructure features computed from that state via the causal running normalization in Equation 28. The table is wide (nine columns), so in the PDF it is placed on its own landscape-oriented page; the HTML version scrolls horizontally instead.

Table 11. Twelve consecutive AAPL LOB events and their transformed microstructure features.

Timestamp (UTC)	Best Bid ($)	Best Ask ($)	Bid Size	Ask Size	Book Pressure	Order Imbalance	Microprice Dev.	OFI
16:53:18.611	265.71	265.74	240	124	0.408	0.284	0.213	0.004
16:53:19.027	265.71	265.74	242	124	0.415	0.288	0.216	0.05
16:53:19.044	265.72	265.74	25	124	-1.418	-0.288	-0.489	0.578
16:53:19.044	265.72	265.74	71	124	-0.689	-0.157	-0.248	1.061
16:53:19.044	265.72	265.74	96	124	-0.42	-0.09	-0.159	0.578
16:53:19.044	265.72	265.74	50	124	-0.974	-0.216	-0.342	-1.052
16:53:19.044	265.72	265.74	25	124	-1.418	-0.288	-0.489	-0.57
16:53:19.044	265.71	265.74	242	124	0.415	0.288	0.216	-0.57
16:53:19.612	265.71	265.74	242	99	0.595	0.358	0.305	0.578
16:53:19.612	265.72	265.74	10	99	-1.701	-0.261	-0.582	0.234
16:53:19.612	265.72	265.74	10	90	-1.67	-0.234	-0.572	0.211
16:53:19.612	265.72	265.74	35	90	-1.001	-0.16	-0.351	0.578

Source: *DataBento Nasdaq MBP-10 feed, AAPL, 2 March 2026.*

Note: *Twelve consecutive order-book events selected from the test split to include multiple bid/ask price changes. Book Pressure, Order Imbalance, Microprice Dev., and OFI are z-score normalized using causal running statistics. A vertical rule separates the raw order-book state (left columns) from the normalized features (right columns).*

F. Feature Distribution Statistics

Table 12 reports distributional statistics for all engineered microstructure features computed over the training split, after skipping the first 500 events to allow rolling-window features to stabilise. Means close to zero and standard deviations near one confirm that causal running normalisation is operating as intended. Q1, Q2, and Q3 show the interquartile spread; Skew and Kurt (excess kurtosis) characterise the shape of each marginal distribution.

Several features — most notably `ofi` and `ofi_rolling_50` — exhibit extreme minimum and maximum values despite their near-unit standard deviations. This is a structural property of order-flow imbalance, not a data-quality issue. Most tick events represent small queue adjustments that produce z-scores close to zero, while occasional aggressive orders consume multiple price levels in rapid succession. Because the running standard deviation is dominated by the quiet majority, these rare spikes emerge as z-scores of ± 290 or more and produce the large positive excess kurtosis visible for OFI features.

The values in Table 12 are post-normalisation but pre-clipping. The environment applies a hard observation clip of $[-5, 5]$ through `obs_clip`, so the policy network never receives the extreme tails shown in the table.

G. Feature–Return Correlations

Table 13 reports Pearson and Spearman rank correlations between each engineered feature and the one-step-ahead log mid-price return, computed on the training split; see Section 5.3.5 for the interpretation of this result.

Table 12. Distributional statistics of engineered microstructure features.

Feature	Mean	Std	Skew	Kurt	Median	Min	Max
book_pressure_l0	-0.0559	1.0040	-0.088	-1.209	-0.0669	-2.1249	1.6668
book_pressure_l1	-0.0264	0.9993	-0.126	-0.995	-0.0123	-2.3312	1.8654
book_pressure_l2	-0.0028	1.0049	-0.134	-0.837	0.0046	-2.3795	2.1048
order_book_imbalance_3l	-0.0011	0.9743	-0.109	-0.580	0.0066	-2.7662	2.4845
order_count_imbalance_l0	-0.0082	0.9623	-0.021	-0.654	-0.0823	-2.5993	2.4929
microprice	-0.4964	1.2389	-0.306	-0.604	-0.4123	-3.7873	2.1839
microprice_divergence	-0.0580	0.7350	-0.018	0.532	-0.0864	-4.7804	3.4762
vwmp_skew	0.0057	0.9139	-0.352	11.077	-0.0056	-11.6931*	11.4695*
price_vamp	-0.4937	1.2460	-0.299	-0.607	-0.4341	-3.7420	2.1853
spread_bps	-0.4088	0.7351	1.095	3.211	-0.5086	-1.7766	5.5628*
spread_ratio_50	-0.0019	0.9194	0.948	3.124	-0.0845	-2.2276	8.6443*
bid_convexity	0.0026	0.5498	0.389	22.060	-0.0455	-6.3083*	6.1122*
ask_convexity	0.0159	0.5845	-0.164	7.577	0.0909	-3.8631	4.0888
bid_slope	-0.0033	0.0102	3.235	19.792	-0.0061	-0.0202	0.2053
ask_slope	-0.0025	0.0114	5.005	62.510	-0.0057	-0.0172	0.2491
depth_ratio	-0.1697	0.7502	4.154	32.654	-0.3624	-0.9259	11.1724*
ofi	-0.0207	0.6708	-6.566	459.691	-0.0064	-44.2877*	17.7070*
ofi_rolling_50	-0.0954	0.6801	-0.378	8.618	-0.0708	-6.1448*	6.2203*
ofi_multilevel_3l	-0.0187	0.6260	-1.582	53.674	-0.0072	-18.2282*	8.9735*
ofi_autocorrelation_50	-0.0532	0.9754	0.058	0.617	-0.1070	-4.4556	4.4679

Computed on the training split; the first 500 events are skipped to allow rolling-window features to reach steady state. Skew and Kurt are the Fisher skewness and excess kurtosis. Median is the 50th percentile. * *the RL environment clips observations to* ± 5*; starred values exceed this bound.*

Table 13. Pearson and Spearman rank correlations between each engineered feature and the next-step log return, computed on the training split.

Feature	Pearson	Spearman
microprice	+0.0082	+0.0189
microprice_divergence	-0.0071	-0.0027
book_pressure_l0	-0.0053	-0.0032
ofi_rolling_50	+0.0043	+0.0060
bid_slope	-0.0040	-0.0076
order_count_imbalance_l0	-0.0023	+0.0046
signed_trade_flow_50	+0.0019	+0.0001
ask_slope	-0.0014	+0.0050
order_book_imbalance_3l	+0.0004	+0.0052
ofi	+0.0000	+0.0019

Correlations computed against one-step-ahead log mid-price returns on the training split after skipping the first 500 events. All |r| < 0.005 across both measures, indicating negligible linear and monotone dependence between individual features and the prediction target. This motivates a non-linear function approximator (the neural network policy) rather than a linear model.

H. Data Preparation Pipeline

The diagram below shows the feature pipeline design that ensures no information from the validation or test splits leaks into the fitted normalization parameters.
Source: *Author's own elaboration.*

I. Offline Feature Selection Pipeline

The feature selection procedure described in Section 4.2 operates in two stages: theory constrains the candidate pool to mechanisms grounded in market microstructure literature; IC/ICIR ranking selects among the available implementations of those mechanisms and removes redundancy via a conditional IC filter. This appendix describes the operational pipeline that implements Stage 2 of that procedure. It takes a theory-defined candidate pool as input, scores each feature against a forward return proxy, and stores the selected feature specification used by the training pipeline.

The procedure scores each candidate feature against a forward return proxy and applies a greedy redundancy filter to produce the reduced set used by the training pipeline.

Proxy target. Features are scored against the forward log-return at horizon h:

$$y_t = \log \frac{P_{t+h}}{P_t} \tag{29}$$

Scoring. The Information Coefficient (IC) of feature f_i is its Spearman rank correlation with the forward return:

$$\text{IC}_{i,t} = \rho_{\text{Sp}}\big(f_{i,t},\, y_t\big) \tag{30}$$

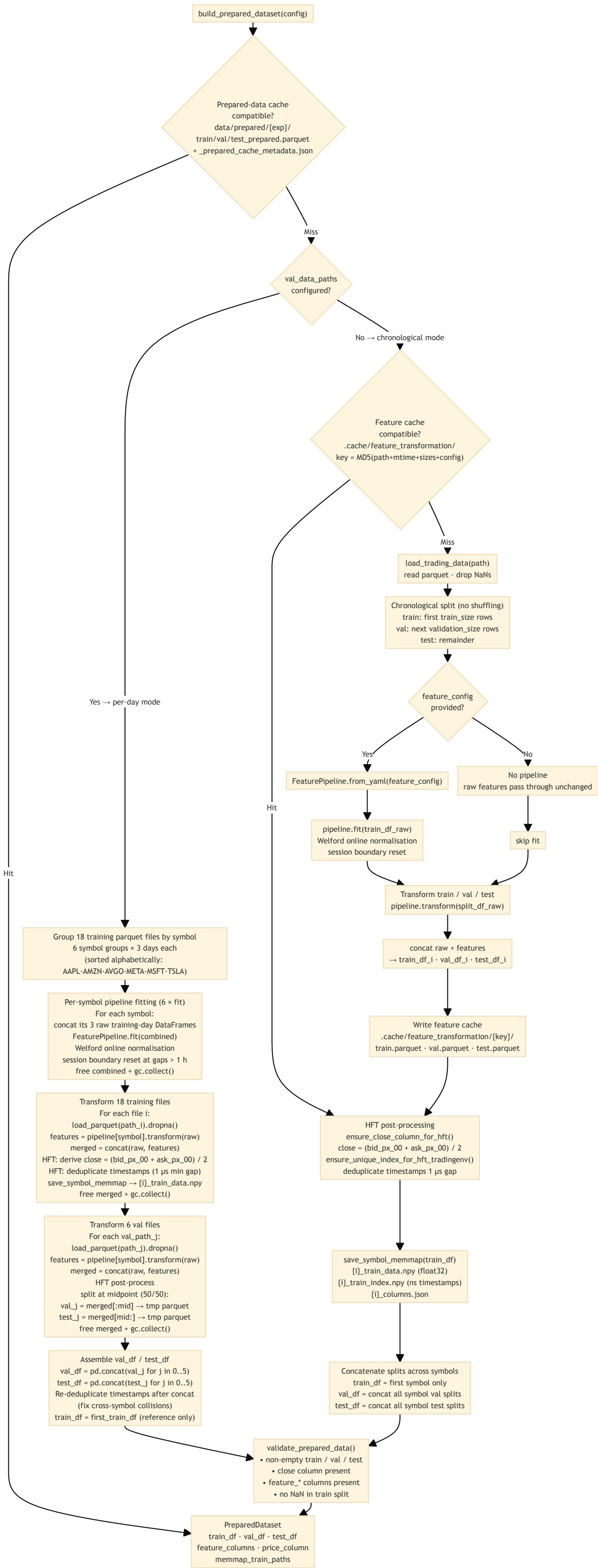
computed over rolling windows on the validation split. Features are ranked by the IC Information Ratio (ICIR):

$$\text{ICIR}_i = \frac{\overline{\text{IC}}_i}{\sigma(\text{IC}_i)} \tag{31}$$

which jointly rewards signal strength and consistency; features with $|\text{ICIR}_i| < 0.02$ are dropped.

Redundancy filter. Features are selected greedily in ICIR order. Each candidate is included only if its ICIR on residuals — after regressing out already-selected features — remains above the

Figure 3. Data preparation pipeline: fitting normalization on the training split only, then transforming train/val/test to prevent leakage.

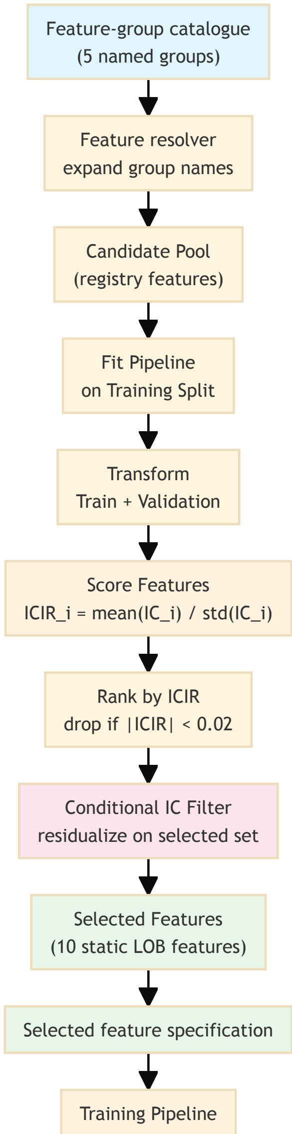


threshold. This conditional IC criterion is more principled than pairwise correlation thresholding: two correlated features can both be retained if each explains a distinct component of the forward return.

Output. The selected feature specification is stored and read by the training pipeline at runtime; no selection logic runs during training.

The diagram below summarizes the pipeline.

Figure 4. Offline feature selection pipeline: score candidates by ICIR, apply a conditional-IC redundancy filter, store the selected specification for training.



Source: *Author's own elaboration.*

The offline nature of this pipeline is deliberate: the proxy target and scoring metrics are not the reward function that the agent optimizes during training. Running selection as a separate step ensures that the agent's training loop remains uncontaminated by any pre-training statistical analysis that could introduce look-ahead bias or circular reasoning.

J. Experimental Pipeline Architecture

The figure below shows the six major stages of the experimental pipeline at a glance. Each stage is described in the corresponding chapter; this overview is intended as a navigation aid.
Source: *Author's own elaboration.*

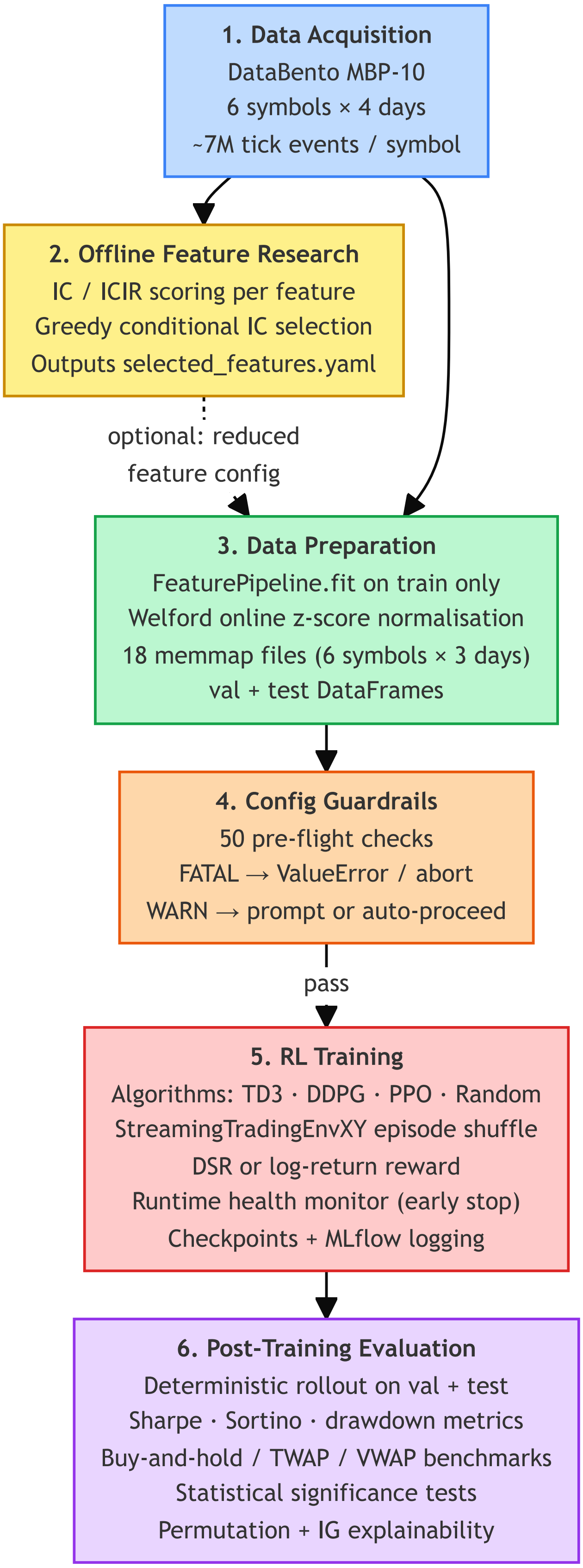
The full machine-readable diagram — including CLI entry points, per-step data flows, all configuration guardrail checks, environment variants, MLflow logging, and asset provenance metadata — is maintained as a D2 source file at `docs/workflow.d2` in the project repository (Wojdalski, 2026). It is too information-dense to reproduce at print resolution; the SVG rendering is available at `docs/workflow.svg`.

Glossary of Abbreviations and Key Terms

This glossary defines abbreviations and domain-specific terms used throughout the thesis. Entries are restricted to terms whose meaning in this context differs from everyday usage or whose precise definition is necessary to interpret the empirical results; widely understood concepts are omitted. Terms are grouped by domain — market microstructure and trading, reinforcement learning and machine learning, and other — and listed alphabetically within each group. For mathematical notation and feature formulas, see Appendix A. The glossary is intentionally selective. Not every technical term that appears in the text receives an entry; terms with widely accepted, unambiguous definitions — standard statistical measures, common financial concepts, and general machine-learning terminology — are omitted where their inclusion would expand the section beyond its practical purpose.

J.0.1. Market Microstructure and Trading

Figure 5. The six major stages of the experimental pipeline, from data acquisition through post-training evaluation.



Term	Definition
Adverse selection	The risk that a counterparty in a trade possesses superior private information, causing uninformed participants (e.g., market makers) to trade at a disadvantage.
Ask slope	Price elasticity of the ask side of the order book across multiple depth levels; a proxy for market impact per unit of sell-side volume.
Backtest	Historical simulation of a trading strategy using past market data to assess performance before live deployment. All performance figures reported in this thesis are based on backtests conducted on held-out out-of-sample data.
Basis points (bps)	One hundredth of a percentage point (0.01%). Used to express spreads and returns at fine granularity.
Bid-ask spread	The difference between the lowest ask price and the highest bid price in the order book; the primary observable measure of transaction cost and adverse selection risk.
Bid slope	Price elasticity of the bid side of the order book across multiple depth levels; a proxy for market impact per unit of buy-side volume.
Book convexity	A measure of curvature in the price schedule of the order book, capturing non-linear spacing between price levels on each side.
CumDelta (Cumulative Delta)	Signed trade flow aggregated over a rolling window; the net difference between buyer-initiated and seller-initiated executed volume.
Depth imbalance	Asymmetry in resting volume between the bid and ask sides of the order book at one or more price levels.
ETF	Exchange-Traded Fund. A publicly traded fund that tracks an index, sector, or asset class.

Term	Definition
HFT	High-Frequency Trading. Algorithmic trading operating at tick or sub-second timescales, relying on order-book data and microstructure signals rather than price bars.
Inter-event time	The elapsed time between consecutive limit order book updates; a proxy for market activity intensity.
Kyle's lambda (λ)	A measure of price impact (Kyle, 1985): the sensitivity of price changes to net order flow. Higher λ implies lower market liquidity.
LOB	Limit Order Book. The electronic record of all outstanding buy (bid) and sell (ask) limit orders at each price level, maintained by an exchange.
Market maker	A liquidity provider that continuously posts limit orders on both sides of the book, profiting from the bid-ask spread while bearing inventory and adverse selection risk.
MFT	Medium-Frequency Trading. Trading strategies operating at intraday bar (minute to hour) timescales, as distinct from high-frequency tick-level strategies.
Microprice	A volume-weighted estimate of the next transaction price, derived from top-of-book bid and ask quantities (Stoikov, 2018). Adjusts the mid-price for queue imbalance.
MicroDiv (Microprice Divergence)	The difference between the microprice and the mid-price; a stationary signal capturing the directional imbalance embedded in the top-of-book queues.
Mid-price	The arithmetic average of the best bid and best ask prices; commonly used as a reference fair value when the spread is symmetric.

Term	Definition
NLV (Net Liquidation Value)	The current market value of all open positions plus available cash in a trading account. Used in this thesis as the basis for computing true portfolio returns during evaluation, as opposed to per-step reward signals.
OBI (Order Book Imbalance)	Normalized difference in resting volume between bid and ask sides, aggregated across multiple depth levels.
OCI (Order Count Imbalance)	Normalized difference in the number of resting orders between bid and ask sides at a given price level; sensitive to order fragmentation masked by volume-based measures.
OFI (Order Flow Imbalance)	Net directional pressure exerted on the best bid and ask queues by limit order arrivals, cancellations, and executions (Cont et al., 2014). The empirically dominant contemporaneous predictor of short-term price changes.
OHLCV	Open, High, Low, Close, Volume. The standard aggregated price bar representation used in low- and medium-frequency trading; does not capture intra-bar queue dynamics.
Odd-lot	A trade smaller than the standard round lot size (typically below 100 shares). Predominantly reflects retail and algorithmic small-lot activity.
Queue depletion	The rate at which the best-bid or best-ask queue is consumed by incoming market orders across consecutive snapshots; a leading indicator of imminent price movement.
RSI	Relative Strength Index. A momentum oscillator measuring the speed and magnitude of price changes; used in rule-based and technical trading strategies.
Spread ratio	The current bid-ask spread divided by its recent rolling mean; a regime signal separating absolute spread level from elevated adverse selection conditions.

Term	Definition
TWAP	Time-Weighted Average Price. An execution benchmark computed as the mean transaction price over a fixed time window, weighting each trade equally in time. Reported alongside VWAP in the performance comparison tables.
VIX	CBOE Volatility Index. A measure of expected 30-day S&P 500 volatility derived from options prices; a common proxy for broad market fear or uncertainty.
VWAP	Volume-Weighted Average Price. The average price of an asset weighted by traded volume over a period; a common execution benchmark. Computed retrospectively over the evaluation window, so the reported VWAP rows are reference values rather than attainable returns (see Section 6.1.1).

J.0.2. Reinforcement Learning and Machine Learning

Term	Definition
Actor-critic	A class of reinforcement learning architectures combining a policy network (actor) that selects actions with a value network (critic) that evaluates them. Reduces gradient variance relative to pure policy gradient methods.
Bellman equation	The recursive identity expressing the value of a state as immediate reward plus the discounted value of the successor state: $V(s) = r + \gamma V(s')$. The theoretical foundation of temporal difference learning and Q-learning.
Bootstrapping	Updating a value estimate using another estimate as the target, rather than waiting for a complete episode return. Introduces bias but reduces variance; the mechanism underlying all temporal difference algorithms.

Term	Definition
DDPG	Deep Deterministic Policy Gradient (Lillicrap et al., 2016). An off-policy actor-critic algorithm for continuous action spaces using deterministic policies and experience replay. Predecessor to TD3.
DQN	Deep Q-Network (Mnih et al., 2015). A value-based deep RL algorithm combining Q-learning with a deep neural network, experience replay, and a frozen target network. Restricted to discrete action spaces.
DSR	Differential Sharpe Ratio (Moody & Saffell, 1998). A reward formulation that approximates the gradient of the Sharpe Ratio with respect to returns, aligning the optimization objective with risk-adjusted performance.
Exploration-exploitation tradeoff	The tension in reinforcement learning between taking actions that yield known rewards (exploitation) and trying new actions to discover potentially better outcomes (exploration). Particularly costly in trading due to transaction costs.
MDP	Markov Decision Process. The formal framework for sequential decision-making under uncertainty, consisting of states, actions, a transition function, and a reward function. Assumes that future states depend only on the current state and action, not on history.
Off-policy	A class of RL algorithms that learn a target policy from data collected by a different behavior policy. Enables experience replay and reuse of historical data. TD3 is off-policy.
On-policy	A class of RL algorithms that must learn from data collected by the same policy being improved. More stable but less sample-efficient than off-policy methods.

Term	Definition
PCA	Principal Component Analysis. A dimensionality reduction technique that projects data onto orthogonal axes of maximum variance.
Policy gradient	A family of RL algorithms that directly optimize the policy parameters through gradient ascent on expected cumulative reward. Handles continuous action spaces naturally.
PPO	Proximal Policy Optimization. An on-policy actor-critic algorithm that constrains policy updates to a trust region to improve stability.
Q-learning	A model-free, off-policy value-based RL algorithm that learns the action-value function $Q(s, a)$ through temporal difference updates (Watkins & Dayan, 1992).
Replay buffer	A data structure storing past environment transitions (s_t, a_t, r_t, s_{t+1}) for reuse in off-policy training. Decorrelates training samples and improves sample efficiency.
REINFORCE	A Monte Carlo policy gradient algorithm (Williams, 1992) that updates policy parameters using full-episode return estimates. High variance; foundational to modern policy gradient methods.
RL	Reinforcement Learning. A machine learning paradigm in which an agent learns to make sequential decisions by interacting with an environment and maximizing cumulative reward.
SAC	Soft Actor-Critic (Haarnoja et al., 2018). An off-policy actor-critic algorithm incorporating a maximum-entropy objective to encourage exploration through stochastic policies.
SARSA	State-Action-Reward-State-Action. An on-policy temporal difference algorithm that updates $Q(s, a)$ using the action actually taken by the current policy.

Term	Definition
Target network	A periodically or slowly updated copy of the Q-network or actor network used to generate stable training targets, preventing the oscillations that arise when both prediction and target change simultaneously. Used in DQN, DDPG, and TD3.
TD3	Twin Delayed Deep Deterministic Policy Gradient (Fujimoto et al., 2018). An off-policy actor-critic algorithm for continuous control that extends DDPG with clipped double Q-learning, delayed policy updates, and target policy smoothing to reduce overestimation bias. The primary algorithm used in this thesis.
TD error	The difference between the Bellman target $y_t = r_t + \gamma Q(s_{t+1}, a_{t+1})$ and the current value estimate $Q(s_t, a_t)$. Minimising the mean squared TD error trains the critic network in all actor-critic algorithms.
Temporal difference (TD) learning	A class of RL methods that update value estimates based on the difference between successive predictions, without requiring full episode trajectories.
Value function	A function estimating the expected cumulative discounted reward from a given state (or state-action pair) under a given policy. The critic network in actor-critic algorithms approximates this function.

J.0.3. Other

Term	Definition
AAPL	The NASDAQ ticker symbol for Apple Inc. One of six Nasdaq-listed equities — alongside MSFT, TSLA, META, AMZN, and AVGO — used as trading instruments in this thesis.
Maximum drawdown	The largest peak-to-trough decline in portfolio value over an evaluation period, expressed as a percentage. A standard tail-risk measure reported alongside the Sharpe and Sortino ratios in the performance tables.
Out-of-sample evaluation	Assessment of model performance on data not used during training or hyperparameter selection. A central validation principle of this thesis: all reported performance figures are computed on a held-out test period.
Sharpe ratio	A risk-adjusted performance measure defined as the ratio of mean excess return to return volatility. Higher values indicate better return per unit of risk.
Sortino ratio	A risk-adjusted return measure analogous to the Sharpe ratio but using downside deviation — the volatility of negative returns only — rather than total volatility in the denominator. Rewards strategies that limit losses without penalising upside volatility.

K. Asset Provenance Audit Log

This audit-only inventory lists rendered image assets, newest first. Sidecar metadata takes precedence; static assets use their latest Git revision.

Generated/modified (UTC)	Commit	Asset	Provenance source
2026-08-28 21:59:24	c393b27f	diagrams/rl-loop.svg	git history

Generated/modified (UTC)	Commit	Asset	Provenance source
2026-08-12 20:42:01	d86d0972	diagrams/fig-rl-taxonomy.svg	git history
2026-08-12 20:42:01	d86d0972	diagrams/fig-feature-selection-pipeline.svg	git history
2026-08-12 20:42:01	d86d0972	diagrams/fig-experimental-pipeline.svg	git history
2026-08-12 20:42:01	d86d0972	diagrams/fig-data-pipeline.svg	git history